

# ESN 10:

## Hardware / Software co-design on FPGAs

Kamel ABDELOUAHAB

PhD, R&D Engineer @Sma-RTy / UCA

[kamel@sma-rty.com](mailto:kamel@sma-rty.com)

V3.0 - 202309

Polytech GE5A & SETSIS M2



# Acknowledgment



This lecture is based on the works of:

- Michel JAMES (2020), “Linux embarqué sur plateforme FPGA”, Polytech Clermont-Ferrand / UCA
- François BERRY (2017), “Advanced FPGA programming: System On Programmable Chip”, UCA

Thanks to both of them ...

# Briefing

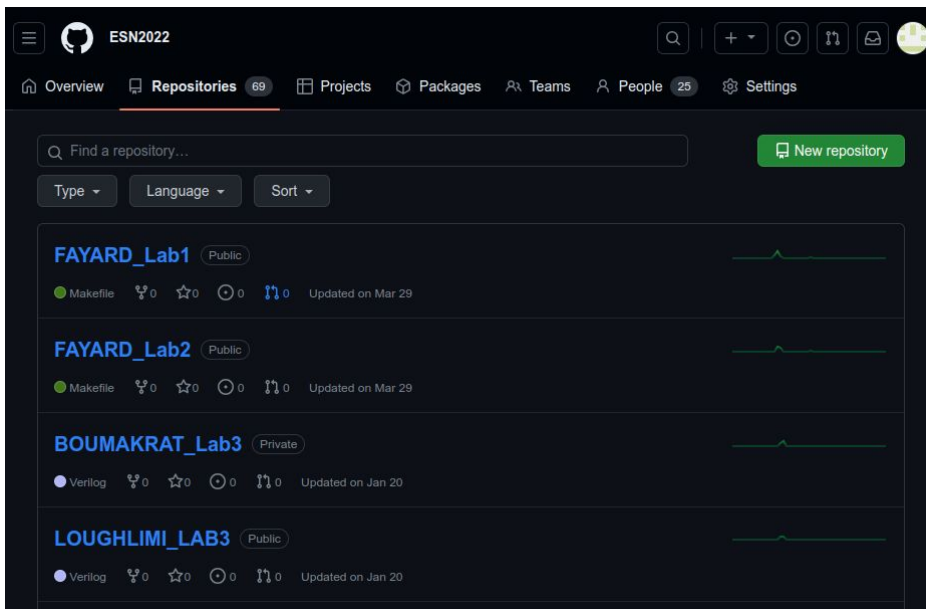
- What you will learn
  - Understand the importance of co-design
  - Improve you FPGA skills
  - Know how to partition software and hardware development
  - Implement an end-to-end SoPC system
- Course
  - ESN 10: 12 hours lecture, practical and theoretical
  - ESN 11: 12 hours of practical labs in ez mode ...
  - ESN 12: 12 hours project



# Evaluation



- ESN 10 : Paper exam (~1 hour)
- ESN 11 + ESN 12: Evaluation of your source code and project !





# Lecture outline - 1



## I. Introduction

- Co-design motivations
- Why FPGAs
- Applications

## II. FPGAs for co-design

- Heterogeneous platforms
- Typical FPGA components
- Processors in the co-design space
  - Softcores
  - HPS
  - Interrupt: Microprocessors 101

# Lecture outline - 2



## III. Conception flow

- Communication between SoPC blocks
- Workload partitioning
- Hardware workflow
- Software workflow

## IV. Useful features

- Debugging through JTAG
- Dealing with IOs
- Handling interrupts
- Timers
- HPS inference

## IV. Conclusion

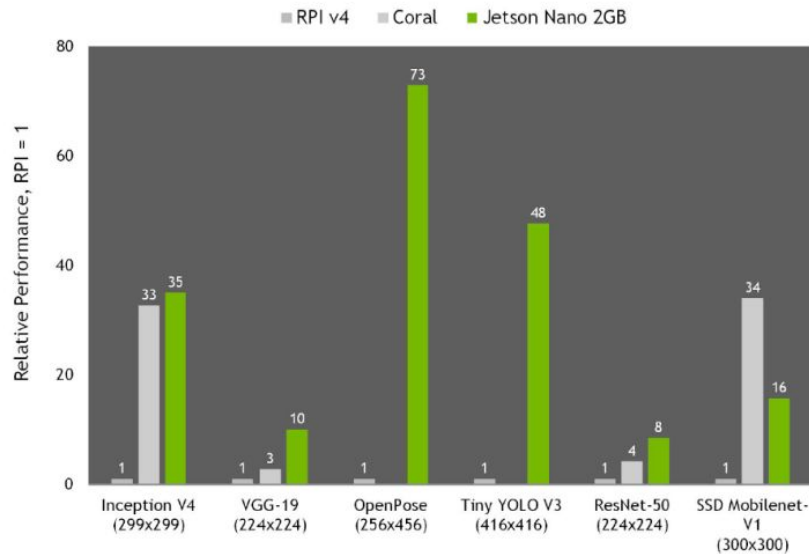


# I. Introduction

# Co-design motivations



- Dedicated hardware provides **significant acceleration to many workloads**
  - video encode/decode
  - IA inference
  - Signal processing
  - Communication protocols
- This comes at the price of high development times.



# Co-design motivations



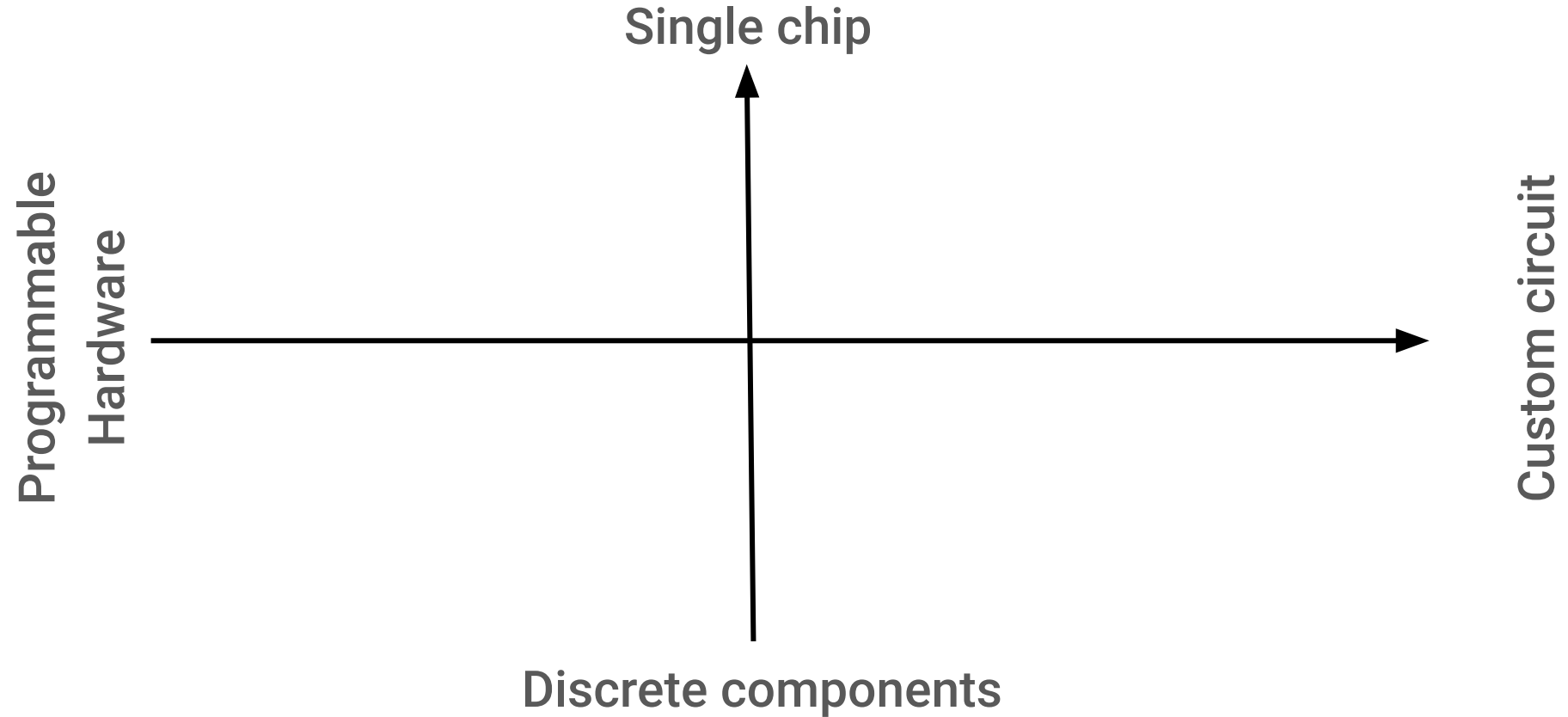
Co-design combines the best of two worlds:

- Productivity: Rapid prototyping, low time to market
- Performance: Decent perf. on embedded systems

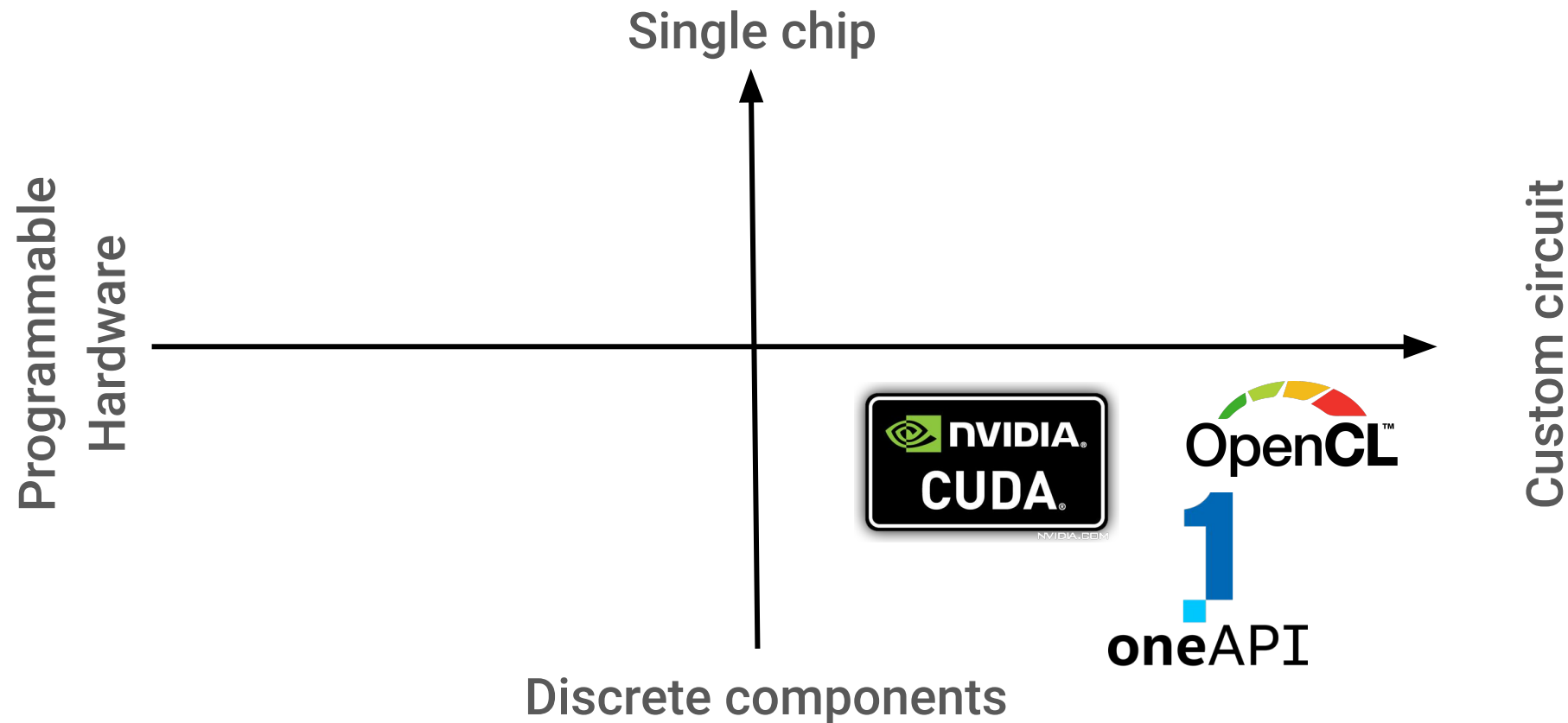
HOW ?

- Implement some tasks on dedicated HW
- Implement others tasks in SW (CPUs)

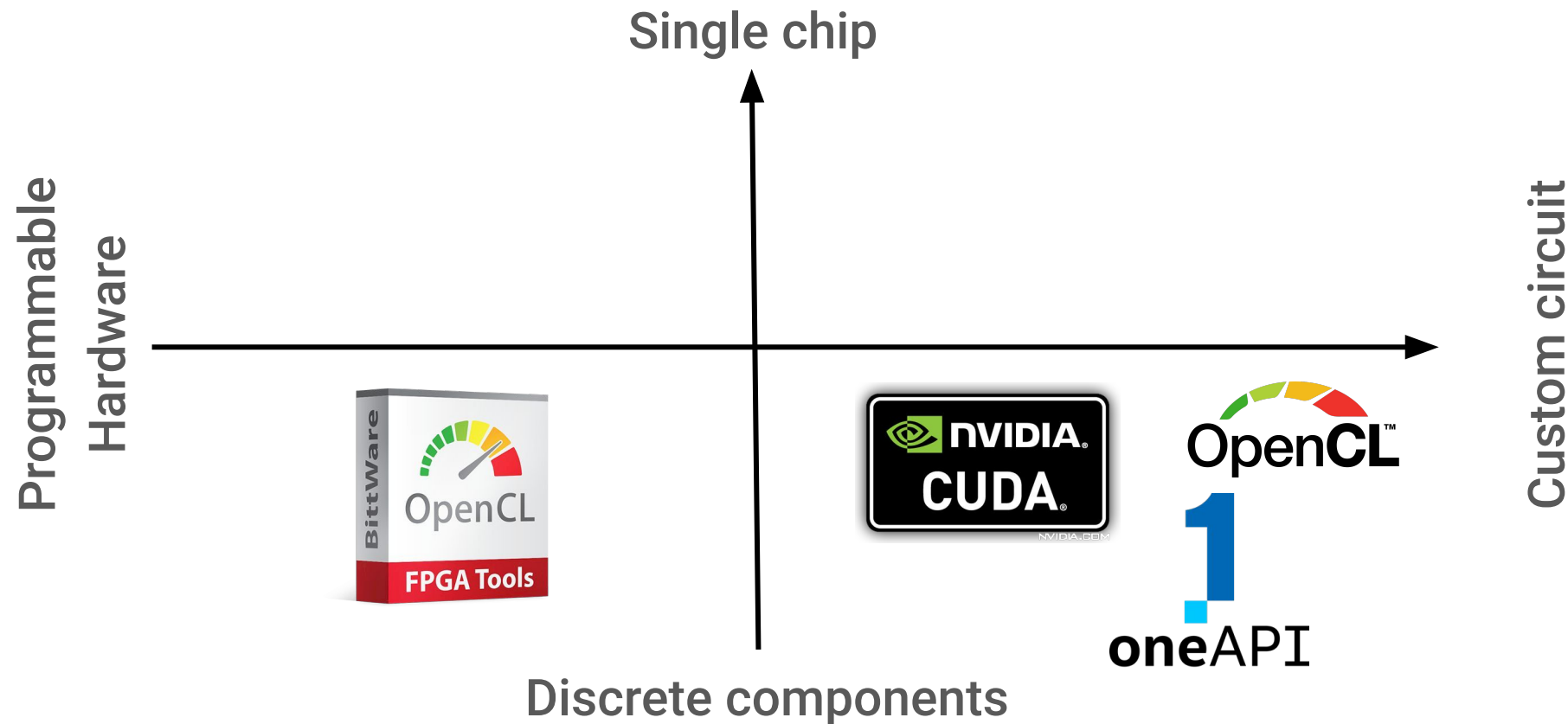
# Co-design solutions



# Co-design solutions

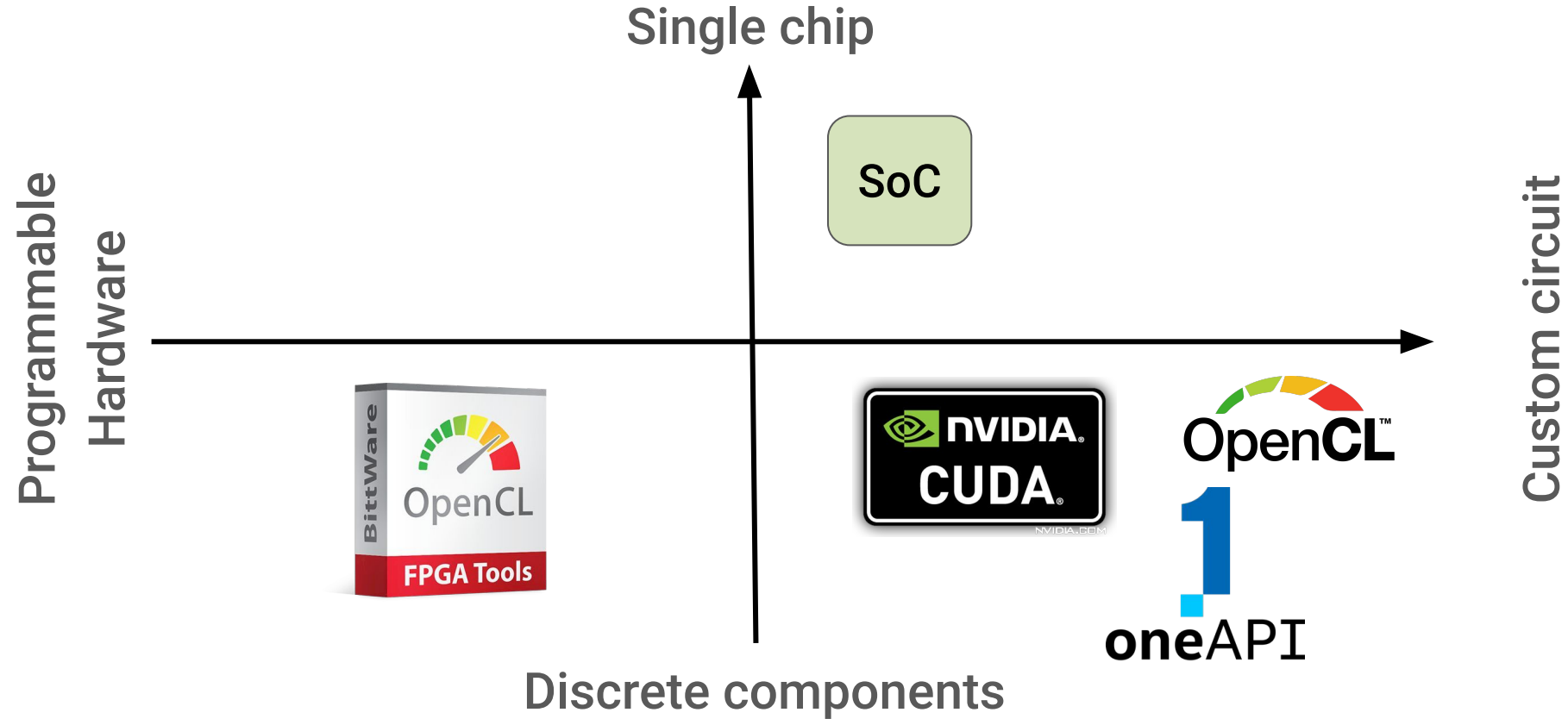


# Co-design solutions

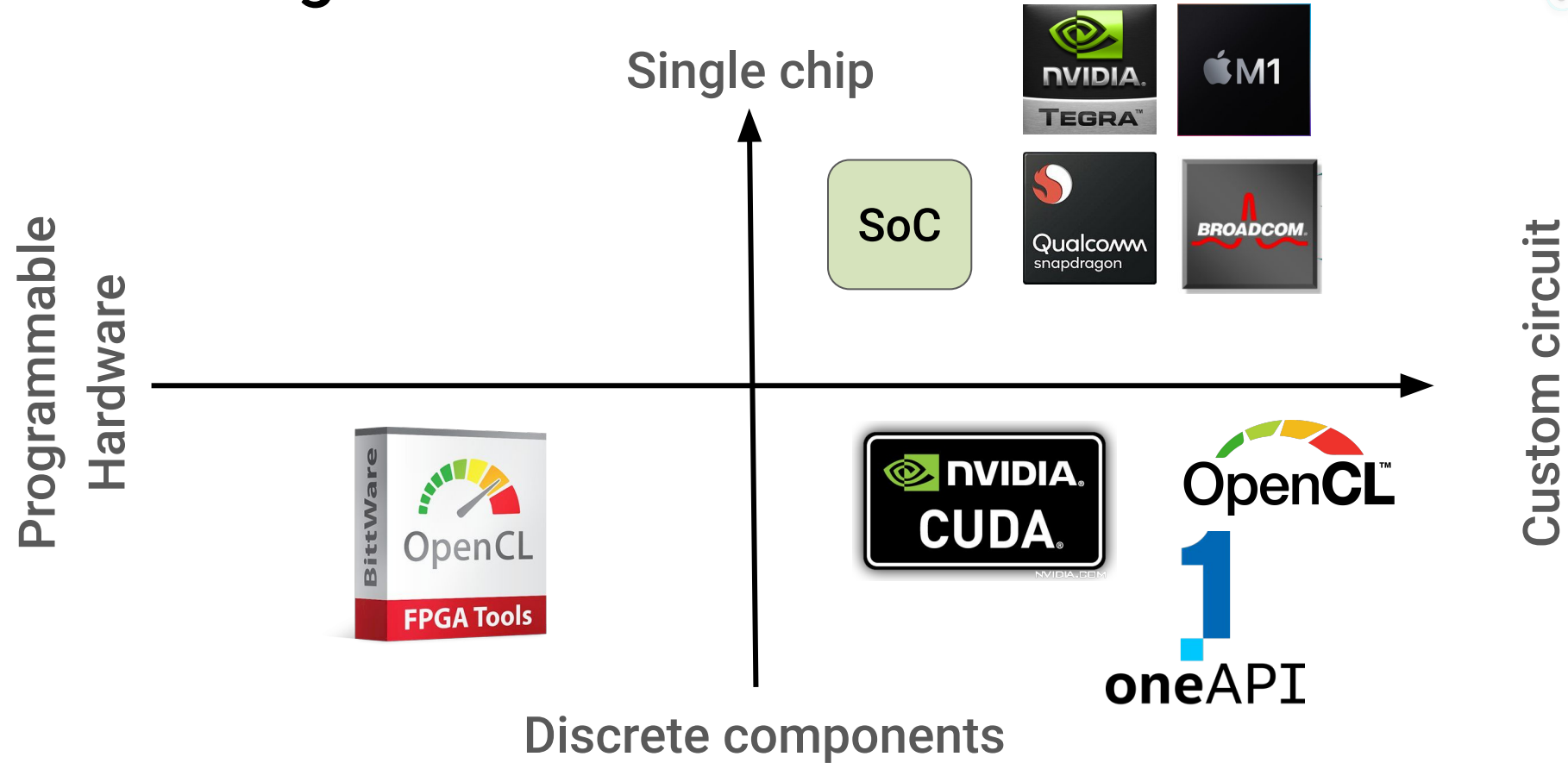




# Co-design solutions



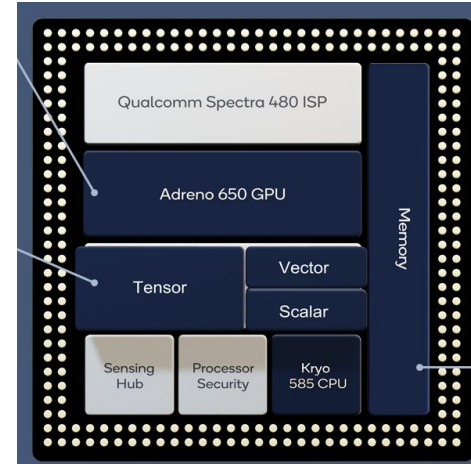
# Co-design solutions



# SoC examples



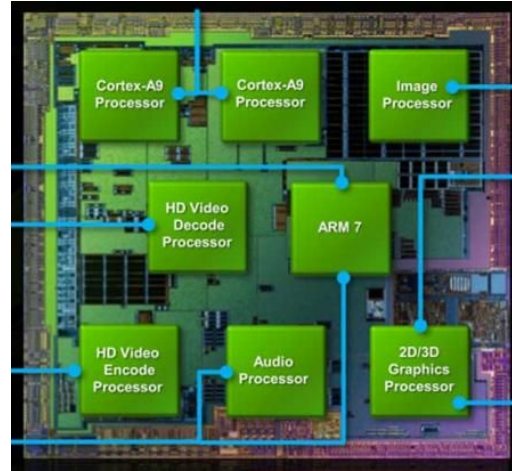
## Snapdragon 672



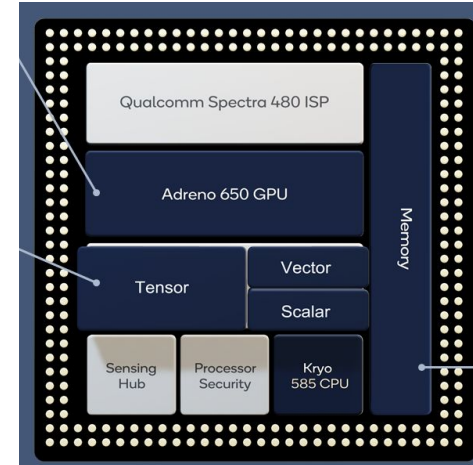
# SoC examples



Tegra K1



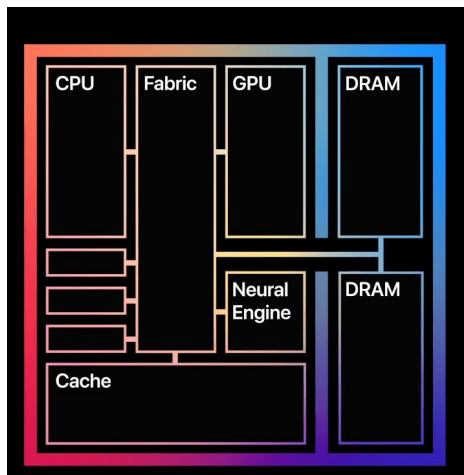
Snapdragon 672



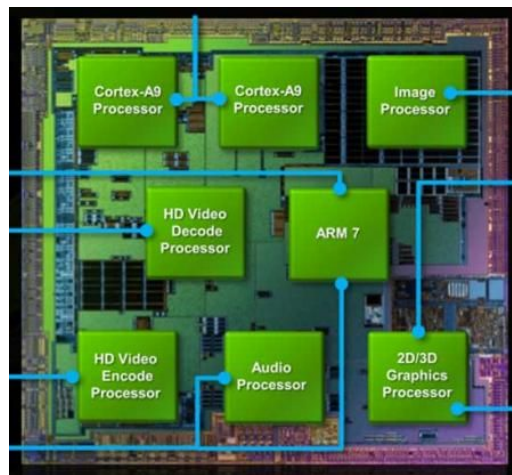
# SoC examples



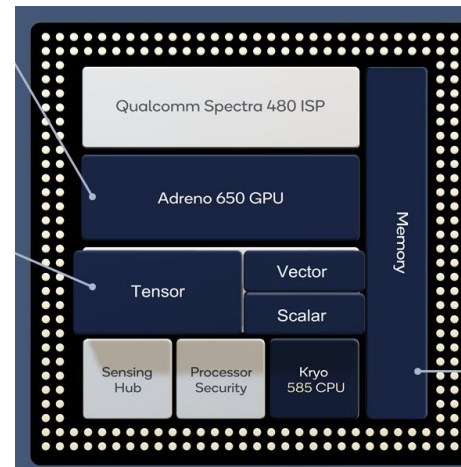
Apple M1



Tegra K1

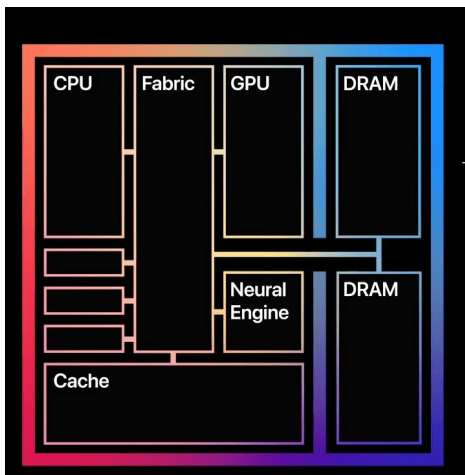


Snapdragon 672



# SoC exemples

## Apple M1



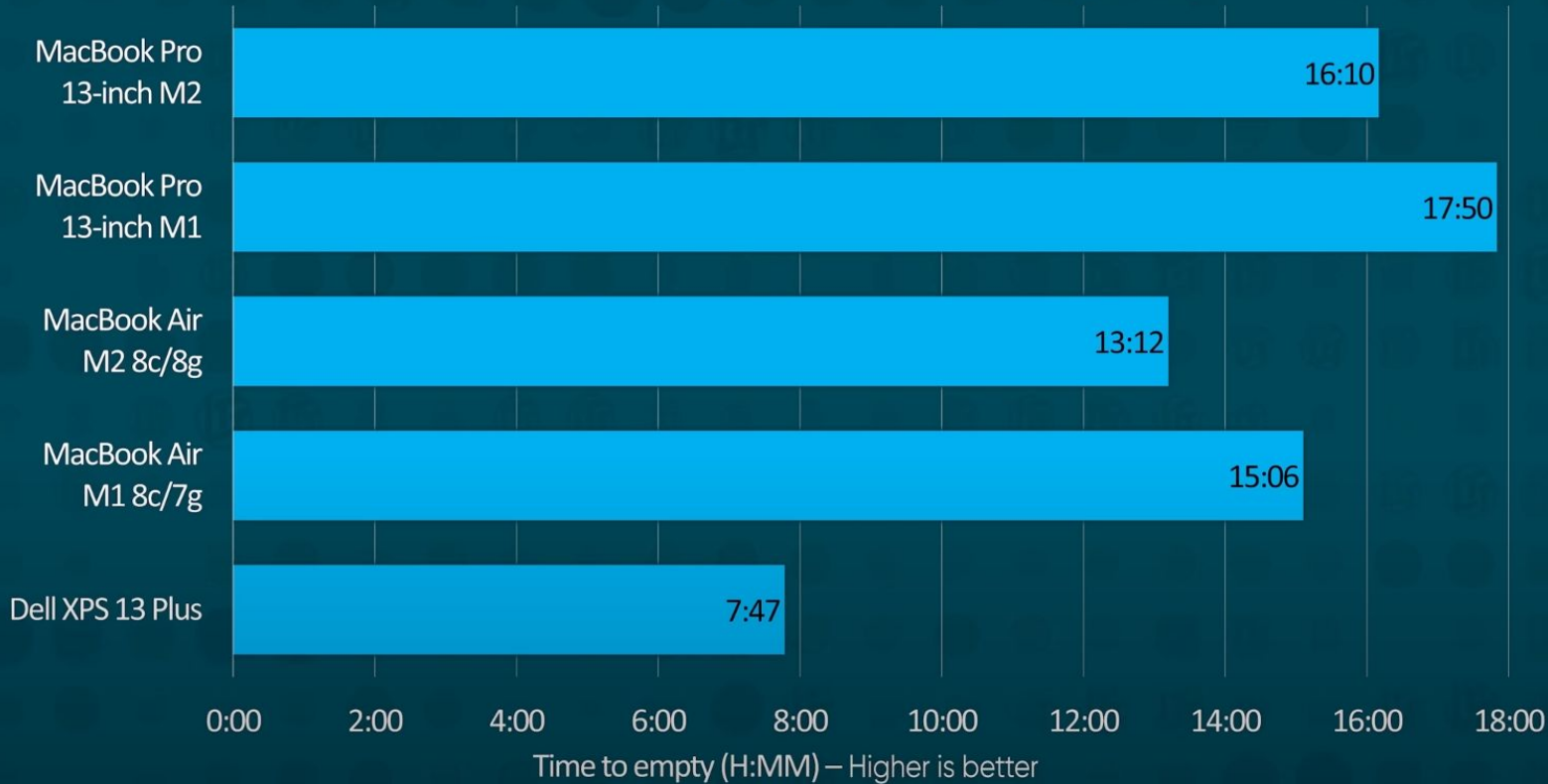
## Caractéristiques techniques [ [modifier](#) | [modifier le code](#) ]

- CPU : 8 cœurs, fréquence maximale à 3,20 GHz<sup>4</sup>
  - 4 cœurs haute performance (3,20 GHz, 192 Ko cache d'instructions, 128 Ko cache de données, 12 Mo mémoire niveau 2 partagée)
  - 4 cœurs haute efficacité (2,10 GHz, 128 Ko cache d'instructions, 64 Ko cache de données, 4 Mo mémoire niveau 2 partagée)
- GPU : 7 ou 8 cœurs
  - 128 unités d'exécution
  - Jusqu'à 8 [threads](#)
  - 2,6 téraflops (FP32)
  - 82 gigatexels par seconde
  - 41 gigapixels par seconde
- Moteur neuronal 16 cœurs, capable d'effectuer 11 000 milliards d'opérations par seconde
- Enclave sécurisée
- Processeur de signal d'image (ISP)
- Contrôleur Thunderbolt/USB 4
- Accélérateur de [Machine learning](#)
- Contrôleur de stockage SSD
- 16 milliards de transistors gravés en 5 nm
- 8 ou 16 Go de [mémoire vive](#) SDRAM [LPDDR4X](#) à 4 266 MHz<sup>5</sup>
- Moteurs d'encodage/décodage H.264/HEVC/VP9



# Battery Life

YouTube video streaming (50% MacBook Air brightness)



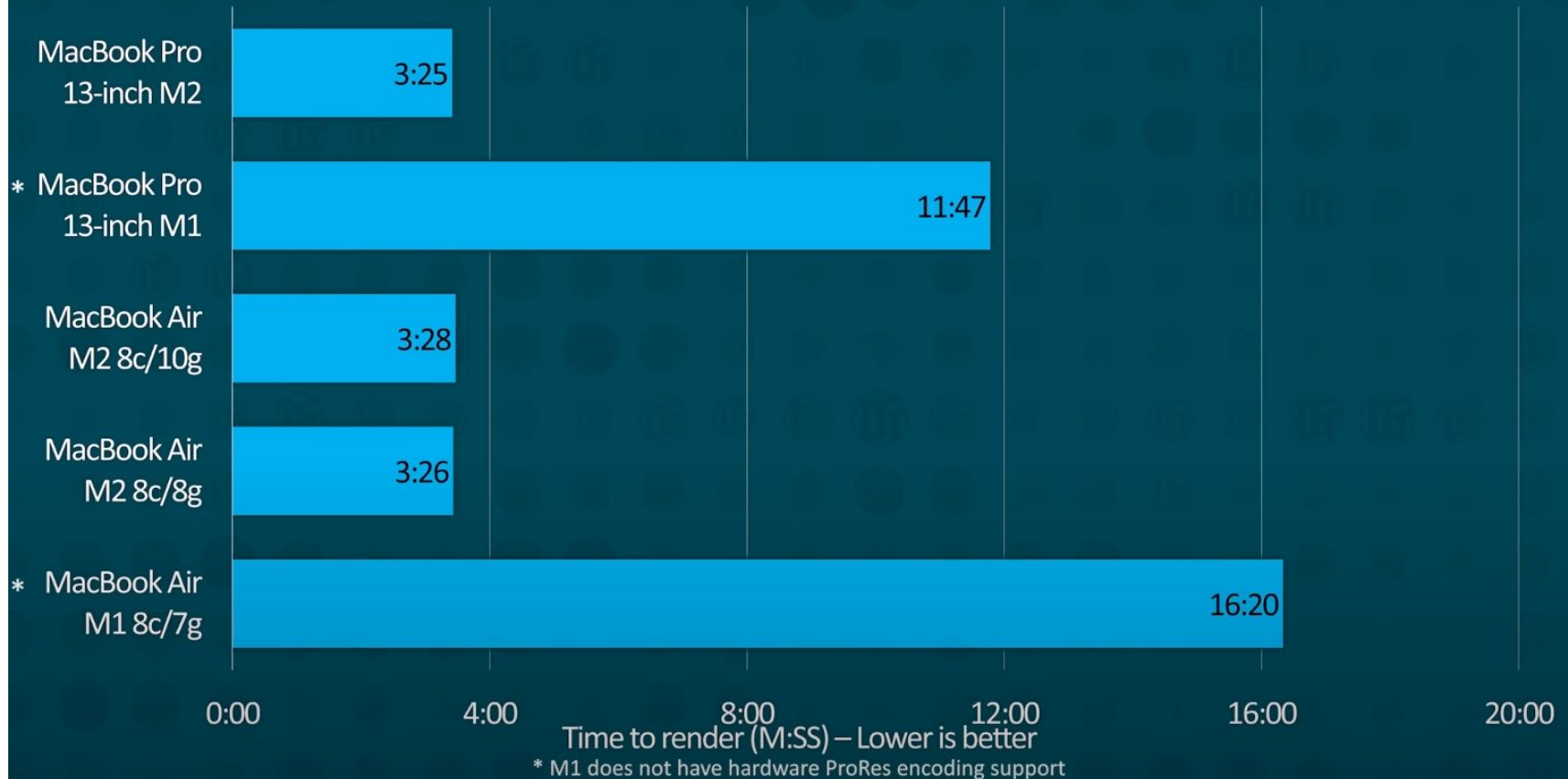
Source : LTT2022 <https://www.youtube.com/watch?v=w6GTf6ATlho>





# Compressor

## ProRes – Dolby Atmos Demo

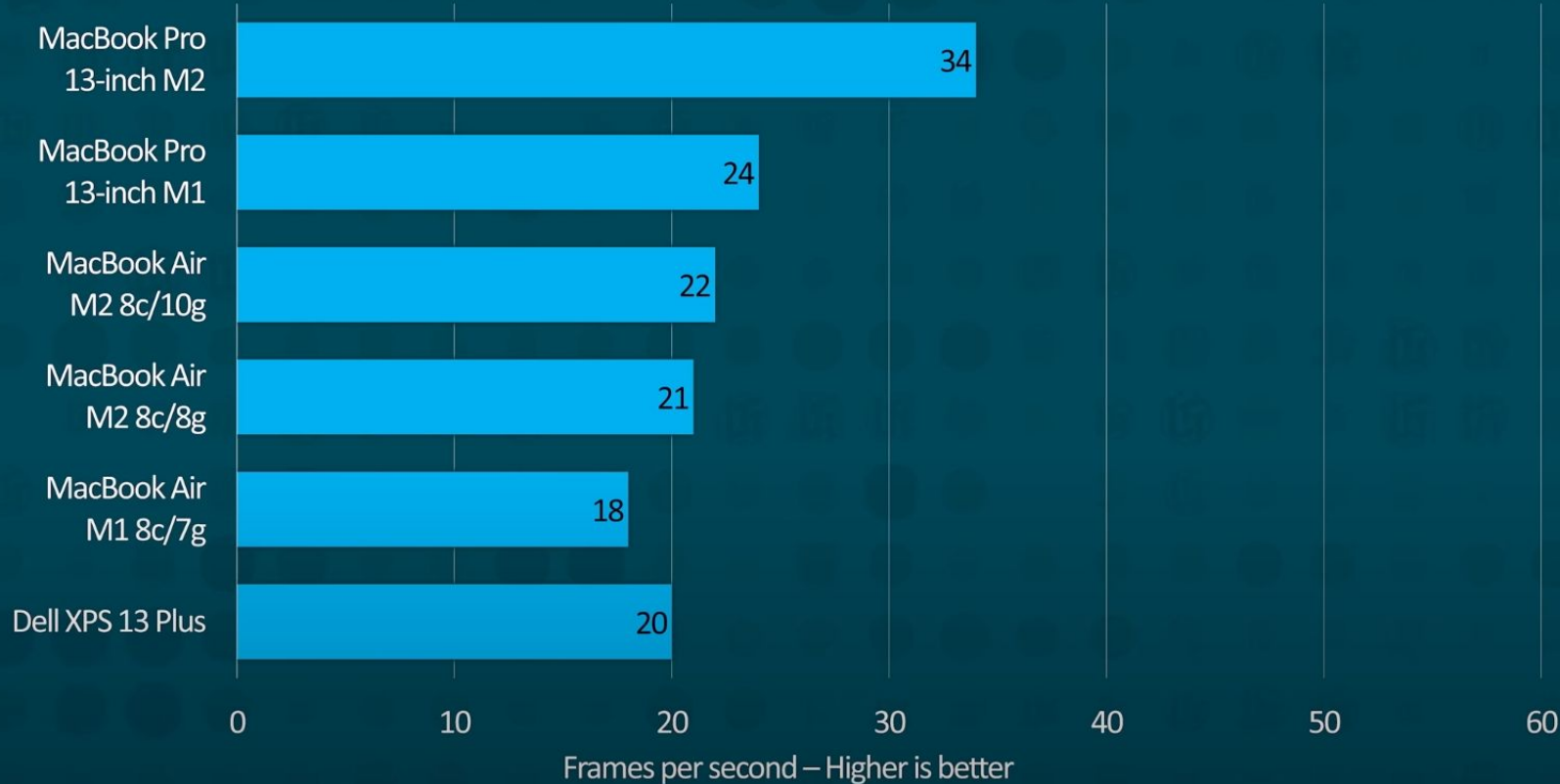






# Shadow of the Tomb Raider

1200p, Medium Preset – Data taken on 3<sup>rd</sup> consecutive run



# SoC examples

- Replacing discrete components by SoCs results in a huge efficiency gain !



# SoC examples

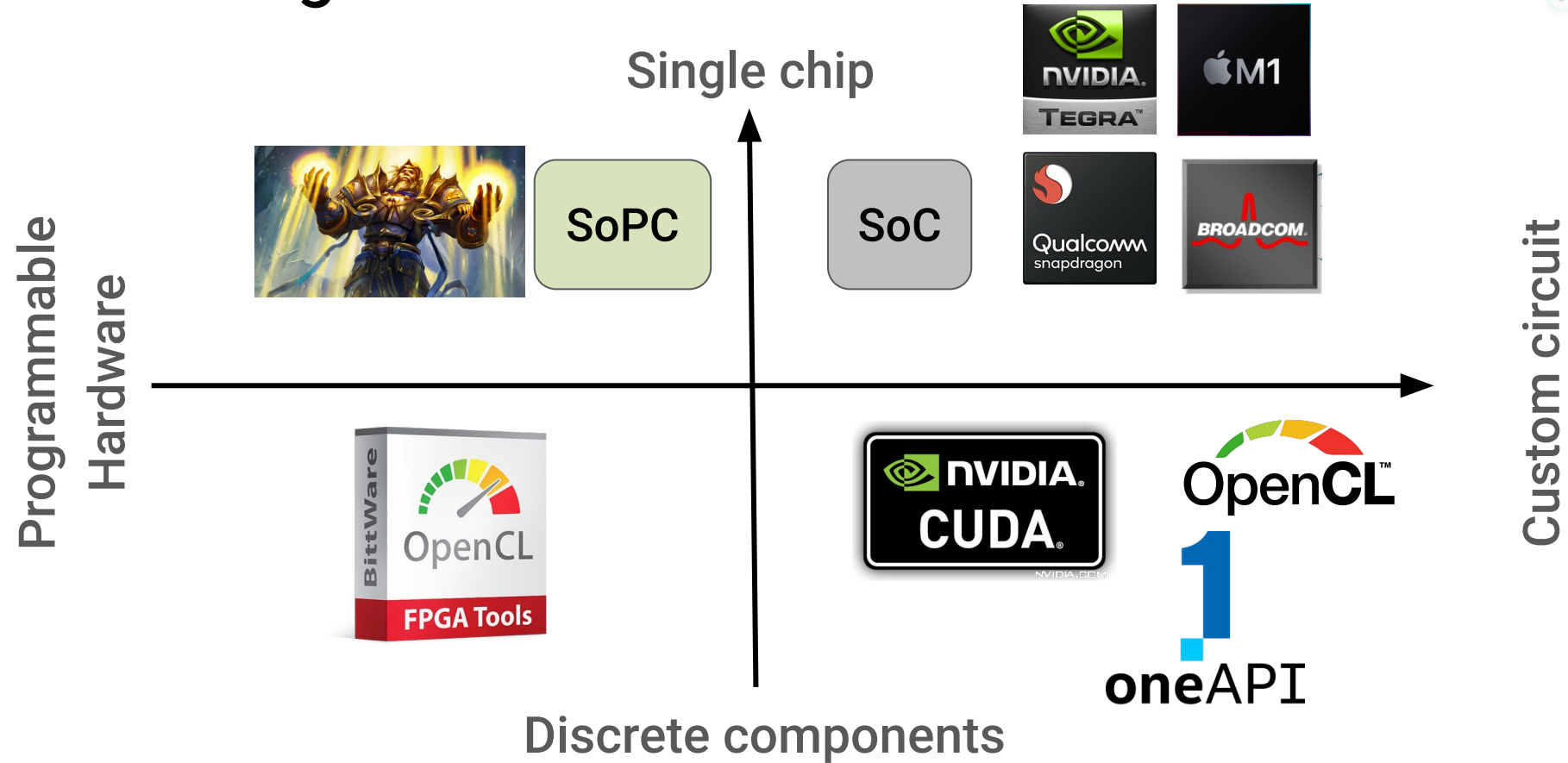
- Replacing discrete components by SoCs results in a huge efficiency gain !



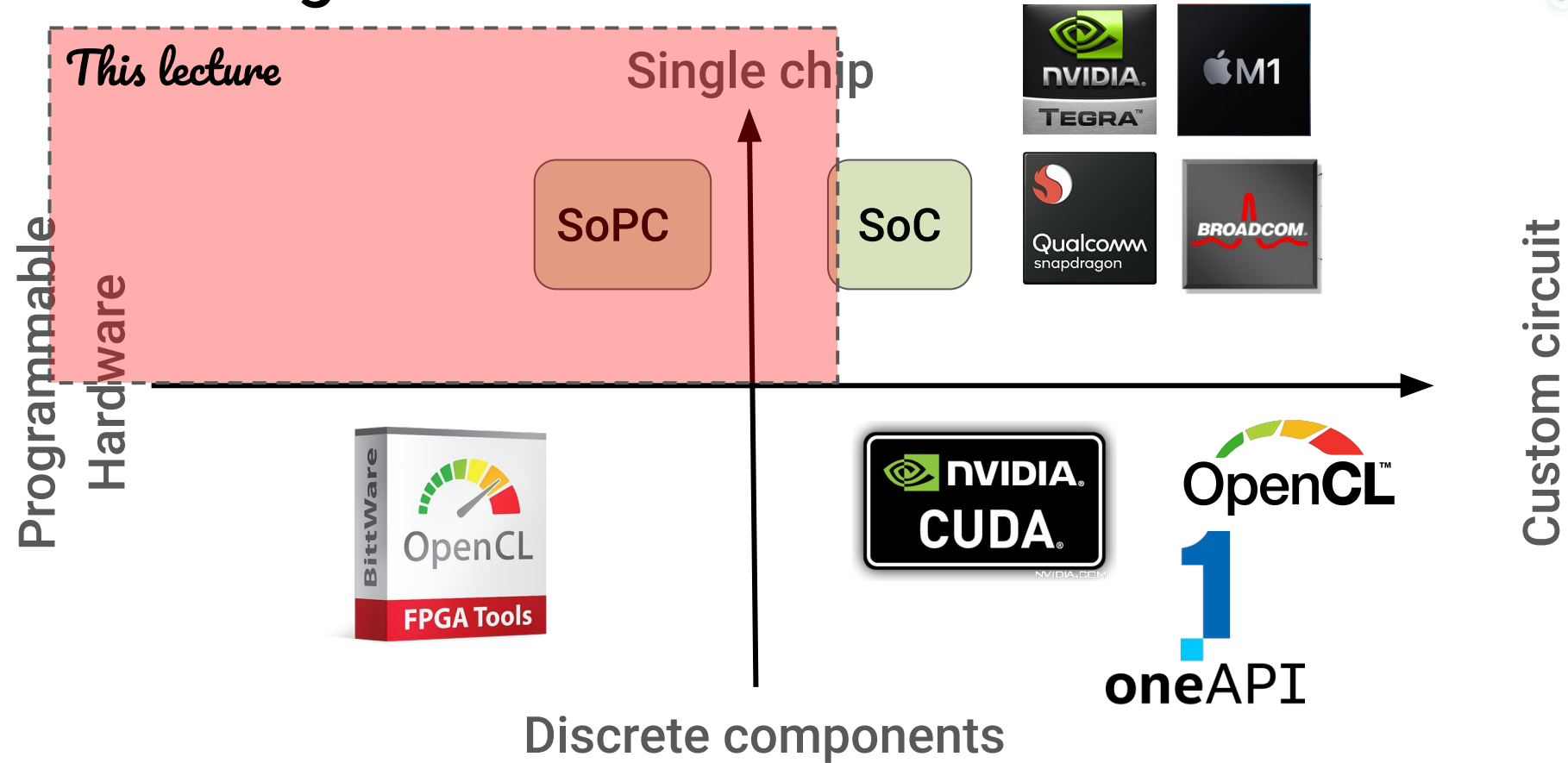
- But remember that designing a custom SoC is time and resource intensive process



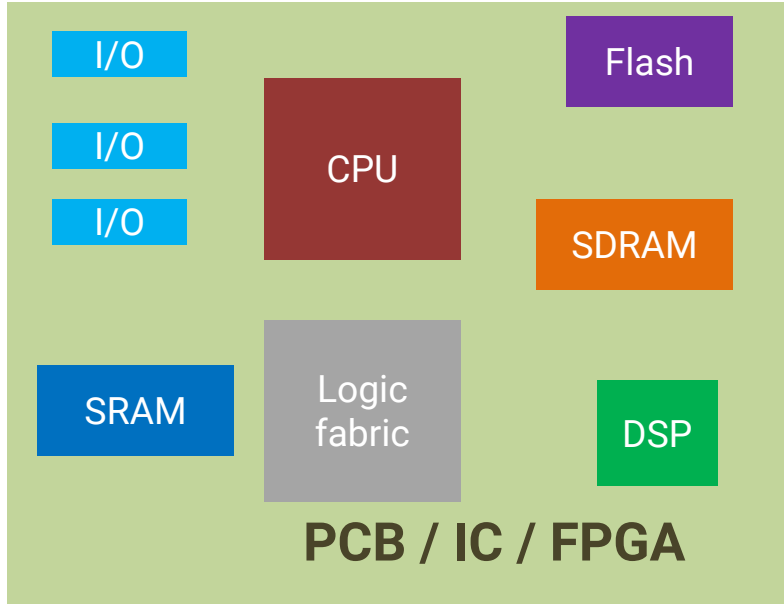
# Co-design solutions



# Co-design solutions



# Systems on PROGRAMMABLE chips



System On Chip : SoC (ASIC)

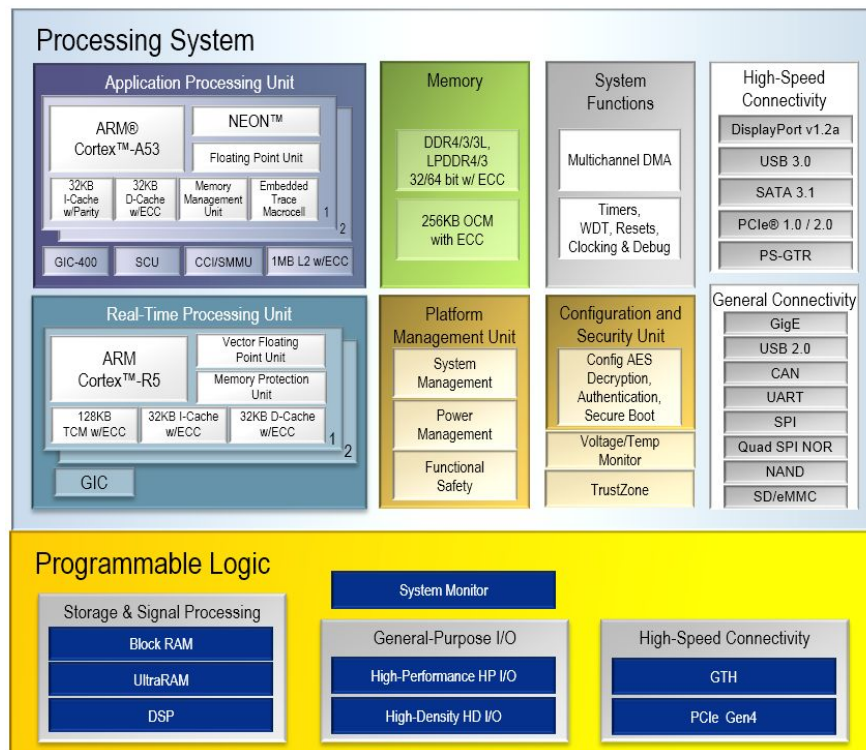


**System On Programmable Chip : SoPC (FPGA)**

# SoPC: Motivations

## Why co-designing on FPGAs ?

- **Flexibility** on the HW
- **Heterogeneously** of current platforms



# SoPC: Motivations



In practice : SoPC → productivity !

**time to 'market** noun [uncountable]

the time it takes to design and make a new product and make it available to buy

The electronics industry is looking for ways to accelerate time to market.

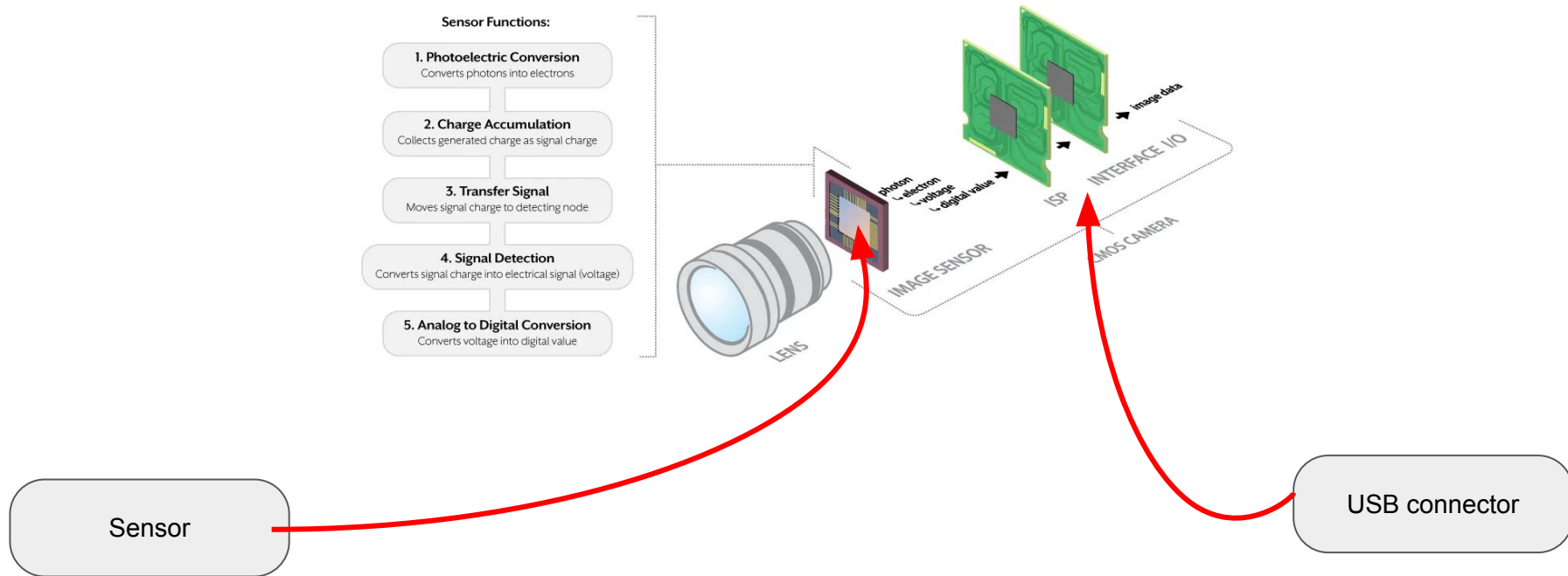
A full HW system would typically have better performance when compared to a SW/HW system, but the productivity gain is worth



# Application and examples of FPGA co-design



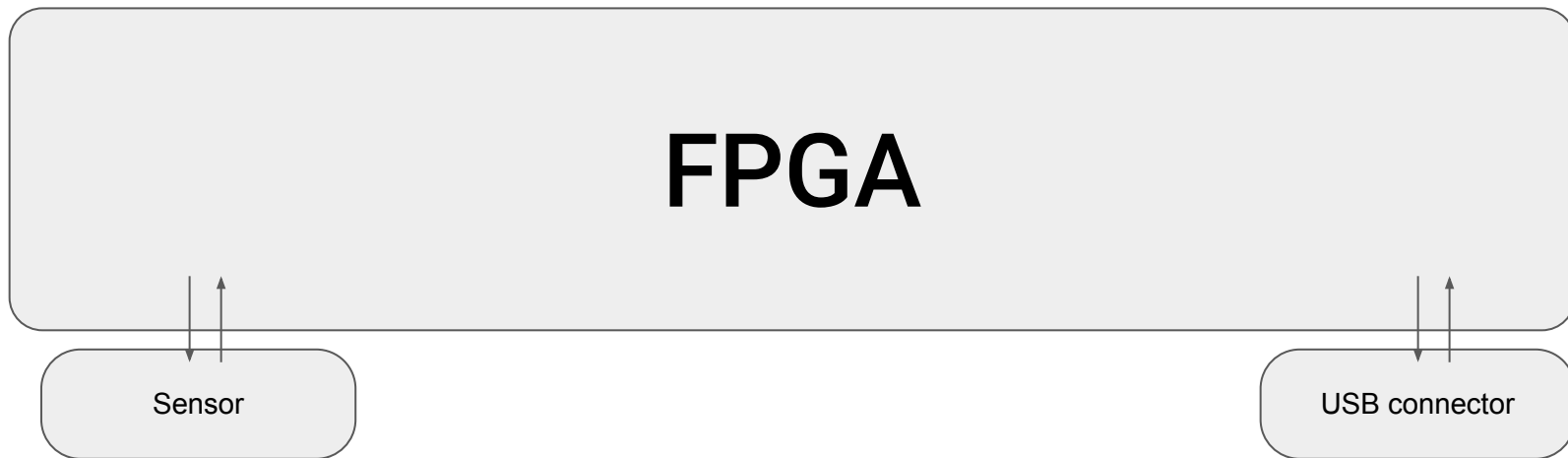
## Image sensor integration: fast prototyping



# Application and examples of FPGA co-design



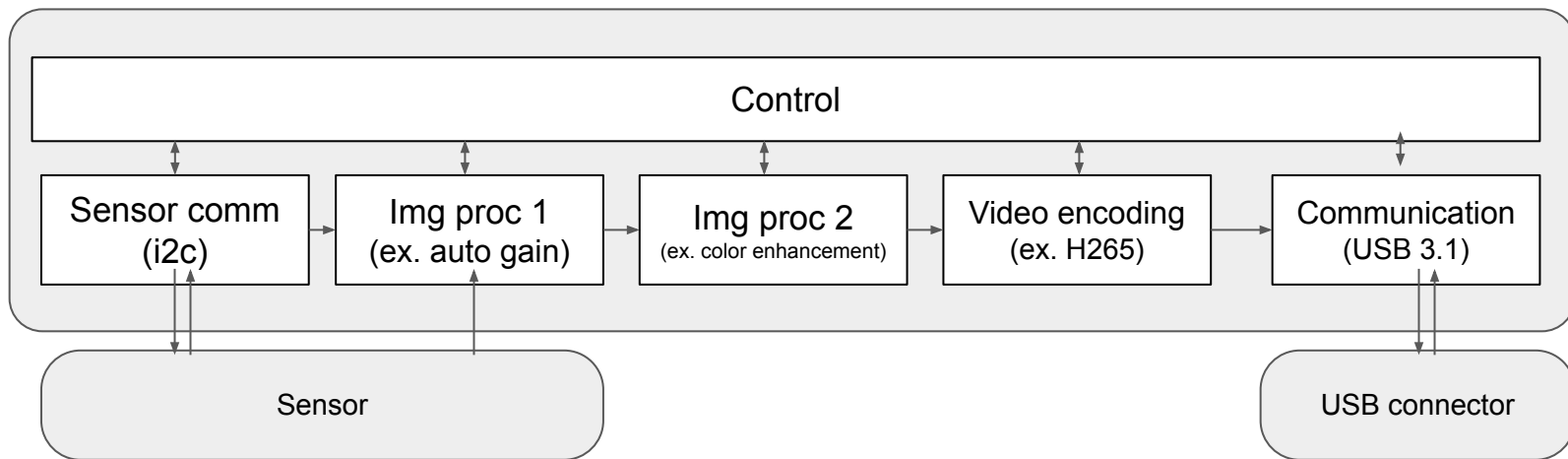
Image sensor integration: fast prototyping



# Application and examples of FPGA co-design



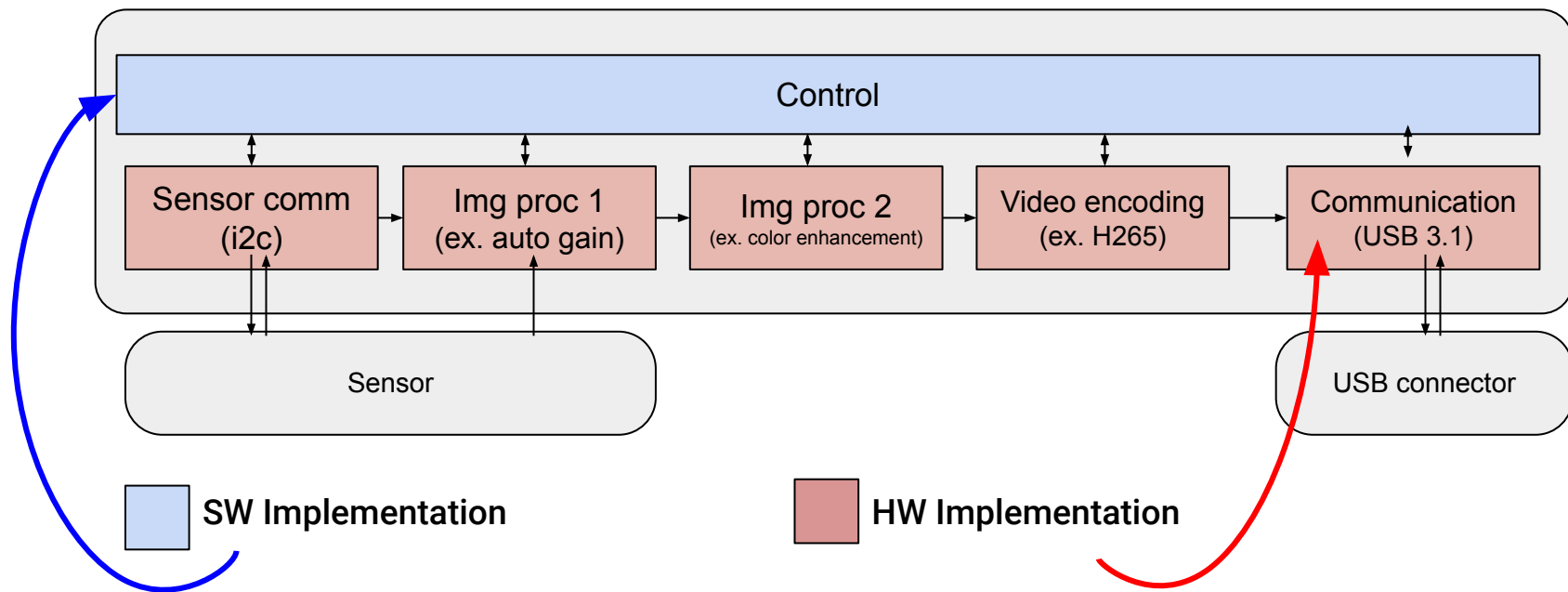
## Image sensor integration: fast prototyping



# Application and examples of FPGA co-design

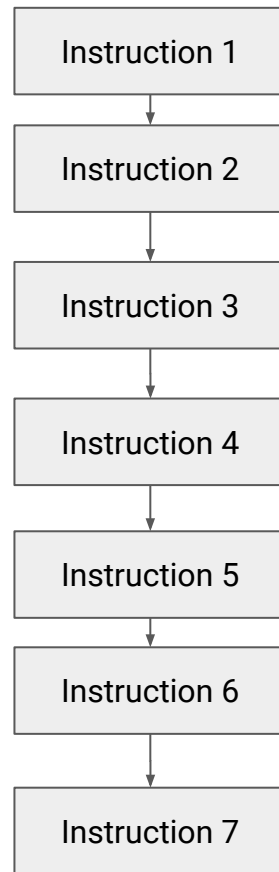


## Image sensor integration: fast prototyping



# Application and examples of FPGA co-design

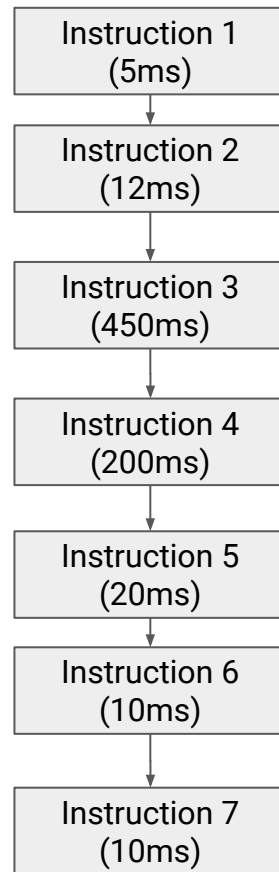
## Hardware acceleration in HPC



# Application and examples of FPGA co-design



- Hardware acceleration in HPC
- Profiling ...

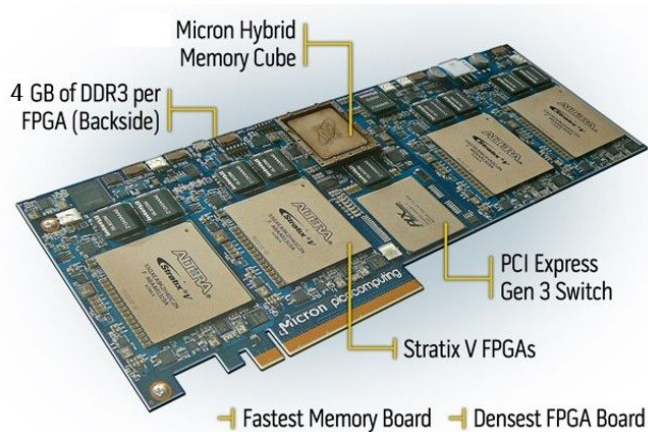


# Application and examples of FPGA co-design



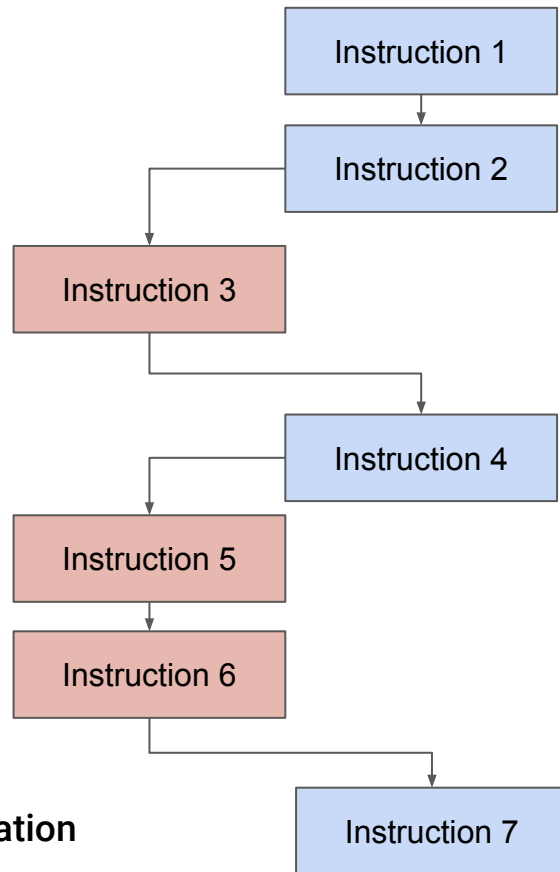
Hardware acceleration in HPC

Profile → Accelerate



 SW Implementation

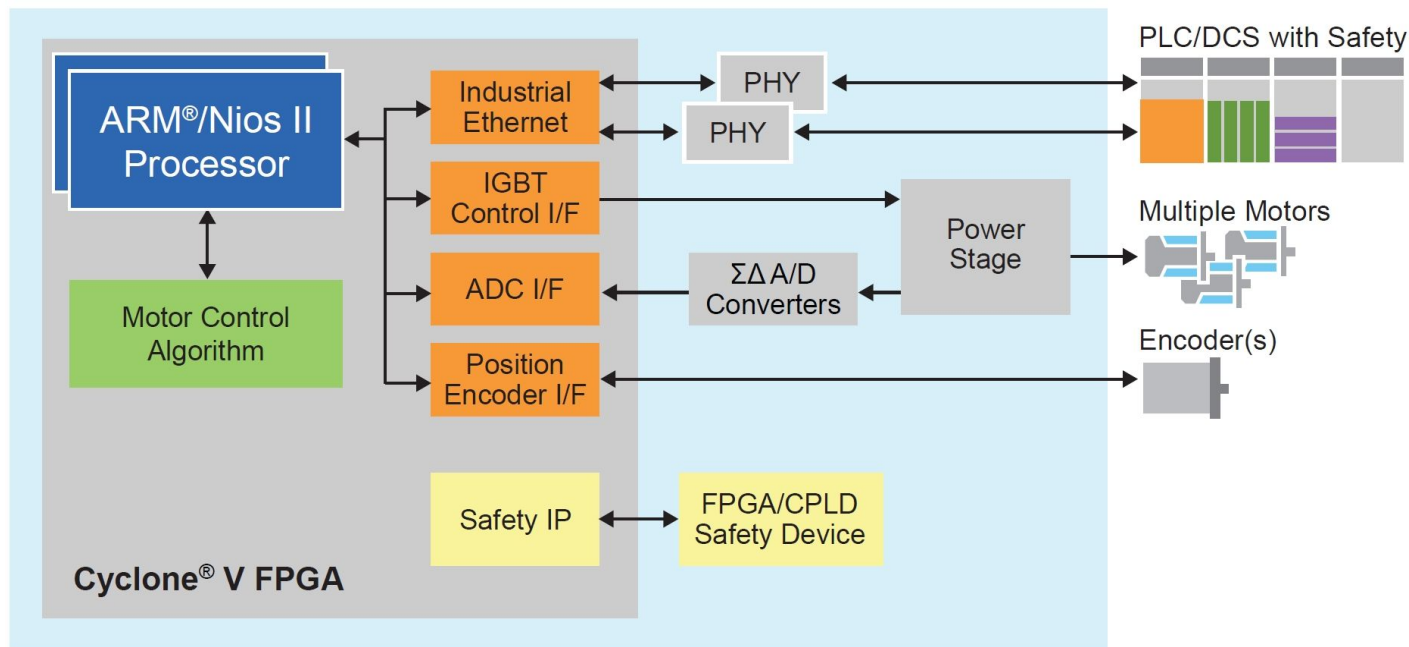
 HW Implementation



# Application and examples of FPGA co-design



## Rapid prototyping







## II. FPGAs for co-design

1. General FPGA overview
2. CPUs in FPGA co-design



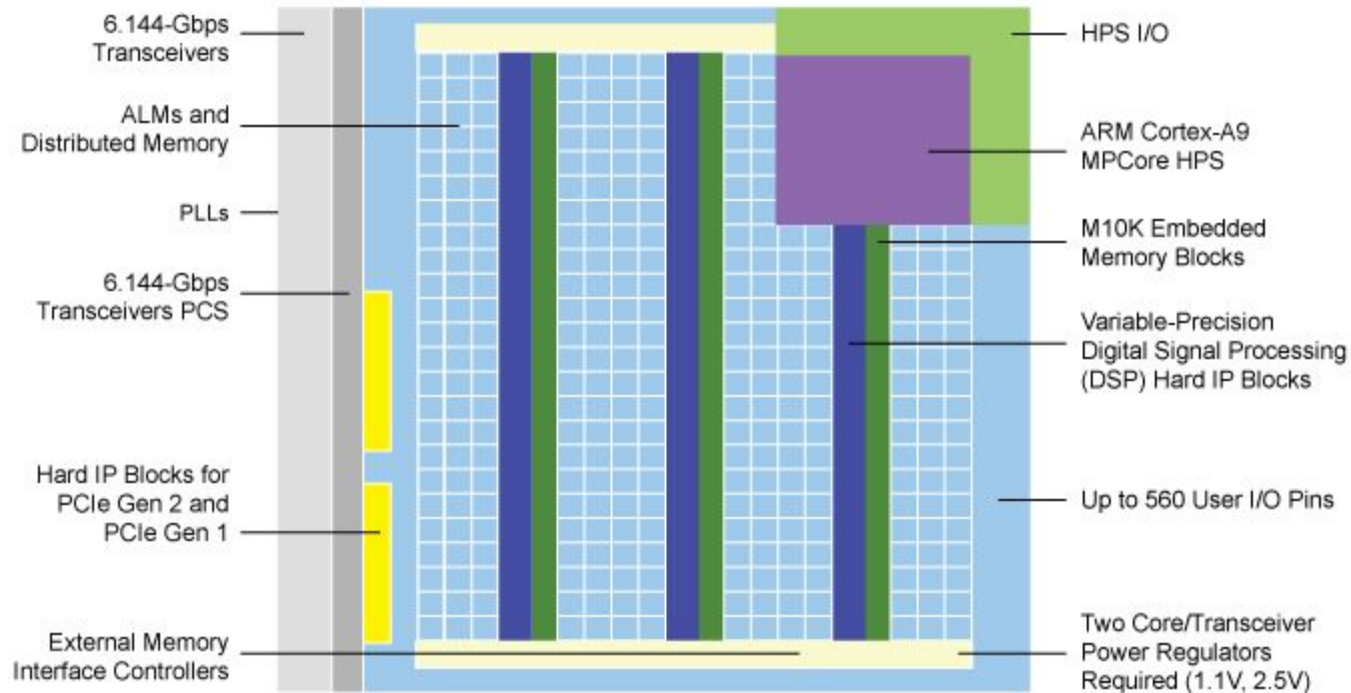
## II. FPGAs for co-design

1. **General FPGA overview**
2. **CPUs in FPGA co-design**

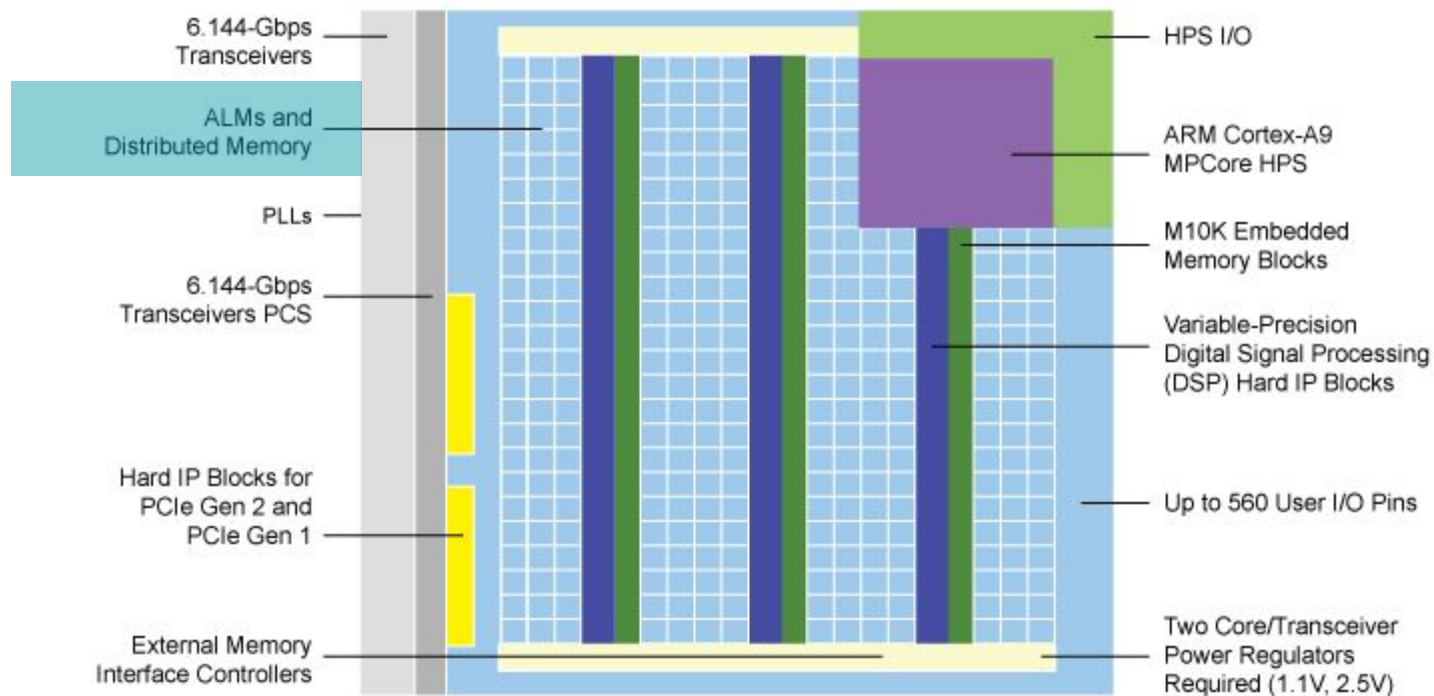
# FPGAs: A heterogeneous architecture



## Cyclone-V SoC Platforms: A case study



# FPGAs: A heterogeneous architecture



# Programmable logic in FPGAs



## Normal mode

- Combinatorial functions of 4/6 inputs

## LUT mode

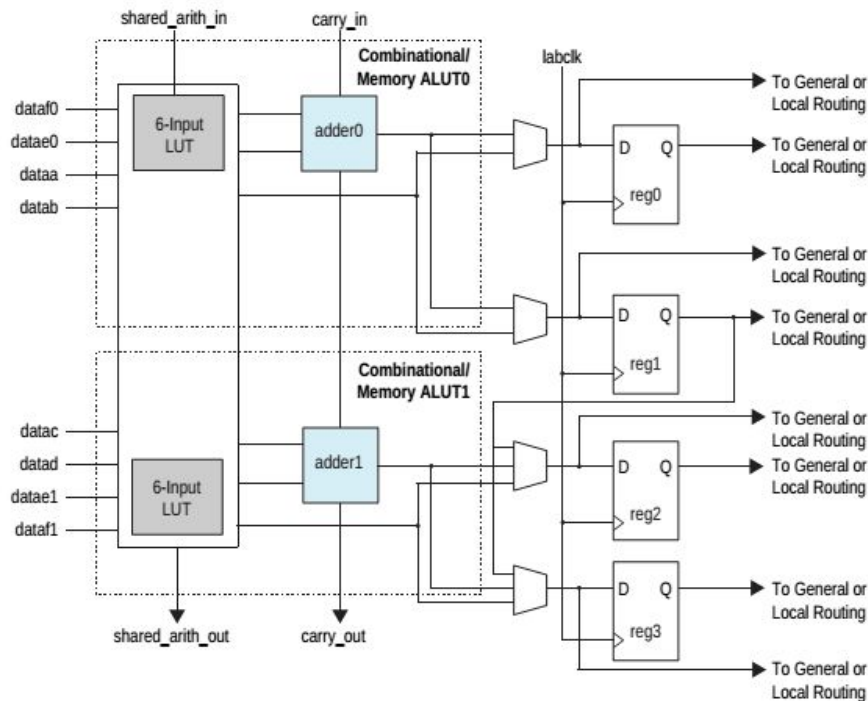
- In-situ memory
- Typically inferred for *if/else* statements

## Arithmetic mode

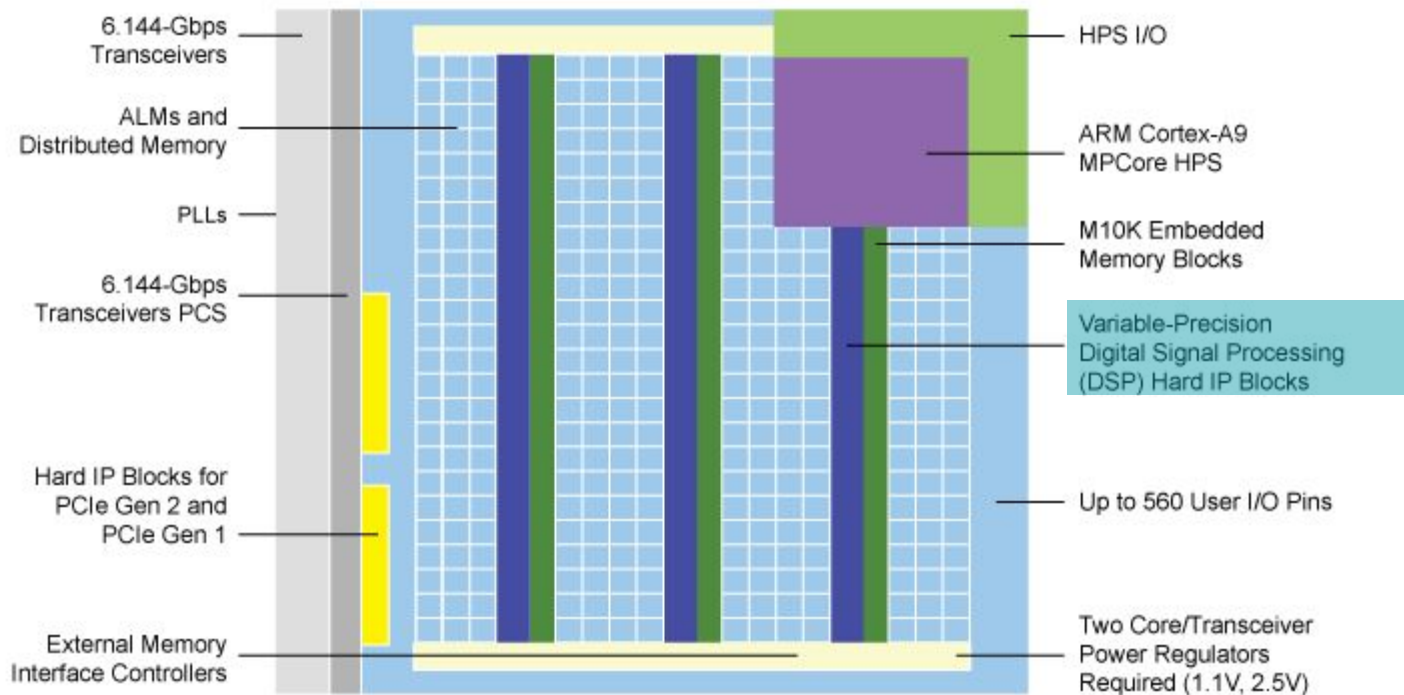
- Full adders with carry

## Shared arithmetic mode

- Implement a three-input add.



# FPGAs: A heterogeneous architecture

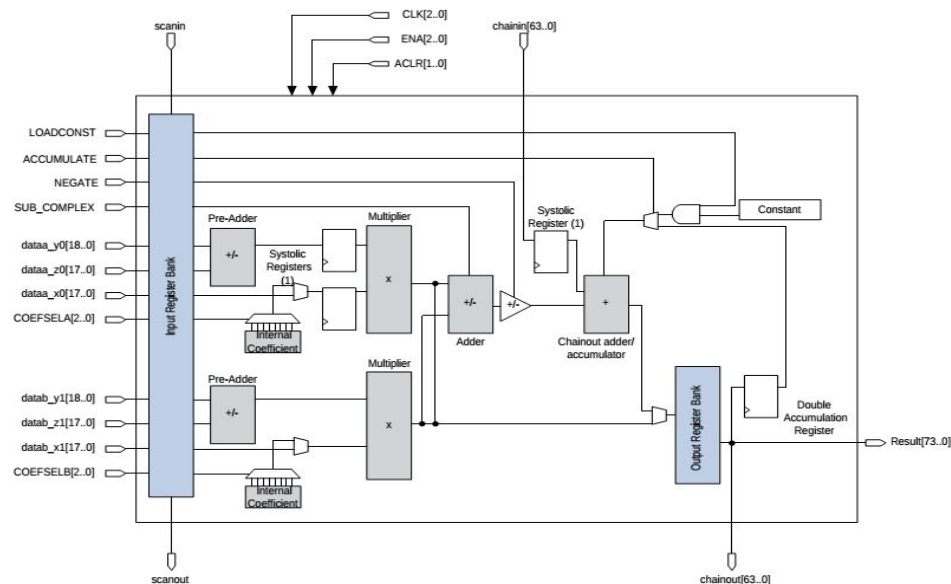


# DSPs in FPGAs

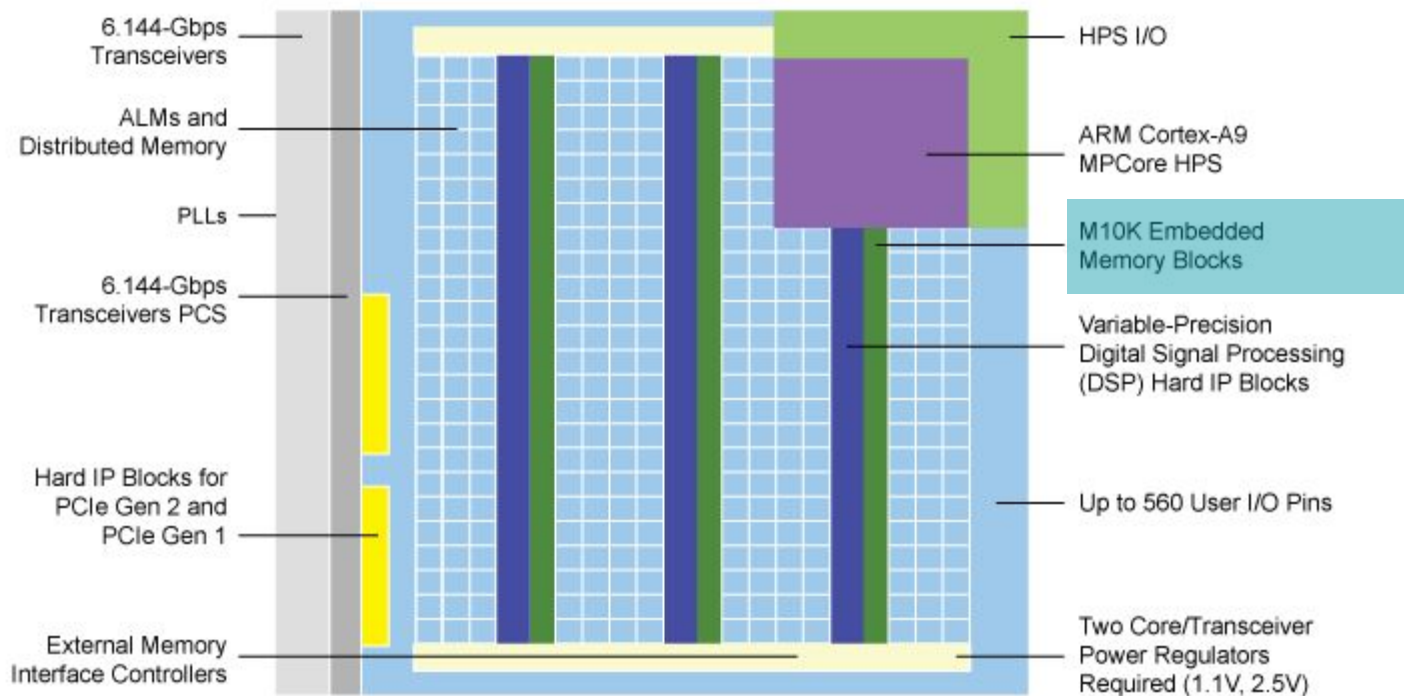


Particularly well suited to MAC operations:

- One (27 x 27) multiplication
- Two independent (18 x 18) multiplications concurrently
- Three independent (9 x 9) multiplications concurrently



# FPGAs: A heterogeneous architecture





# FPGAs: A heterogeneous architecture



Ideal to store large blocks of data

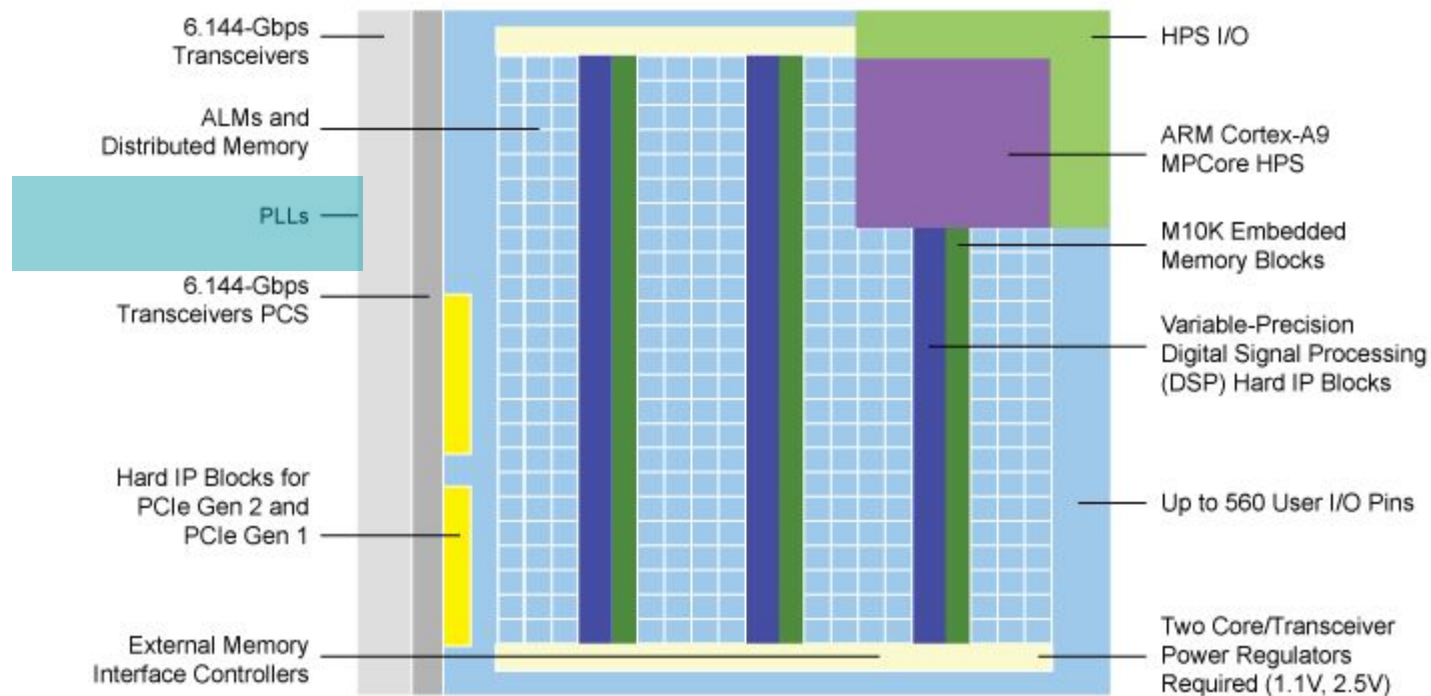
Can be used as:

- RAM (single /dual port)
- Shift registers
- ROMs
- FIFOs

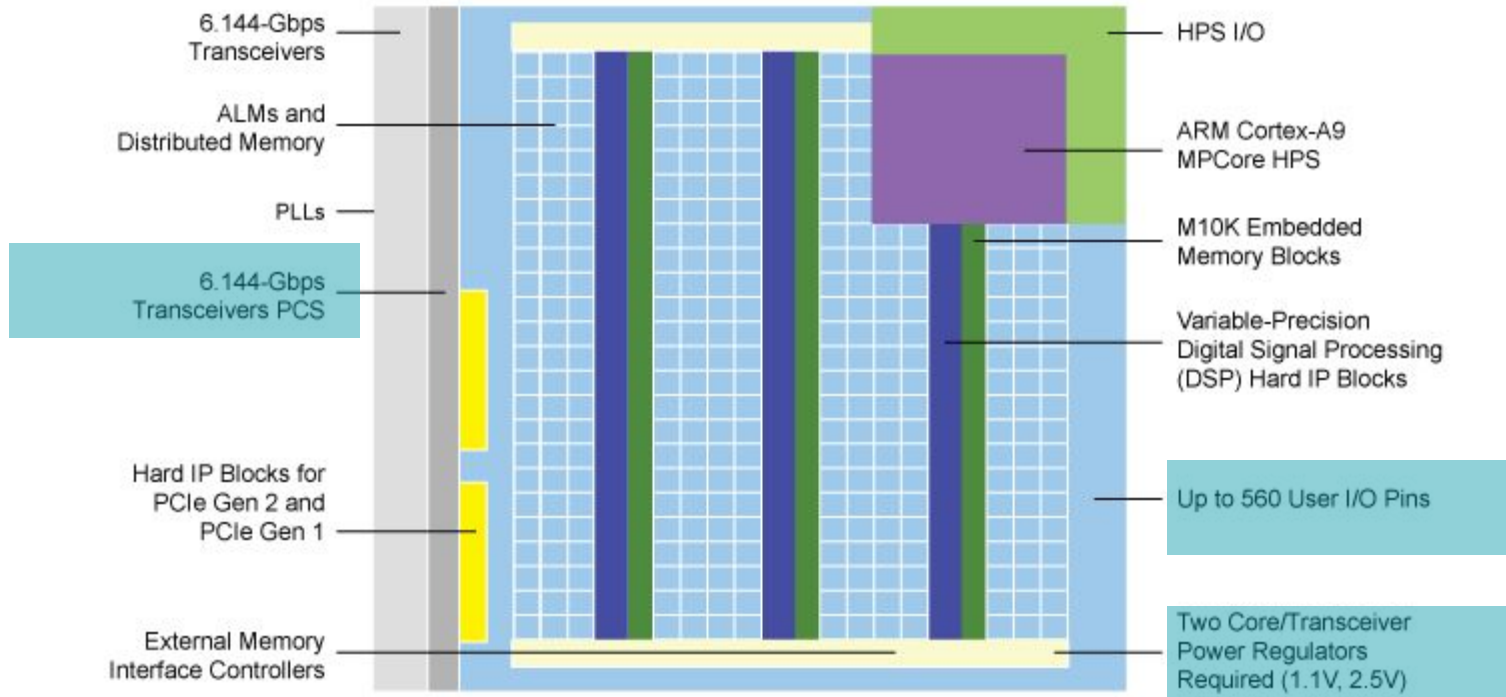
Memory blocks can also be created through MLABs : A combination of 10 ALMs in LUT modes

| Variant      | Member Code | M10K  |              | MLAB  |              | Total RAM Bit (Kb) |
|--------------|-------------|-------|--------------|-------|--------------|--------------------|
|              |             | Block | RAM Bit (Kb) | Block | RAM Bit (Kb) |                    |
| Cyclone V GX | C3          | 119   | 1,190        | 255   | 159          | 1,349              |
|              | C4          | 250   | 2,500        | 472   | 295          | 2,795              |
|              | C5          | 446   | 4,460        | 679   | 424          | 4,884              |
|              | C7          | 686   | 6,860        | 1338  | 836          | 7,696              |
|              | C9          | 1,220 | 12,200       | 2748  | 1,717        | 13,917             |
| Cyclone V GT | D5          | 446   | 4,460        | 679   | 424          | 4,884              |
|              | D7          | 686   | 6,860        | 1338  | 836          | 7,696              |
|              | D9          | 1,220 | 12,200       | 2748  | 1,717        | 13,917             |
| Cyclone V SE | A2          | 140   | 1,400        | 221   | 138          | 1,538              |
|              | A4          | 270   | 2,700        | 370   | 231          | 2,460              |
|              | A5          | 397   | 3,970        | 768   | 480          | 4,450              |
|              | A6          | 557   | 5,570        | 994   | 621          | 5,761              |
| Cyclone V SX | C2          | 140   | 1,400        | 221   | 138          | 1,538              |
|              | C4          | 270   | 2,700        | 370   | 231          | 2,460              |
|              | C5          | 397   | 3,970        | 768   | 480          | 4,450              |
|              | C6          | 557   | 5,570        | 994   | 621          | 5,761              |
| Cyclone V ST | D5          | 397   | 3,970        | 768   | 480          | 4,450              |
|              | D6          | 557   | 5,570        | 994   | 621          | 5,761              |

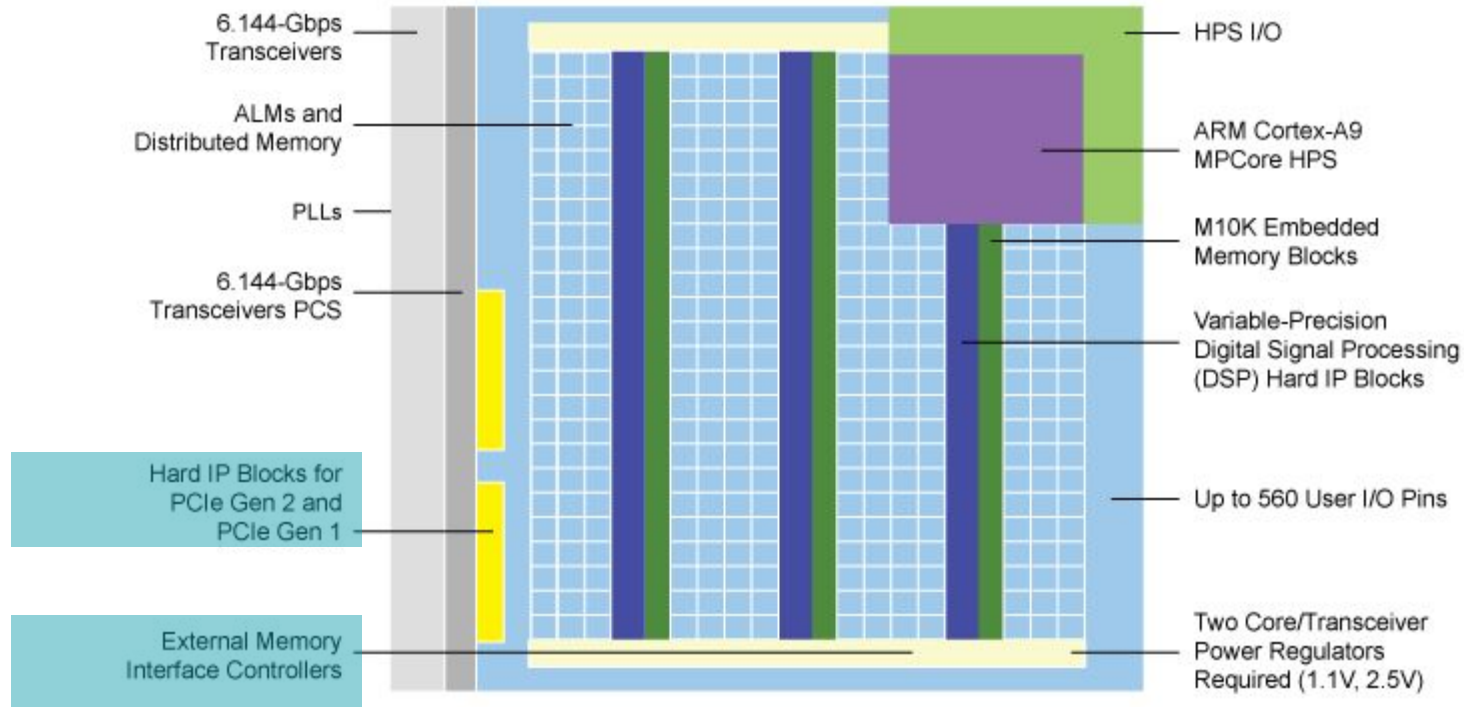
# FPGAs: A heterogeneous architecture



# FPGAs: A heterogeneous architecture



# FPGAs: A heterogeneous architecture



# FPGA integration: a development kit

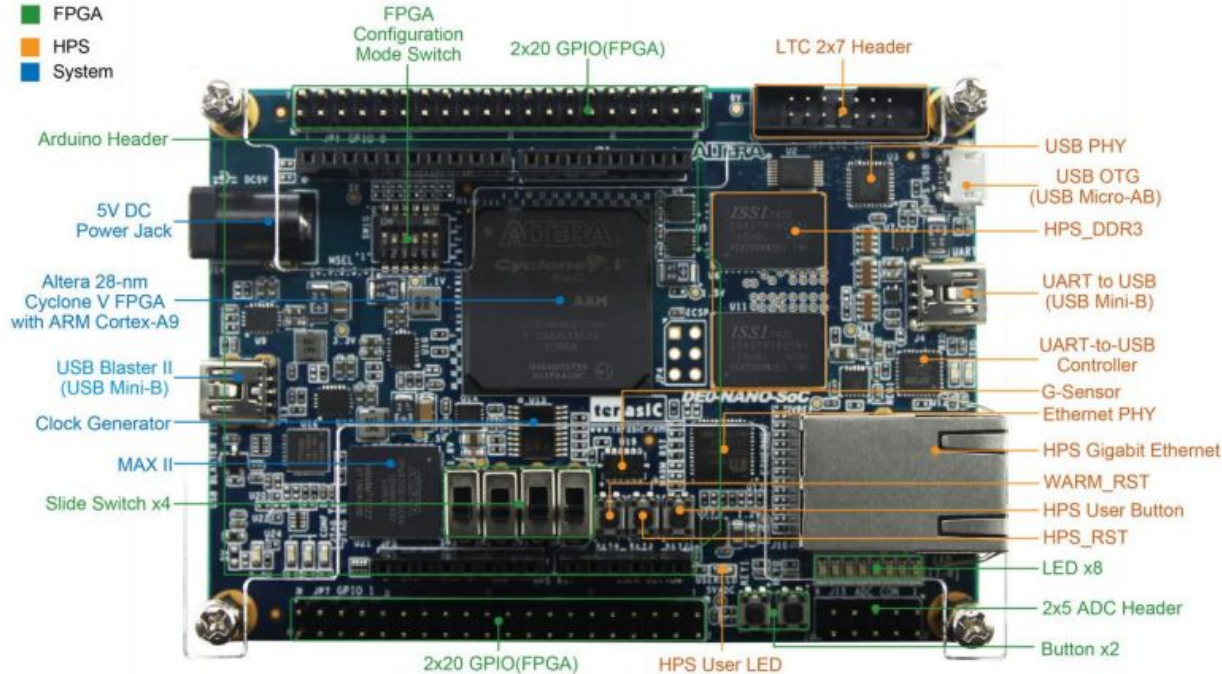


Figure 2-1 DE0-Nano-SoC development board (top view)

# FPGA integration: a development kit

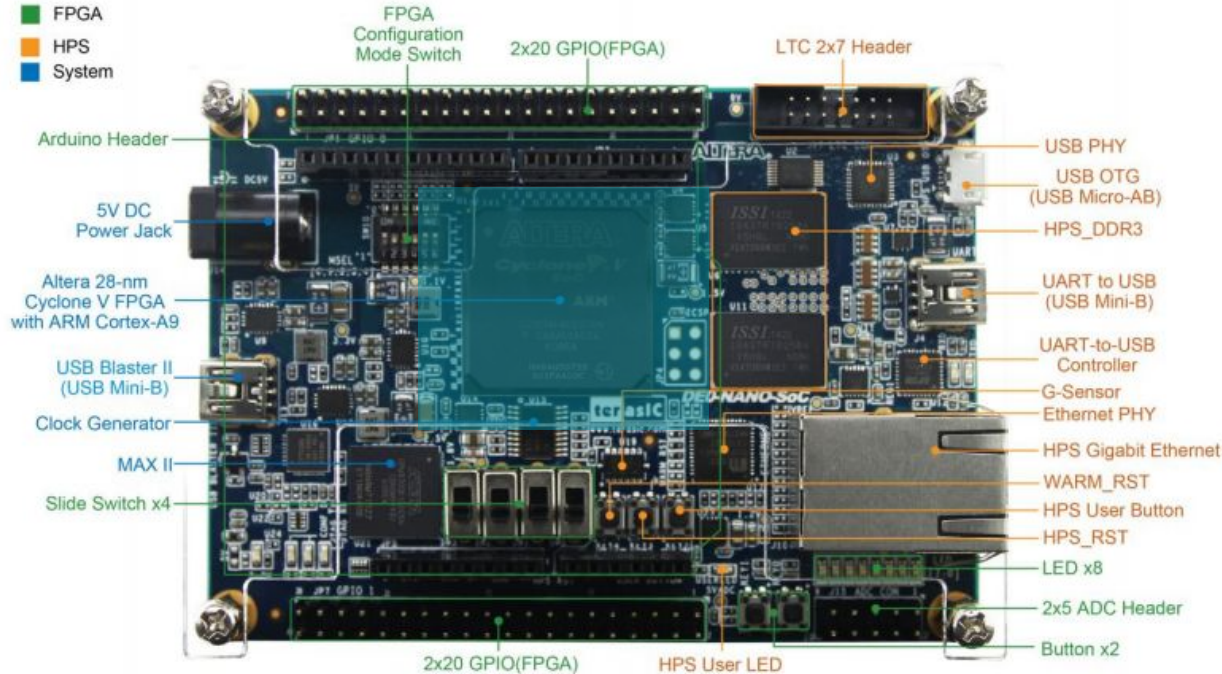


Figure 2-1 DE0-Nano-SoC development board (top view)



# A typical FPGA devkit for co-design

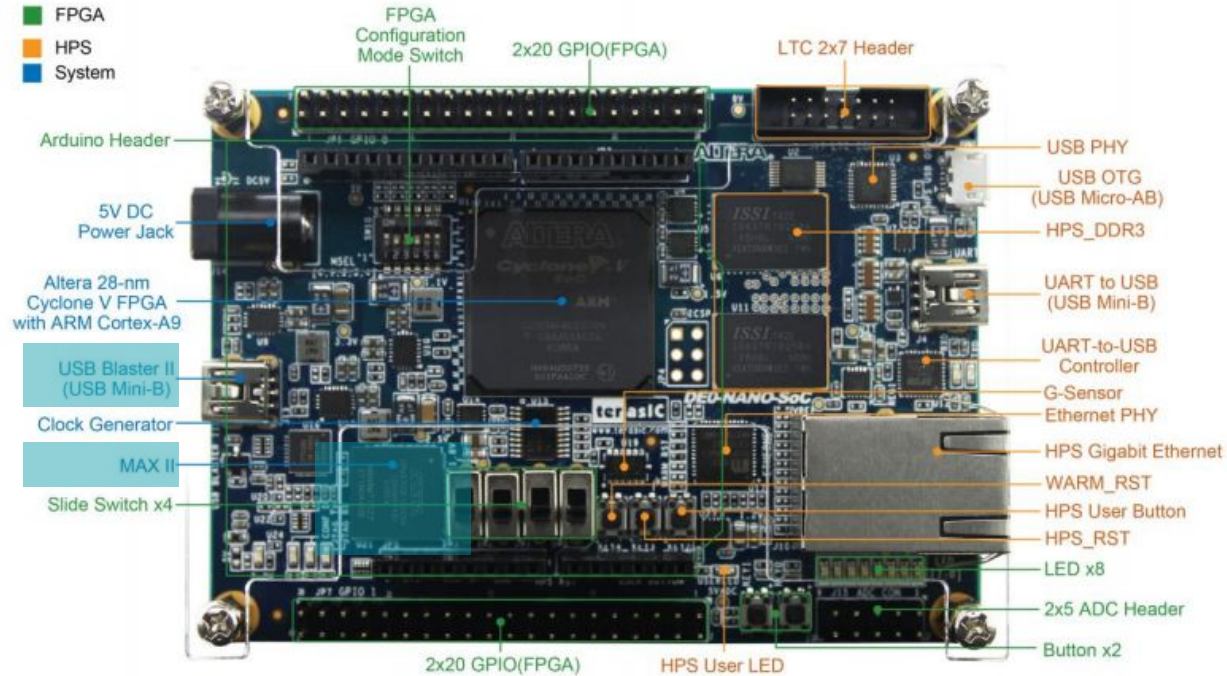


Figure 2-1 DE0-Nano-SoC development board (top view)

# A typical FPGA devkit for co-design

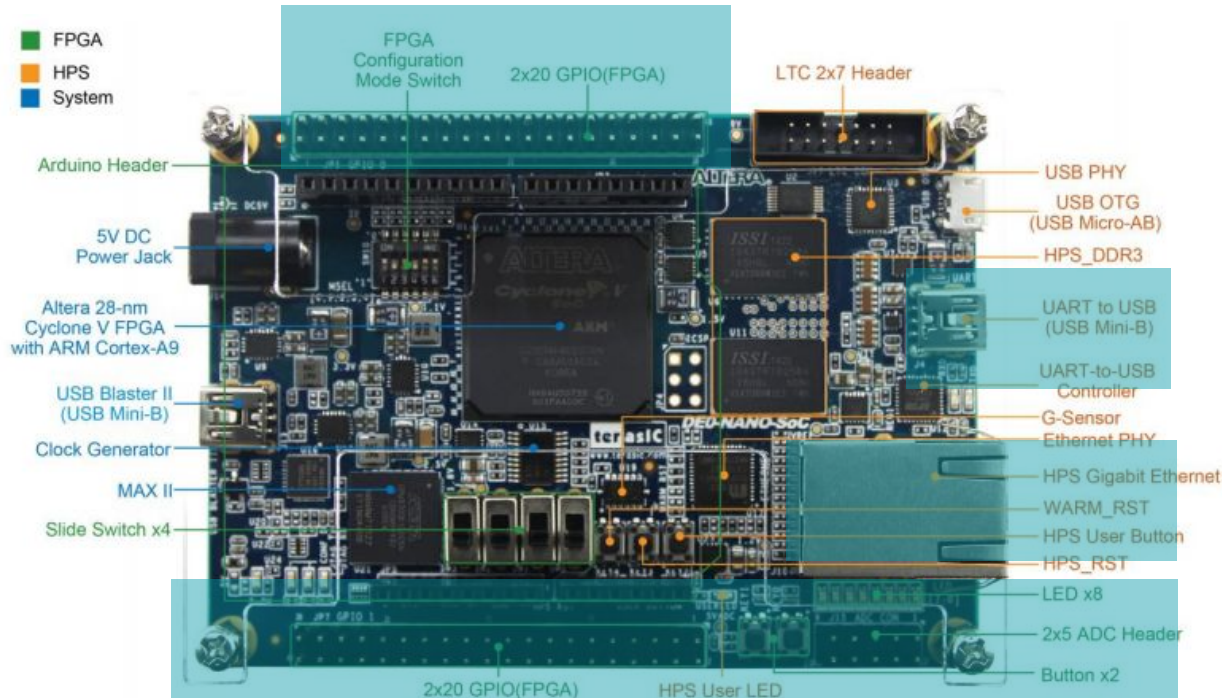


Figure 2-1 DE0-Nano-SoC development board (top view)



# A typical FPGA devkit for co-design

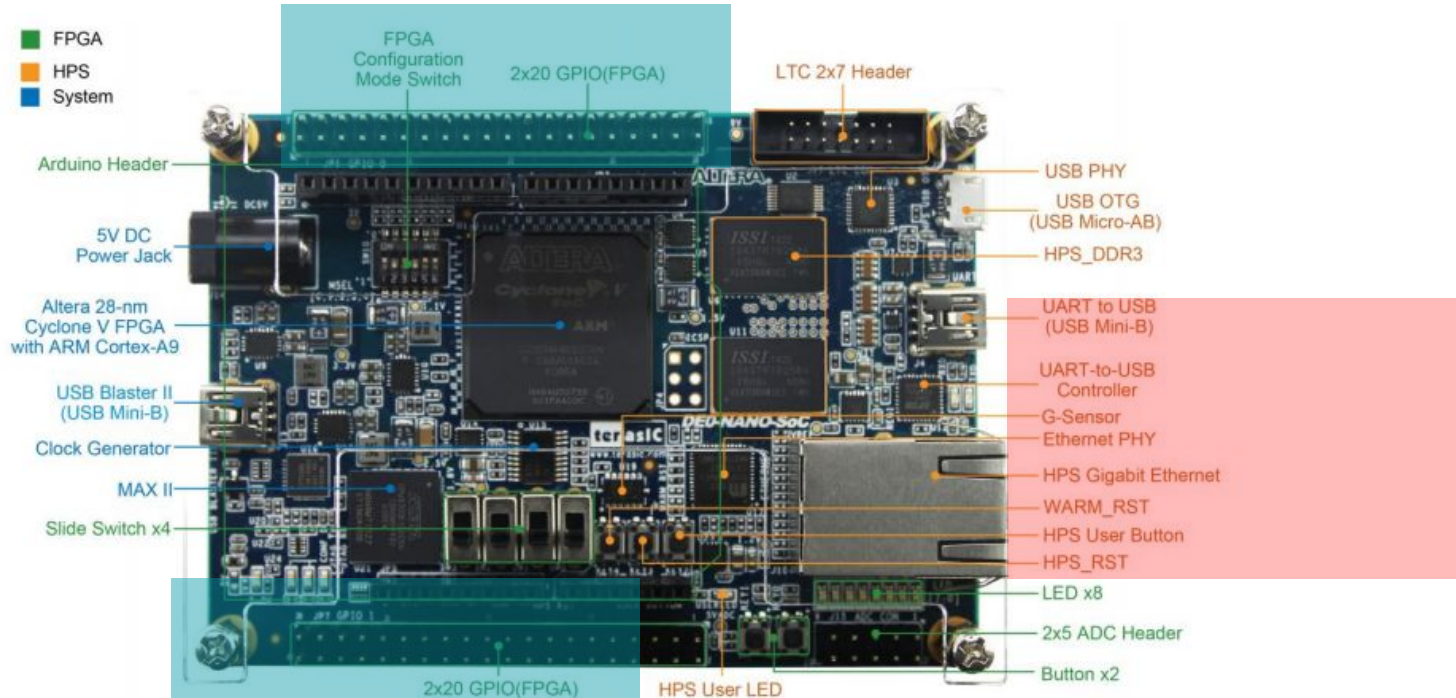


Figure 2-1 DE0-Nano-SoC development board (top view)

# A typical FPGA devkit for co-design

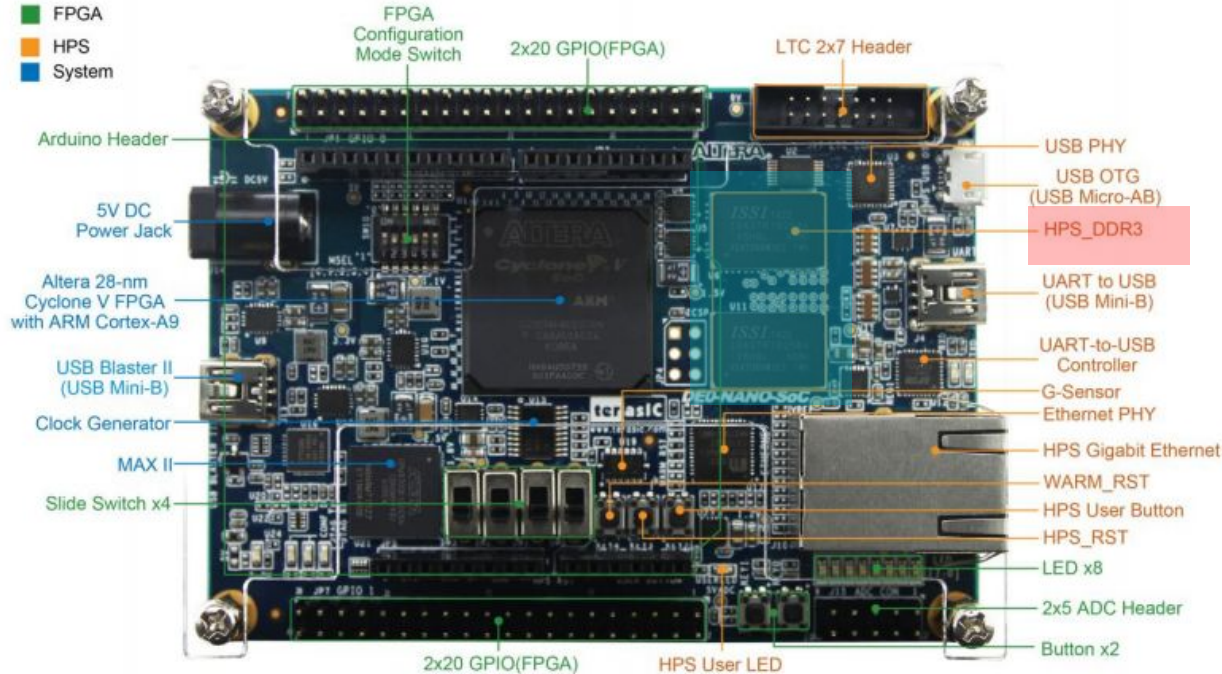


Figure 2-1 DE0-Nano-SoC development board (top view)



## II. FPGAs for co-design

1. **General FPGA overview**
2. **CPUs in FPGA co-design**



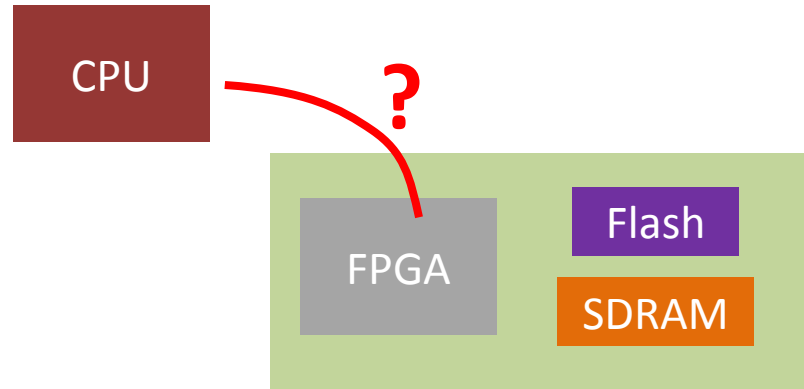
## II. FPGAs for co-design

1. General FPGA overview
2. CPUs in FPGA co-design

# CPU in the co-design space



- A CPU handles a critical control function in the SoPC paradigm
- How can I integrate a CPU in my system ?



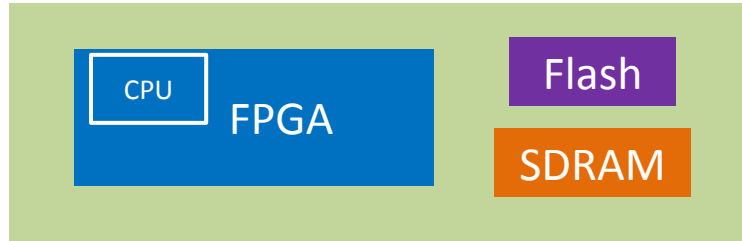
# CPU in the co-design space



A CPU completes a critical control function in the SoPC paradigm

## Two approaches

### Soft-core processor



### Hard-core processor



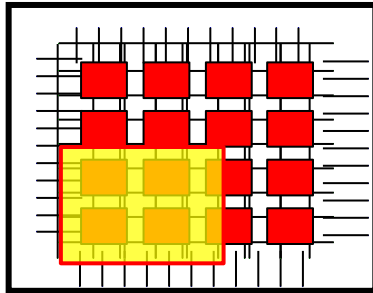
# softcore vs Hard Core processor



## Soft-core processor

synthesized onto the FPGA's fabric, just like any others component.

**Processor  
HDL  
description**



Nios II from Intel FPGAs

Microblaze from Xilinx

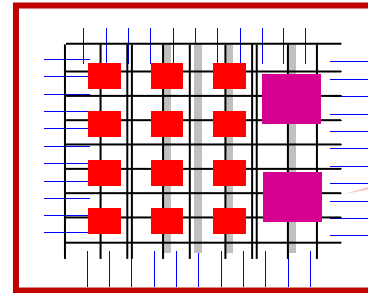
And many open source cores:

- VexRiscV, SpinalHDL ...



## Hard-core processor

laid out on the chip next to the FPGA's configurable logic fabric



Proc is an  
element of the  
chip

Cyclone V SoC for Intel  
Zynq 7000 for Xilinx

**ARM**

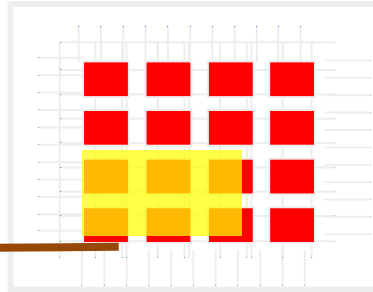


# softcore vs Hard Core processor



A soft-core processor is synthesized onto the FPGA's fabric, just like any others circuits.

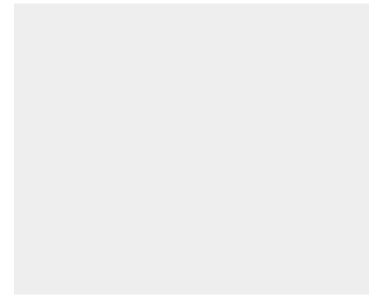
**Processor  
HDL  
description**



## Nios II from Intel FPGAs



A hard-core processor is laid out on the chip next to the FPGA's configurable logic fabric



Processor is in  
silicon!

Cyclone V SoC for Intel  
Zynq 7000 for Xilinx



# Softcore vs Hard Core processor



## Hard-core processors offer:

- better performance in terms of operating frequencies
- reduced energy consumption

## Soft-core processors offer :

- a higher degree of configurability, implementation options
- lower cost per device.



Currently, hard-core processors tend to reappear in SoPC platforms

## Soft-core processors advantages:

- Utilizing standard mass-produced
- Enabling a custom number of processor cores

## Soft-core processors disadvantages:

- Reduced processor performances
- Higher power consumption
- Larger size.

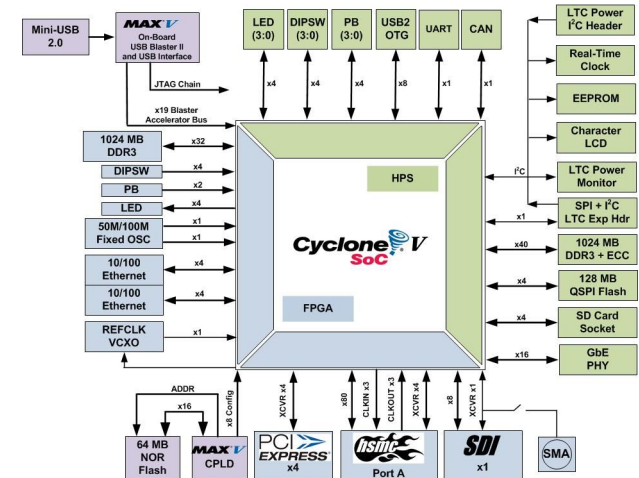
# Hardcore processors in FPGAs



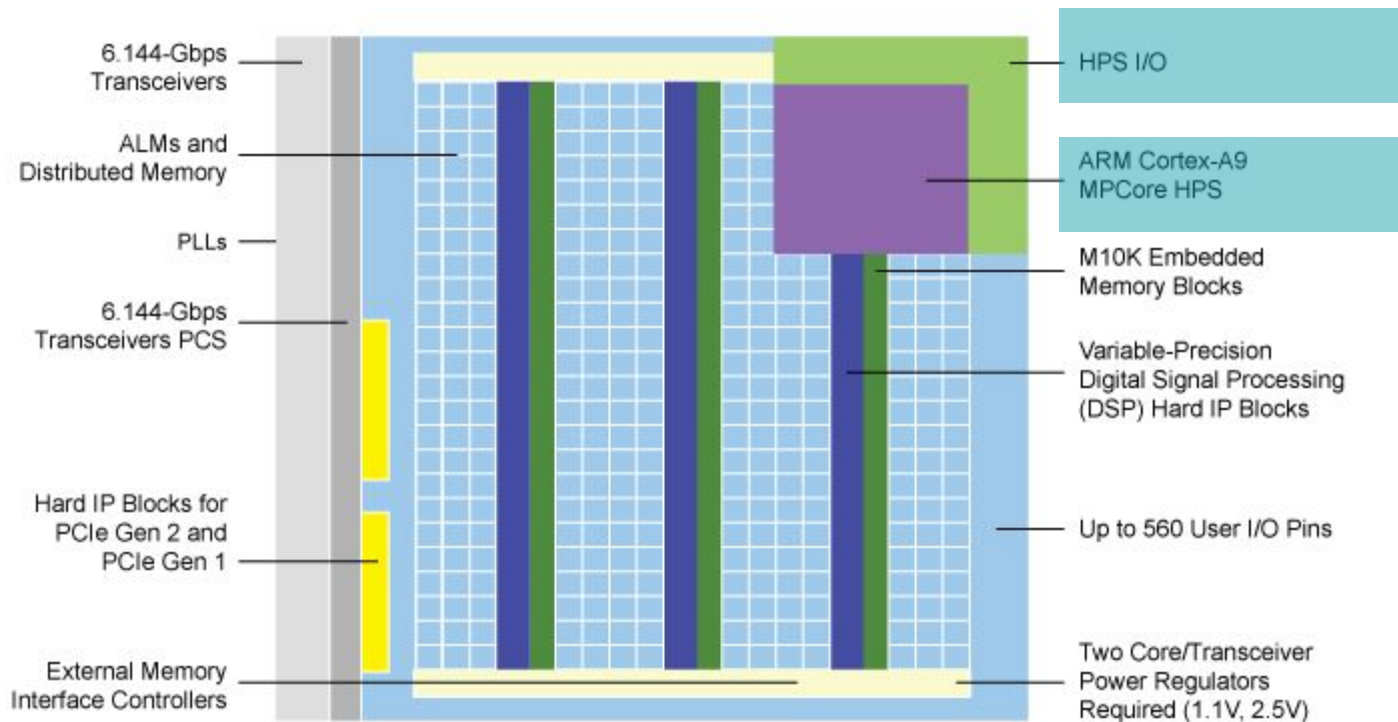
| Resources | Product Line                  | Cyclone V E FPGAs <sup>1</sup> |        |         |         |
|-----------|-------------------------------|--------------------------------|--------|---------|---------|
|           |                               | 5CEA2                          | 5CEA4  | 5CEA5   | 5CEA7   |
|           | LEs (K)                       | 25                             | 49     | 77      | 149.5   |
|           | ALMs                          | 9,434                          | 18,480 | 29,080  | 56,480  |
|           | Registers                     | 37,736                         | 73,920 | 116,320 | 225,920 |
|           | M10K memory blocks            | 176                            | 308    | 446     | 686     |
|           | M10K memory (Kb)              | 1,760                          | 3,080  | 4,460   | 6,860   |
|           | MLAB memory (Kb)              | 196                            | 303    | 424     | 836     |
|           | Variable-precision DSP blocks | 25                             | 66     | 150     | 156     |
|           | 18 x 18 multipliers           | 50                             | 132    | 300     | 312     |

| Cyclone V SE SoCs <sup>1</sup> |        |         |         |
|--------------------------------|--------|---------|---------|
| 5CSEA2                         | 5CSEA4 | 5CSEA5  | 5CSEA6  |
| 25                             | 40     | 85      | 110     |
| 9,434                          | 15,094 | 32,075  | 41,509  |
| 37,736                         | 60,376 | 128,300 | 166,036 |
| 140                            | 270    | 397     | 557     |
| 1,400                          | 2,700  | 3,970   | 5,570   |
| 138                            | 231    | 480     | 621     |
| 36                             | 84     | 87      | 112     |
| 72                             | 168    | 174     | 224     |

- Remember the SoC matrix from last slide: some FPGAs are also SoCs
- An HPS occupies “space” on the chip, at the price of FPGA resources !



# Hardcore processors (HPS)





# Hardcore processors (HPS)

A typical HPS in FPGAs: the ARM Cortex A9 MPCore

- An ARMv7 CPU : 2GHz, dual core, 2.50 DMIPS/MHz/core
- 32 KB of L1 cache, up to 8 MB of L2 cache
- NEON SIMD instruction set extension
- VFPv3 floating point unit
- Thumb-2 instruction set encoding

The same core equipped devices like the iPad 3, iPhone 4S or the Playstation VITA

# HPS : Memory

An HPS typically uses dedicated RAM

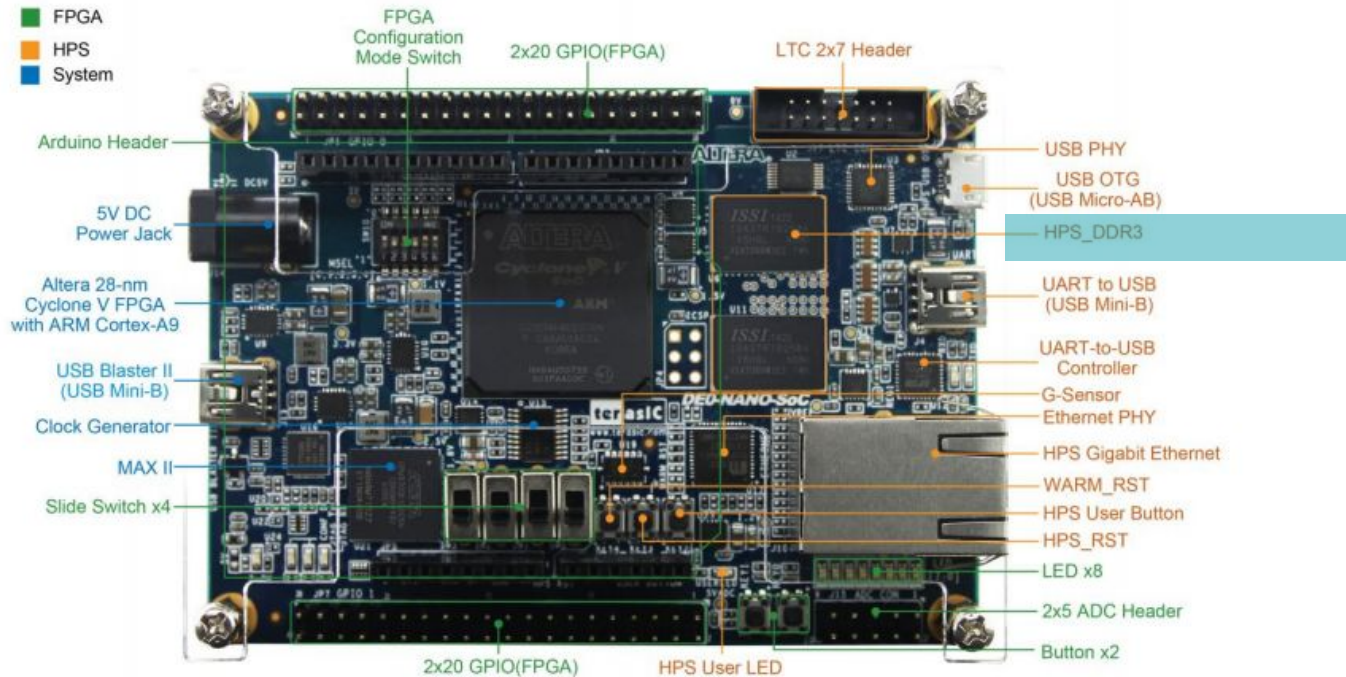


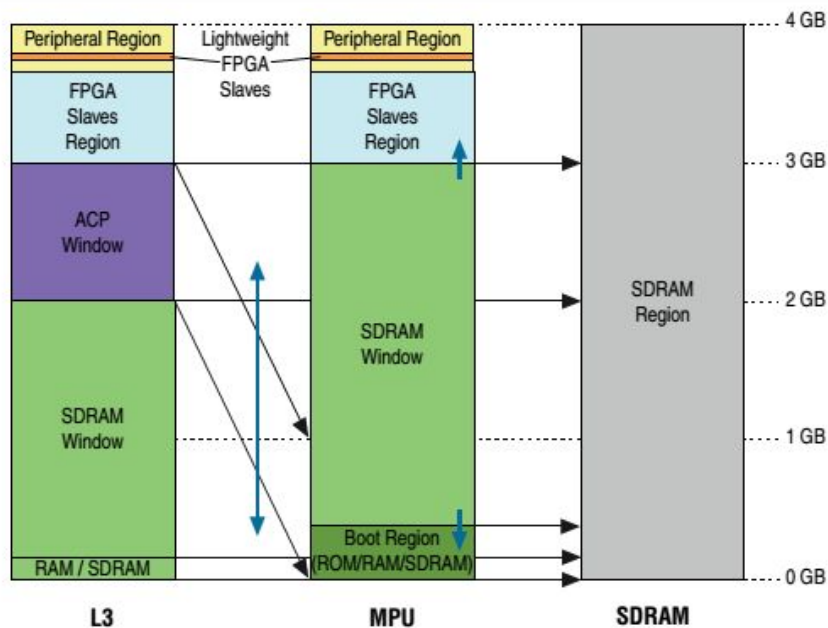
Figure 2-1 DE0-Nano-SoC development board (top view)

# HPS : Memory



The HPS can also share memory with the FPGA ...

**Figure 1-3. HPS Address Space Relationships**





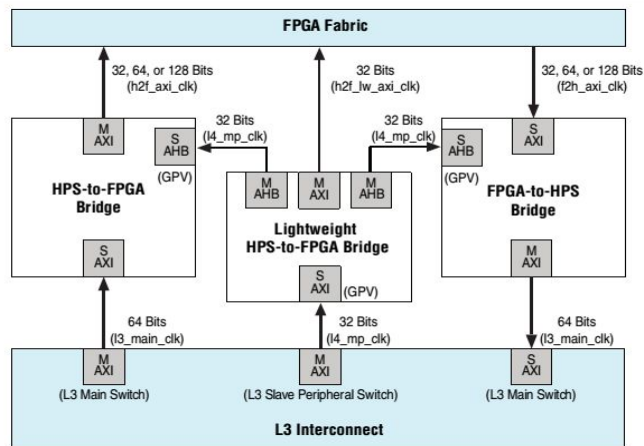
# HPS: Connection to FPGA

Table 5–1. AXI Bridge Features

| Feature                                                                                                                  | FPGA-to-HPS Bridge | HPS-to-FPGA Bridge | Lightweight HPS-to-FPGA Bridge |
|--------------------------------------------------------------------------------------------------------------------------|--------------------|--------------------|--------------------------------|
| Supports the AMBA AXI3 interface protocol                                                                                | ✓                  | ✓                  | ✓                              |
| Implements clock crossing and manages the transfer of data across the clock domains in the HPS logic and the FPGA fabric | ✓                  | ✓                  | ✓                              |
| Performs data width conversion between the HPS logic and the FPGA fabric                                                 | ✓                  | ✓                  | ✓                              |
| Allows configuration of FPGA interface widths at instantiation time                                                      | ✓                  | ✓                  |                                |

Mainly uses the AXI-3 protocol

Figure 5–1. AXI Bridge Connectivity



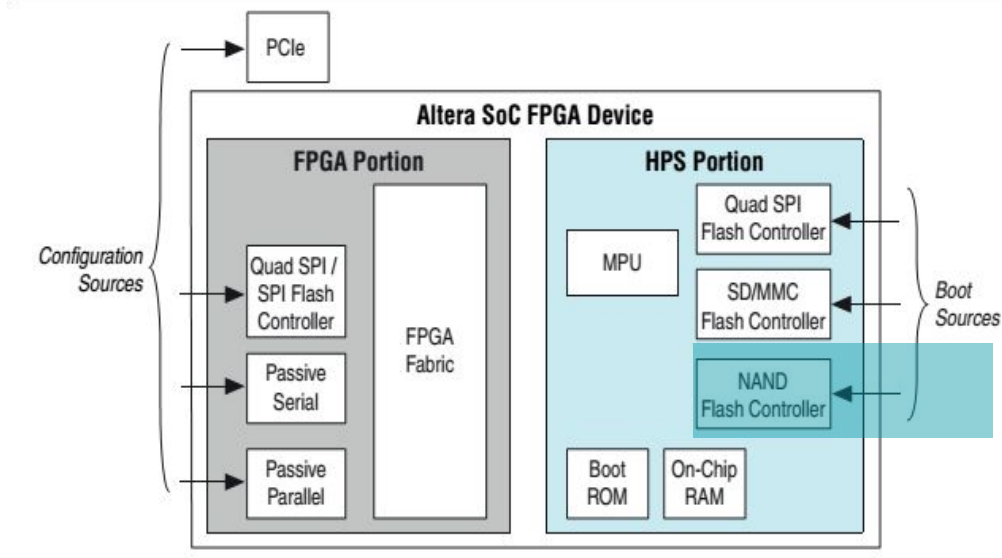
# HPS: Boot selection



The HPS processors can boot from 4 sources:

- FLASH NAND memory
- SD / MMC memory
- QSPI Flash
- The FPGA design

**Figure A-1. Independent FPGA Configuration and HPS Booting**





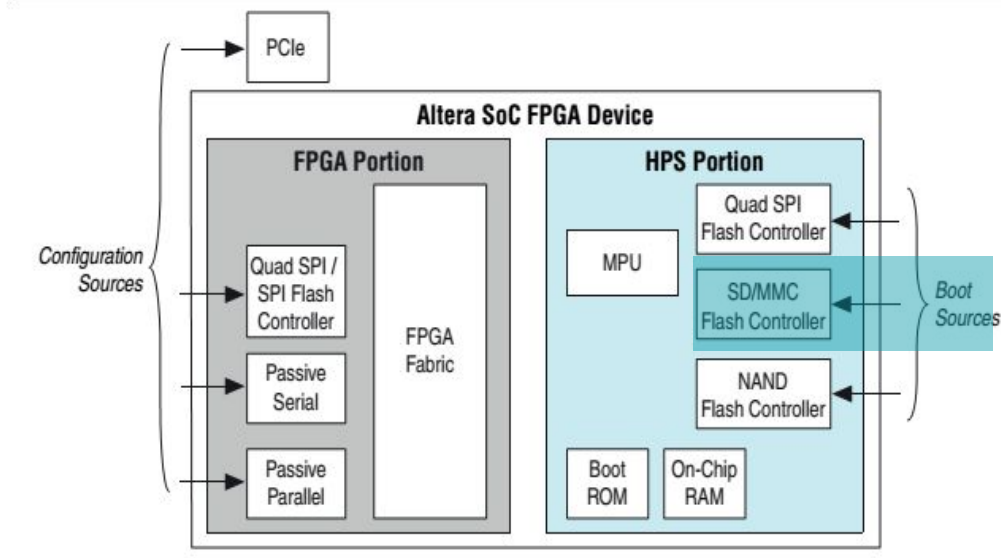
# HPS: Boot options



The HPS processors can boot from 4 sources:

- FLASH NAND memory
- SD / MMC memory
- QSPI Flash
- The FPGA design

**Figure A-1. Independent FPGA Configuration and HPS Booting**



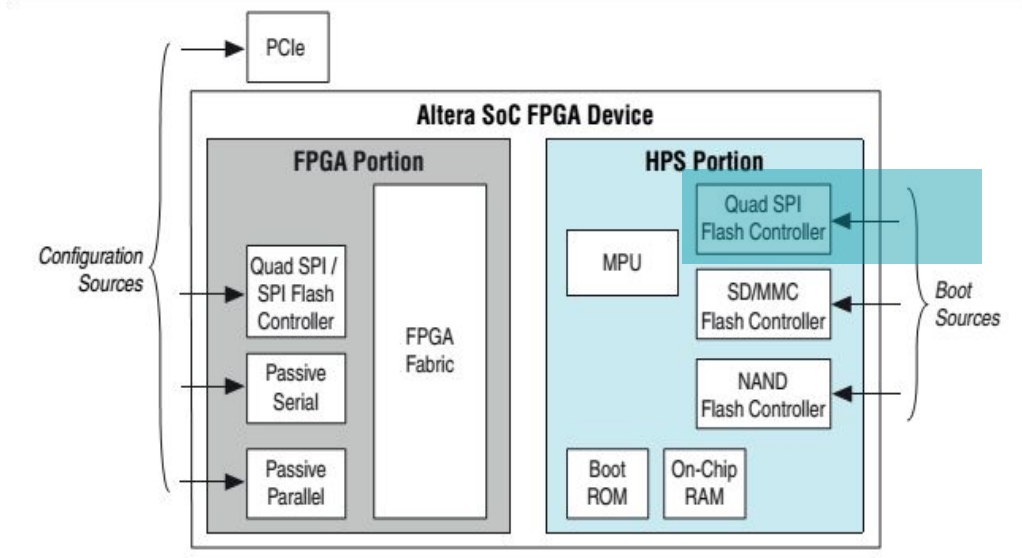
# HPS: Boot options



The HPS processors can boot from 4 sources:

- FLASH NAND memory
- SD / MMC memory
- QSPI Flash
- The FPGA design

**Figure A-1. Independent FPGA Configuration and HPS Booting**



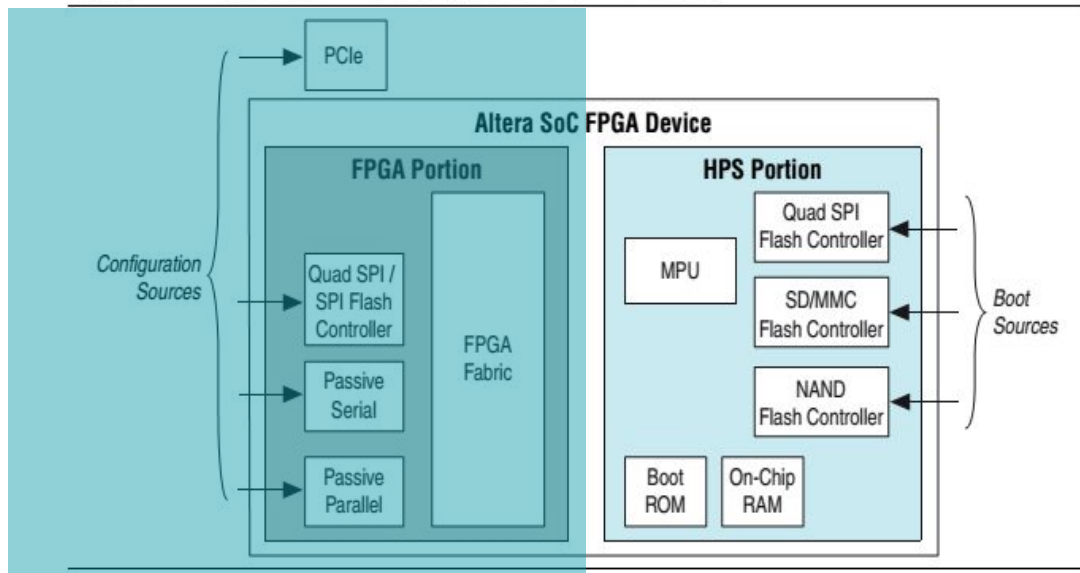
# HPS: Boot options



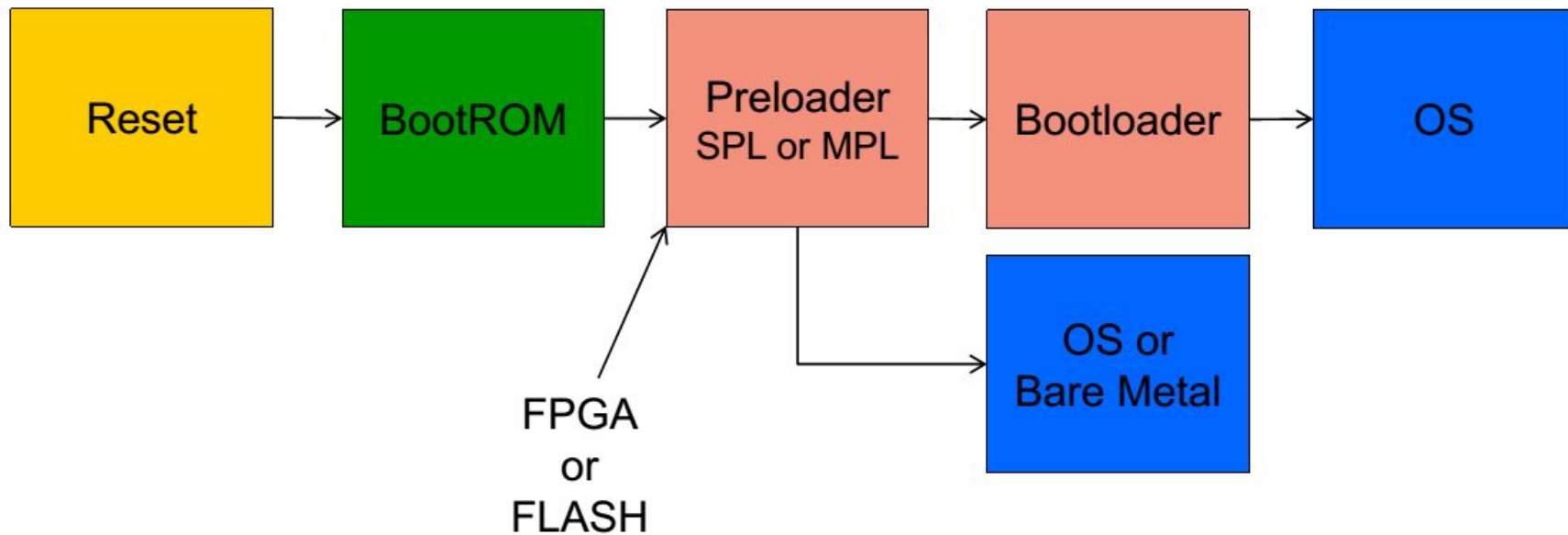
The HPS processors can boot from 4 sources:

- FLASH NAND memory
- SD / MMC memory
- QSPI Flash
- The FPGA design

**Figure A-1. Independent FPGA Configuration and HPS Booting**



# HPS Boot sequence





**Time to check your microprocessor  
architecture lecture (in French)**



# Last lecture

- From an ALU to a microprocessor
- Von Neumann vs Harvard
- Instruction set architectures
- RISC ...

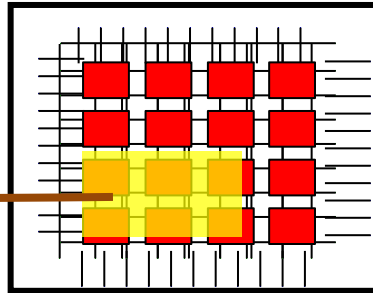
# Softcore vs Hard Core processor



## Soft-core processor

synthesized onto the FPGA's fabric, just like any others circuits.

Processor  
HDL description



Nios II from Intel FPGAs

Microblaze from Xilinx

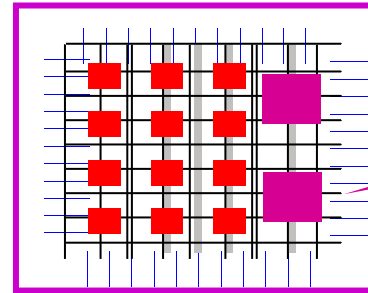
And many open source cores:

- VexRiscV, SpinalHDL ...



## Hard-core processor

laid out on the chip next to the FPGA's configurable logic fabric



Processor is in  
silicon!

Cyclone V SoC for Intel  
Zynq 7000 for Xilinx

ARM®



# Softcore Processors



- **Are soft**, i.e. specified through field programming just like programmable logic
  - Shipped as hardware description files, which can be mapped onto FPGA. e.g: Nios II
  - Are bundled with software development tools (compiler, simulator, etc.)
- **Offer flexibility** as microprocessor parameters can be tuned to the application with tight on-chip interconnection with additional circuitry.
- Designs can be **marketed quickly**. You can test and validate many designs quickly without making any specific board; no soldering, no wiring and no foundry!



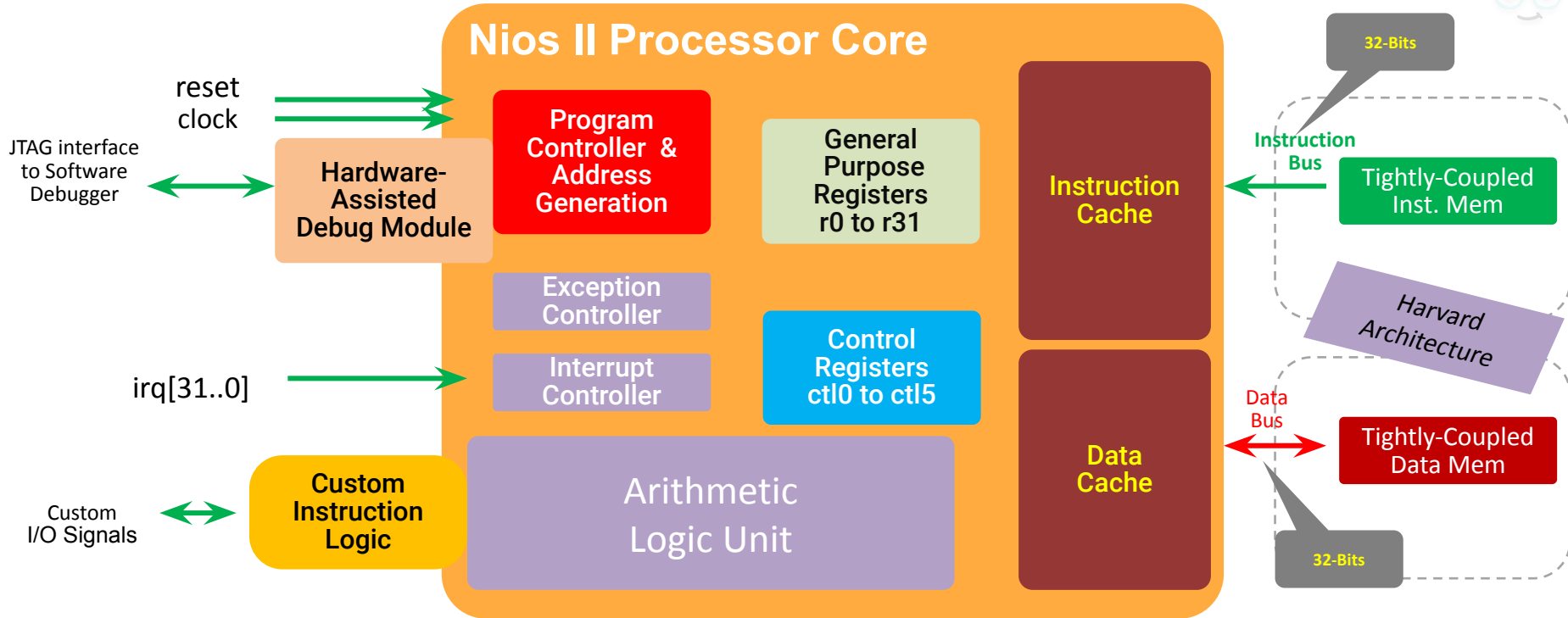


# Nios II: Intel softcore



- A proprietary 32-bit softcore from Altera
- Comes in two flavors: Fast, Light
- The two cores trade FPGA area and power consumption for speed of execution.
- Is a RISC, Harvard Architecture: Simple instructions, separate data and instruction bus.
- Has 32 levels of interrupts.
- **Uses the Avalon Bus interface (Avalon Switch Fabric)**
- Programs compiled using GNU C/C++ toolchain

# Nios II Architecture



- Thirty two 32-bit general registers, up to six 32-bit control registers
- ALU- 32bit two input to one input, does shifts, logic and arithmetic.
- On-Chip Hardware (Shift, Rotate, Multiply in fast )

# Nios II Architecture



**Nios II General-Purpose Registers:** These registers give the state of the Nios at each time

| Register | Name | Function              | Register | Name | Function                      |
|----------|------|-----------------------|----------|------|-------------------------------|
| r0       | zero | 0x00000000            | r16      |      | Callee-saved register         |
| r1       | at   | Assembler temporary   | r17      |      | Callee-saved register         |
| r2       |      | Return value          | r18      |      | Callee-saved register         |
| r3       |      | Return value          | r19      |      | Callee-saved register         |
| r4       |      | Register arguments    | r20      |      | Callee-saved register         |
| r5       |      | Register arguments    | r21      |      | Callee-saved register         |
| r6       |      | Register arguments    | r22      |      | Callee-saved register         |
| r7       |      | Register arguments    | r23      |      | Callee-saved register         |
| r8       |      | Caller-saved register | r24      | et   | Exception temporary           |
| r9       |      | Caller-saved register | r25      | bt   | Breakpoint temporary (1)      |
| r10      |      | Caller-saved register | r26      | gp   | Global pointer                |
| r11      |      | Caller-saved register | r27      | sp   | Stack pointer                 |
| r12      |      | Caller-saved register | r28      | fp   | Frame pointer                 |
| r13      |      | Caller-saved register | r29      | ea   | Exception return address      |
| r14      |      | Caller-saved register | r30      | ba   | Breakpoint return address (2) |
| r15      |      | Caller-saved register | r31      | ra   | Return address                |

# Nios II Architecture



**Nios II Control Register:** These registers give information on the Nios

**Actually, there are 32 control registers but the others are reserved**

| Register | Name            | Register Contents                                                                                           |
|----------|-----------------|-------------------------------------------------------------------------------------------------------------|
| 0        | <i>status</i>   | This register controls/holds the state of the NIOS-ii processor. This register contains the <i>PIE</i> bit. |
| 1        | <i>estatus</i>  | This register holds a copy of <i>status</i> register during non-break interrupt processing.                 |
| 2        | <i>bstatus</i>  | This register holds a copy of <i>status</i> register during break interrupt processing.                     |
| 3        | <i>ienable</i>  | This register holds interrupt-enable bits for <i>irq0</i> through <i>irq31</i> .                            |
| 4        | <i>ipending</i> | This register holds pending-interrupt bits that correspond to <i>irq0</i> through <i>irq31</i> .            |



The special instructions `__builtin_rdctl` and `__builtin_wrctl` are the only way to read and write to the control registers.

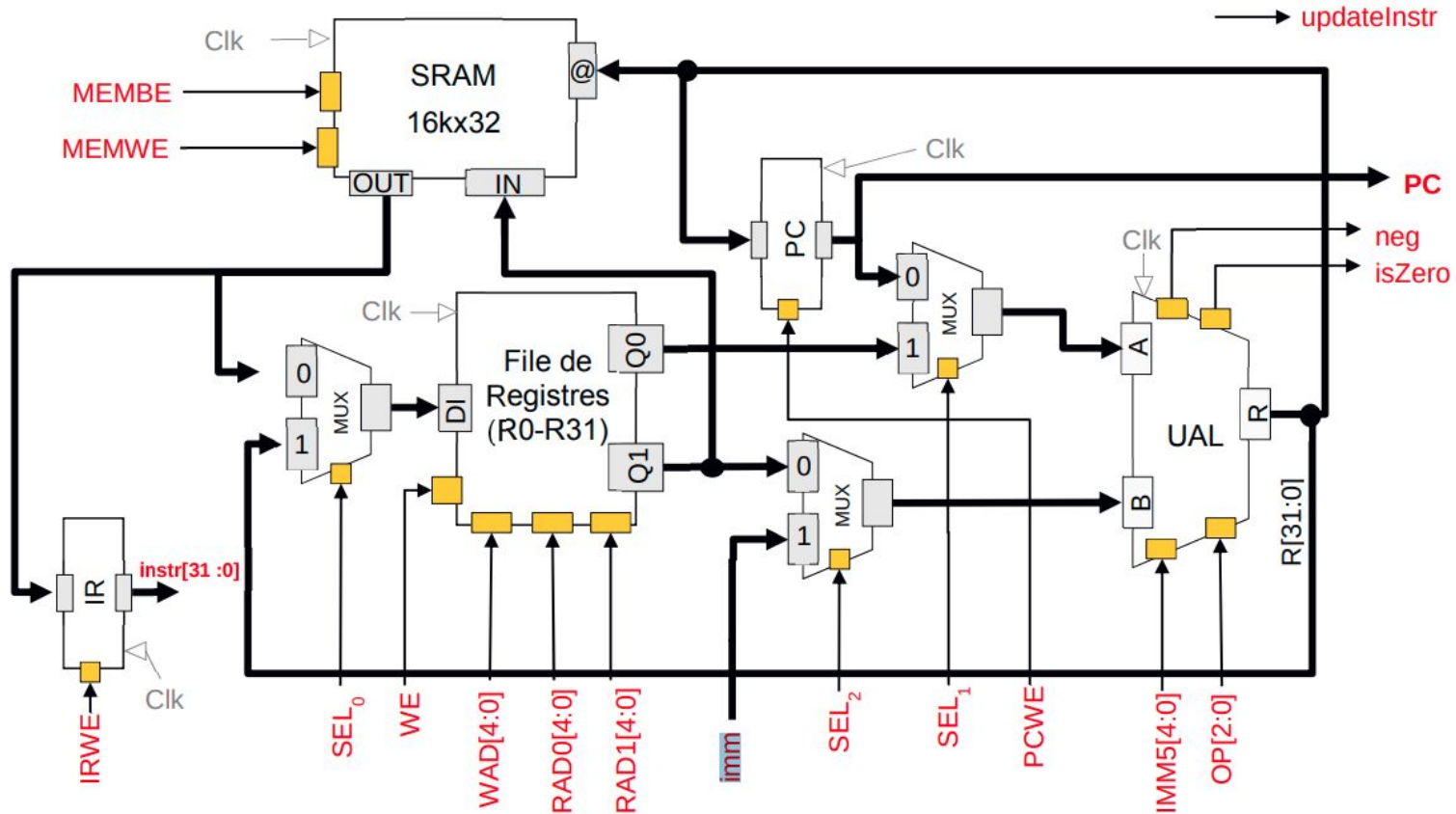
`a=__builtin_rdctl__(3);` // The value of Control register 3 is stored in a

# Three forms of Nios II



- **Nios II/e**—The Nios II/e “economy” core is designed to achieve the smallest possible core size. As a result, this core has a limited feature set, and many settings are not available when the Nios II/e core is selected.
- ~~Nios II/s—The Nios II/s “standard” core is designed for small size while maintaining performance.~~
- **Nios II/f**—The Nios II/f “fast” core is designed for fast performance. As a result, this core presents the most configuration options allowing you to fine-tune the processor for performance.

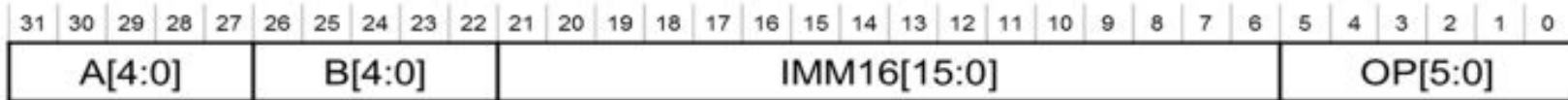
# Nios ISA



# Nios ISA



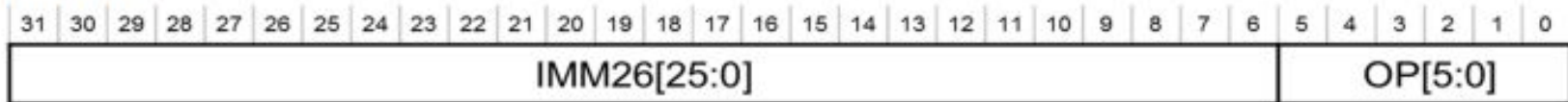
## Format I :



## Format R



## Format J



**immediate** value is a constant number embedded into the instruction itself. These constants are called immediates, because their values are immediately available from the instruction and do not require a register or memory access.

<https://www.sciencedirect.com/topics/computer-science/immediate-operand>

# Nios ISA



## >> logical operations

| Instruction       | Type | effet                                                      |
|-------------------|------|------------------------------------------------------------|
| xor rc,ra,rb      | R    | $rc \leftarrow ra \wedge rb$                               |
| and rc,ra,rb      | R    | $rc \leftarrow ra \& rb$                                   |
| or rc,ra,rb       | R    | $rc \leftarrow ra \mid rb$                                 |
| nor rc,ra,rb      | R    | $rc \leftarrow \sim(ra \mid rb)$                           |
| xori rb,ra,imm16  | I    | $rb \leftarrow ra \wedge imm16$                            |
| andi rb,ra,imm16  | I    | $rb \leftarrow ra \& imm16$                                |
| ori rb,ra,imm16   | I    | $rb \leftarrow ra \mid imm16$                              |
| andhi rb,ra,imm16 | I    | $rb \leftarrow ra \& ((imm16 \ll 16) \mid 0x00000000)$     |
| orhi rb,ra,imm16  | I    | $rb \leftarrow ra \mid ((imm16 \ll 16) \mid 0x00000000)$   |
| xorhi rb,ra,imm16 | I    | $rb \leftarrow ra \wedge ((imm16 \ll 16) \mid 0x00000000)$ |

Source: <http://people.irisa.fr/Simon.Rokicki/files/arcsys2/AideMemoire.pdf>



# Nios ISA



>> conditions, GOTOs

| Instruction      | Type | effet                |
|------------------|------|----------------------|
| beq ra,rb,imm16  | I    | PC+4+imm16 si rb=ra  |
| bne ra,rb,imm16  | I    | PC+4+imm16 si rb!=ra |
| blt ra,rb,imm16  | I    | PC+4+imm16 si ra<rb  |
| bltu ra,rb,imm16 | I    | PC+4+imm16 si ra<rb  |

# Nios ISA



## >> Arithmetic operations

| Instruction      | Type | effet                                          |
|------------------|------|------------------------------------------------|
| add rc,ra,rb     | R    | $rc \leftarrow ra + rb$                        |
| sub rc,ra,rb     | R    | $rc \leftarrow ra - rb$                        |
| addi rb,ra,imm16 | I    | $rb \leftarrow ra + \text{sext}(\text{imm16})$ |
| div rc,ra,rb     | R    | $rc \leftarrow ra / rb$                        |
| mul rc,ra,rb     | R    | $rc \leftarrow ra * rb$                        |

# Nios ISA



## >> Arithmetic operations

| Instruction      | Type | effet                                          |
|------------------|------|------------------------------------------------|
| add rc,ra,rb     | R    | $rc \leftarrow ra + rb$                        |
| sub rc,ra,rb     | R    | $rc \leftarrow ra - rb$                        |
| addi rb,ra,imm16 | I    | $rb \leftarrow ra + \text{sext}(\text{imm16})$ |
| div rc,ra,rb     | R    | $rc \leftarrow ra / rb$                        |
| mul rc,ra,rb     | R    | $rc \leftarrow ra * rb$                        |

Source: <http://people.irisa.fr/Simon.Rokicki/files/arcsys2/AideMemoire.pdf>

# Who forms of Nios II



|                                    | Nios II /f<br>Fast | Nios II /e<br>Economy   |
|------------------------------------|--------------------|-------------------------|
| Pipeline                           | 6 Stage            | None                    |
| H/W Multiplier &<br>Barrel Shifter | 1 Cycle            | Emulated<br>In Software |
| Branch Prediction                  | Dynamic            | None                    |
| Instruction Cache                  | Configurable       | None                    |
| Data Cache                         | Configurable       | None                    |
| Logic Usage (Logic<br>Elements)    | 1400 - 1800        | 600 - 700               |
| Custom<br>Instructions             | Up to 256          |                         |

# Considerations on Hardware Arithmetic

Different options for multiplier (at the processor design stage):

- **Nios II Economy version** - No Multiply Hardware
  - 32 x 32 in **250** Clock Cycles using GNUPro Math Library
- **Nios II Fast** - Full Hardware Multiplier
  - 32 x 32 in **1** Clock Cycles if DSP block present, else uses SW multiplier



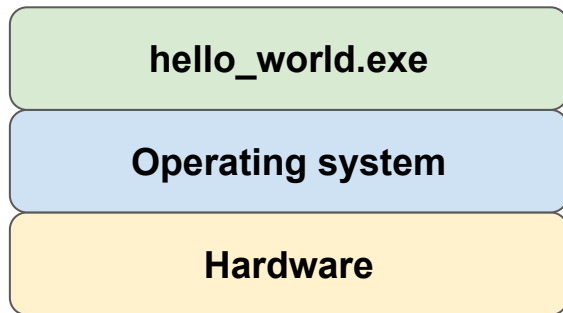
We discussed the processor, the microcode, the ISA, the machine code, the assembler, and the program ...



# The bare metal vs OS dilemma



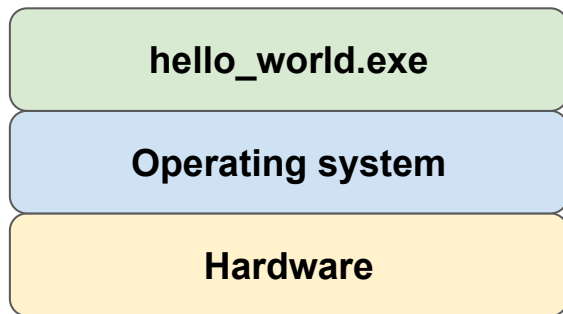
Programs, especially in SoCs, can either run on “BareMetal” or within as OS



# The bare metal vs OS dilemma



Programs, especially in SoCs, can either run on “BareMetal” or within as OS



## Having an OS insures

1. Portability
2. Threading support
3. **Memory safe operations**
4. Support ...

## Running bar metal provides

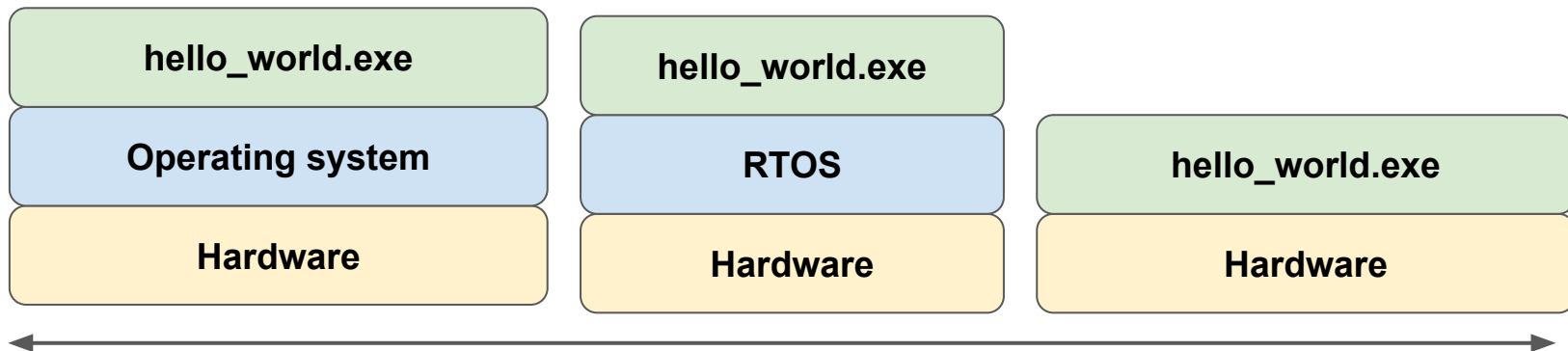
1. Low memory footprint
2. Maximum performance



# The bare metal vs OS dilemma



Programs, especially in SoCs, can either run on “BareMetal” or within as OS



## Having an OS insures

1. Portability
2. Threading support
3. **Memory safe operations**
4. Support ...

## Running bar metal provides

1. Low memory footprint
2. Maximum performance

# Linux on Nios II



## Embedded Linux for the Nios II Processor

With the introduction of a memory management unit (MMU) for the Nios® II processor, you have a wide range of Linux providers from which to choose. From commercial support from industry leaders to a free open-source distribution, Altera's embedded Linux partners offer the right level of support for your Linux project.

### Embedded Linux Distributions and Tools

Embedded Linux for the Nios II processor is supported by a comprehensive set of partners.

[Timesys](#)

# Linux on Nios II



## 2. Feature status on nios2 architecture

| Subsystem | Feature             | Kconfig                   | Status | Description                                                                     |
|-----------|---------------------|---------------------------|--------|---------------------------------------------------------------------------------|
| core      | cBPF-JIT            | HAVE_CBPF_JIT             | TODO   | arch supports cBPF JIT optimizations                                            |
| core      | eBPF-JIT            | HAVE_EBPF_JIT             | TODO   | arch supports eBPF JIT optimizations                                            |
| core      | generic-idle-thread | GENERIC_SMP_IDLE_THREAD   | TODO   | arch makes use of the generic SMP idle thread facility                          |
| core      | jump-labels         | HAVE_ARCH_JUMP_LABEL      | TODO   | arch supports live patched, high efficiency branches                            |
| core      | thread-info-in-task | THREAD_INFO_IN_TASK       | TODO   | arch makes use of the core kernel facility to embedd thread_info in task_struct |
| core      | tracehook           | HAVE_ARCH_TRACEHOOK       | ok     | arch supports tracehook (ptrace) register handling APIs                         |
| debug     | debug-vm-pgtable    | ARCH_HAS_DEBUG_VM_PGTABLE | TODO   | arch supports pgtable tests for semantics compliance                            |
| debug     | gcov-profile-all    | ARCH_HAS_GCOV_PROFILE_ALL | TODO   | arch supports whole-kernel GCOV code coverage profiling                         |
| debug     | KASAN               | HAVE_ARCH_KASAN           | TODO   | arch supports the KASAN runtime memory checker                                  |
| debug     | kcov                | ARCH_HAS_KCOV             | TODO   | arch supports kcov for coverage-guided fuzzing                                  |
| debug     | kgdb                | HAVE_ARCH_KGDB            | ok     | arch supports the kGDB kernel debugger                                          |
| debug     | kmemleak            | HAVE_DEBUG_KMEMLEAK       | TODO   | arch supports the kernel memory leak detector                                   |
| debug     | kprobes             | HAVE_KPROBES              | TODO   | arch supports live patched kernel probe                                         |
| debug     | kprobes-on-ftrace   | HAVE_KPROBES_ON_FTRACE    | TODO   | arch supports combined kprobes and ftrace live patching                         |
| debug     | kretprobes          | HAVE_KRETPROBES           | TODO   | arch supports kernel function-return probes                                     |
| debug     | optprobes           | HAVE_OPTPROBES            | TODO   | arch supports live patched optprobes                                            |
| debug     | stackprotector      | HAVE_STACKPROTECTOR       | TODO   | arch supports compiler driven stack overflow protection                         |

# Nios II Alternatives



- The latest Intel FPGA softcores (Nios-V) is based on RISC-V
- Royalty free and open-source soft-cores are available !
  - <https://github.com/SpinalHDL/VexRiscv>
  - <https://hackaday.com/2020/07/12/softcore-cpu-comparison>
- Vendor / device independent:
  - Supports on Intel, Xilinx and Lattice FPGAs
- Used in many open source projects
- Immature development stack (vs. QSYS, Platform designer ...)

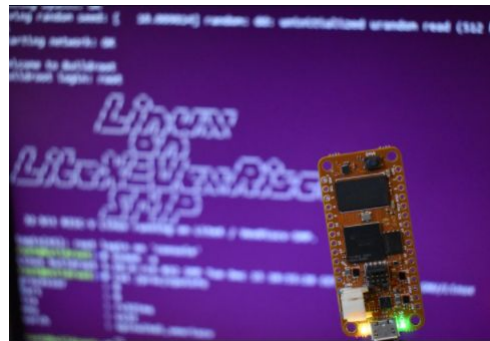
# Linux on an softcore LiteX system



In this repository, we experiment running Linux with **VexRiscv** CPU, a 32-bits Linux Capable RISC-V CPU written in Spinal HDL. A SoC around the VexRiscv CPU is created using LiteX as the SoC builder and LiteX's cores written in Migen Python DSL (LiteDRAM, LiteEth, LiteSDCard). All the components used to create the SoC are open-source and the flexibility of Spinal HDL/Migen allow targeting easily very various FPGA devices/boards: Lattice, Altera, Xilinx, Microsemi FPGAs with SDRAM/DDR/DDR2/DDR3/DDR4 RAMs, RMII/MII/RGMII/1000BASE-X Ethernet PHYs. On Lattice ECP5 FPGAs, the **open source toolchain** allows creating full open-source SoC with open-source cores **and** toolchain!

This project demonstrates how high level HDLs (Spinal HDL, Migen) enable new possibilities and complement each other. Results shown here are the results of a productive collaboration between open-source communities.

<https://github.com/litex-hub/linux-on-litex-vexriscv>





# Peripheral interfaces



# Inter-block communication

The communication between the different building blocks of a n SoC/SoPC have a huge influence on the overall performance of the circuit.

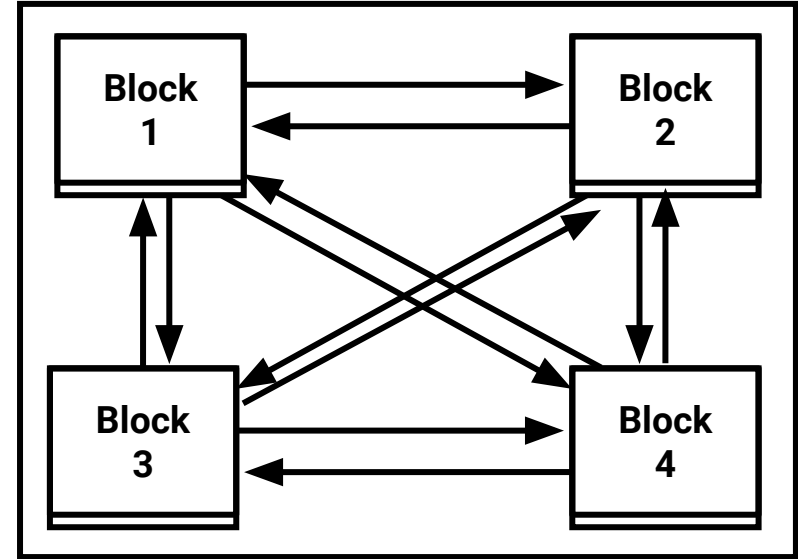
Three types of interface are available:

- Mesh network (crossbar)
- Shared bus (shared bus)
- Multi level network

# Crossbar interface

Point-to-point connection  
between each block of the  
system:

- High resource utilization
- High bandwidth



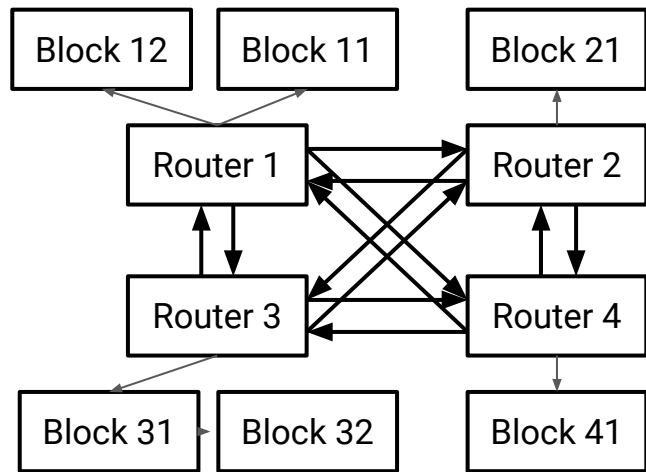


# Multi-level network

Level 1: Point-to-point connection between all the routers of the system

Level 2: Custom connection pattern between a router and a block

- Higher resource utilization
- High bandwidth



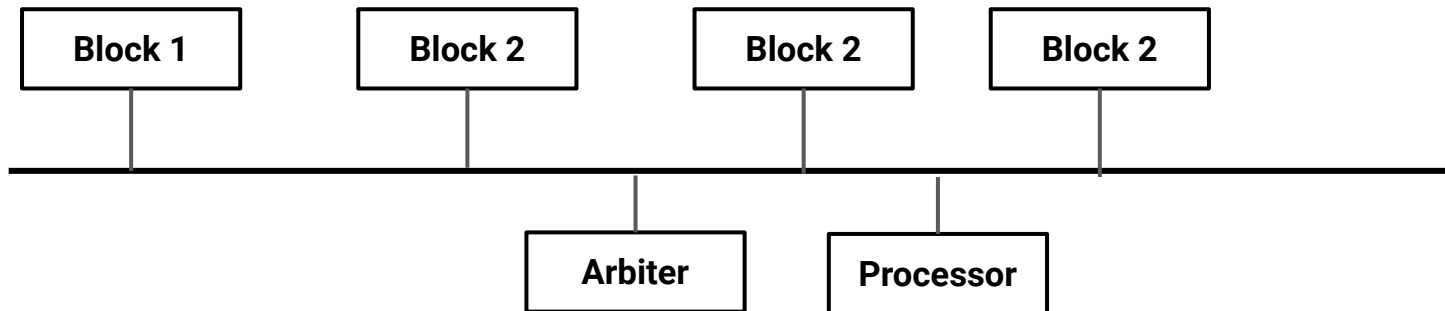
# Shared bus



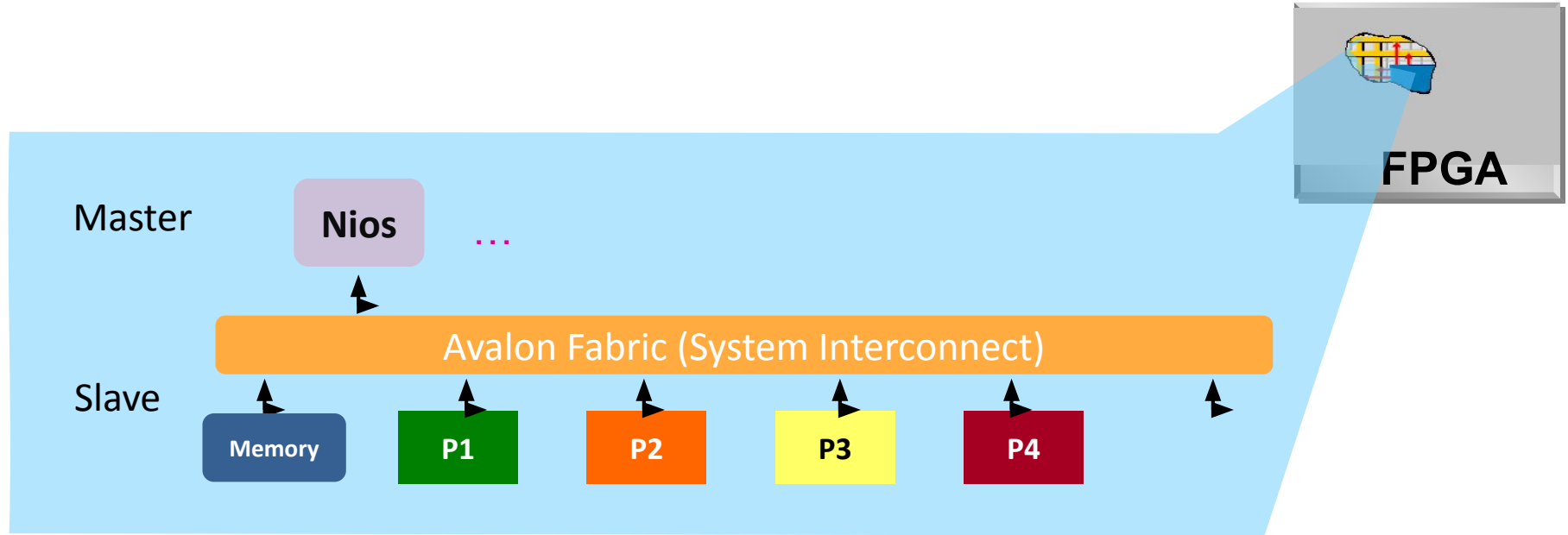
All the blocks share the same standard bus to communicate.

A single transaction allowed at a given time

- Low resource utilization
- Low bandwidth



# In our case ...





### III. Conception flow



# The AVALON bus

AVALON is a proprietary bus introduced by Intel FPGA

It follows master/slave config with support with 4 main interfaces:

- Memory mapped (AV-MM)
- Memory mapped tri state
- Streaming (AV-ST)
- Conduit interface

# Avalon Memory-Mapped



| Use                                 | Connect... | Module Name                                                 | Description                          | Clock | Base       | End        | Tags | IRQ |
|-------------------------------------|------------|-------------------------------------------------------------|--------------------------------------|-------|------------|------------|------|-----|
| <input checked="" type="checkbox"/> |            | <input checked="" type="checkbox"/> <b>cpu</b>              | Nios II Processor                    | clk   |            |            |      |     |
|                                     |            | instruction_master                                          | Avalon Memory Mapped Master          |       |            |            |      |     |
|                                     |            | data_master                                                 | Avalon Memory Mapped Master          |       |            |            |      |     |
|                                     |            | jtag_debug_module                                           | Avalon Memory Mapped Slave           |       | 0x01802800 | 0x01802fff |      |     |
| <input checked="" type="checkbox"/> |            | <input checked="" type="checkbox"/> <b>tri_state_bridge</b> | Avalon-MM Tristate Bridge            | clk   |            |            |      |     |
|                                     |            | avalon_slave                                                | Avalon Memory Mapped Slave           |       |            |            |      |     |
|                                     |            | tristate_master                                             | Avalon Memory Mapped Tristate Master |       |            |            |      |     |
| <input checked="" type="checkbox"/> |            | <input checked="" type="checkbox"/> <b>timer</b>            | Interval Timer                       | clk   | 0x01803000 | 0x0180301f |      |     |
|                                     |            | s1                                                          | Avalon Memory Mapped Slave           |       |            |            |      |     |
| <input checked="" type="checkbox"/> |            | <input checked="" type="checkbox"/> <b>onchip_memory</b>    | On-Chip Memory (RAM or ROM)          | clk   | 0x01801000 | 0x01801fff |      |     |
|                                     |            | s1                                                          | Avalon Memory Mapped Slave           |       |            |            |      |     |
| <input checked="" type="checkbox"/> |            | <input checked="" type="checkbox"/> <b>sdram</b>            | SDRAM Controller                     | clk   | 0x00800000 | 0x00ffffff |      |     |
|                                     |            | s1                                                          | Avalon Memory Mapped Slave           |       |            |            |      |     |
| <input checked="" type="checkbox"/> |            | <input checked="" type="checkbox"/> <b>cfi_flash</b>        | Flash Memory Interface (CFI)         | clk   | 0x01400000 | 0x017fffff |      |     |
|                                     |            | s1                                                          | Avalon Memory Mapped Tristate Slave  |       |            |            |      |     |
| <input checked="" type="checkbox"/> |            | <input checked="" type="checkbox"/> <b>switch_0_7</b>       | PIO (Parallel I/O)                   | clk   | 0x01803020 | 0x0180303f |      |     |
|                                     |            | s1                                                          | Avalon Memory Mapped Slave           |       |            |            |      |     |
| <input checked="" type="checkbox"/> |            | <input checked="" type="checkbox"/> <b>switch_8_15</b>      | PIO (Parallel I/O)                   | clk   | 0x01803060 | 0x0180306f |      |     |
|                                     |            | s1                                                          | Avalon Memory Mapped Slave           |       |            |            |      |     |

|                               | cpu.instruction_master  | cpu.data_master         |
|-------------------------------|-------------------------|-------------------------|
| cpu.jtag_debug_module         | 0x01802800 - 0x01802fff | 0x01802800 - 0x01802fff |
| onchip_memory.s1              | 0x01801000 - 0x01801fff | 0x01801000 - 0x01801fff |
| sdram.s1                      | 0x00800000 - 0x00ffffff | 0x00800000 - 0x00ffffff |
| jtag_uart.avalon_jtag_slave   |                         | 0x01803110 - 0x01803117 |
| switch_0_7.s1                 |                         | 0x01803020 - 0x0180302f |
| ledr_0_7.s1                   |                         | 0x01803030 - 0x0180303f |
| lcd.control_slave             |                         | 0x01803040 - 0x0180304f |
| seg_lcd_0.s1                  |                         | 0x01803050 - 0x0180305f |
| switch_8_15.s1                |                         | 0x01803060 - 0x0180306f |
| ledr_8_15.s1                  |                         | 0x01803070 - 0x0180307f |
| seg_lcd_1.s1                  |                         | 0x01803080 - 0x0180308f |
| seg_lcd_2.s1                  |                         | 0x01803090 - 0x0180309f |
| seg_lcd_3.s1                  |                         | 0x018030a0 - 0x018030af |
| cfi_flash.s1                  | 0x01400000 - 0x017fffff | 0x01400000 - 0x017fffff |
| tri_state_bridge.avalon_slave |                         |                         |
| timer.s1                      |                         | 0x01803000 - 0x0180301f |
| push_button_0_3.s1            |                         | 0x018030b0 - 0x018030bf |
| ledg_0_7.s1                   |                         | 0x018030c0 - 0x018030cf |
| seg_lcd_4.s1                  |                         | 0x018030d0 - 0x018030df |
| seg_lcd_5.s1                  |                         | 0x018030e0 - 0x018030ef |
| seg_lcd_6.s1                  |                         | 0x018030f0 - 0x018030ff |
| seg_lcd_7.s1                  |                         | 0x01803100 - 0x0180310f |

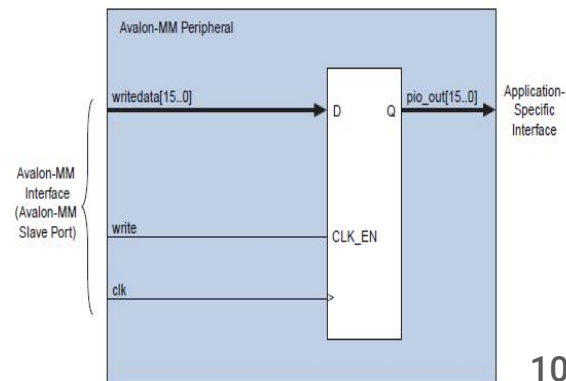
Each peripheral is address mapped:  
Controlling a peripheral consists in writing  
or reading at these addresses.

# AVALON bus: Memory mapped interfaces



- Can be configured in minimal mode
- Can have a variable data bus
- Support arbiter configuration
- Support data burst mode
- Can be configured as masters or slaves

| Signal Type                | Width                                   | Dir | Req'd | Description                                                                                                                                                                                                                                                  |
|----------------------------|-----------------------------------------|-----|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Fundamental Signals</b> |                                         |     |       |                                                                                                                                                                                                                                                              |
| read<br>read_n             | 1                                       | In  | No    | Asserted to indicate a read transfer. If present, readdata is required.                                                                                                                                                                                      |
| write<br>write_n           | 1                                       | In  | No    | Asserted to indicate a write transfer. If present, writedata is required.                                                                                                                                                                                    |
| address                    | 1-32                                    | In  | No    | Specifies an offset into the slave address space. Each slave address value selects a word of slave data. For example, address=0 selects the first <slave data width> bits of slave data; address=1 selects the second <slave data width> bits of slave data. |
| readdata                   | 8,16,32,<br>64,128,<br>256,512,<br>1024 | Out | No    | The readdata provided by the slave in response to a read transfer.                                                                                                                                                                                           |



# You can write your own AV-MM peripherals



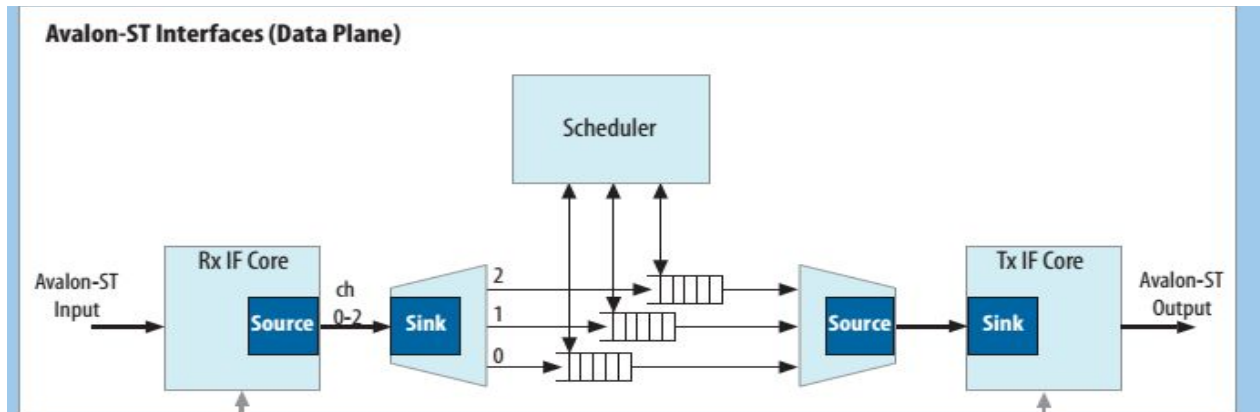
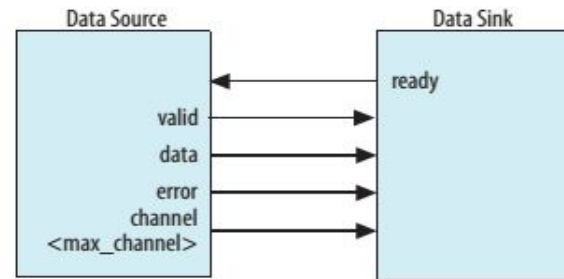
```
entity dummy is
  generic (
    REG_LENGTH  : positive := 32;
    CMD_LENGTH  : positive := 8; --useless for now
    VAL_LENGTH  : positive := 24 --useless for now
  );
  port(
    -- Clocks and resets
    clk_i      : in  std_logic;
    reset_n_i  : in  std_logic;
    -- Nios AV-MM slave
    nios_addr_rel_i : in  std_logic_vector(1 downto 0);
    nios_wr_i      : in  std_logic;
    nios_rd_i      : in  std_logic;
    nios_datawr_i  : in  std_logic_vector(REG_LENGTH - 1 downto 0);
    nios_datard_o  : out std_logic_vector(REG_LENGTH - 1 downto 0);
    -- Host AV-MM slave
    host_addr_rel_i : in  std_logic_vector(1 downto 0);
    host_wr_i       : in  std_logic;
    host_rd_i       : in  std_logic;
    host_datawr_i   : in  std_logic_vector(REG_LENGTH - 1 downto 0);
    host_datard_o   : out std_logic_vector(REG_LENGTH - 1 downto 0);
    -- Interrupts
    irq_o          : out std_logic
  );
end dummy;
```



# AVALON bus- Streaming interfaces



- Are designed for point-to-point communication
- Typically works on dataflow
- Implement time mux of data



# You can write your own AV-ST peripherals



```
entity d is
  generic(
    BITWIDTH_PIXEL : positive := 16;
  );
  port (
    -- System clock
    clk_sys      : in  std_logic;
    reset_n      : in  std_logic;
    --AVST in video stream interface
    pixel_i      : in  std_logic_vector(BITWIDTH_PIXEL -1 downto 0);
    dv_i         : in  std_logic;
    sop_i        : in  std_logic;
    eop_i        : in  std_logic;
    --AVST out video stream interface
    pixel_o      : out std_logic_vector(BITWIDTH_PIXEL -1 downto 0);
    dv_o         : out std_logic;
    sop_o        : out std_logic;
    eop_o        : out std_logic;
    --AVMM interface
    addr_rel_i   : in  std_logic_vector(2 downto 0);
    wr_i         : in  std_logic;
    datawr_i     : in  std_logic_vector(31 downto 0);
    rd_i        : in  std_logic;
    datard_o     : out std_logic_vector(31 downto 0)
  );
end d;
```

# AMBA and AXI Bus: A short overview



The ARM Advanced Microcontroller Bus Architecture (AMBA) and its sub parts:  
**Advanced eXtensible Interface (AXI)**



Figure 5-3: AXI Memory-mapped Interface

Avalon has a single channel for data, address, and control, as shown in Figure 5-4.

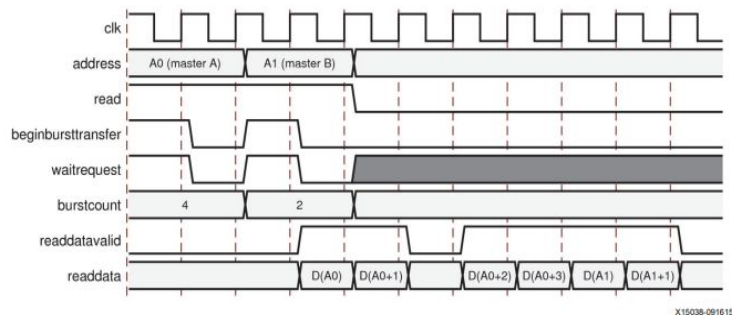


Figure 5-4: Avalon Memory-mapped Interface

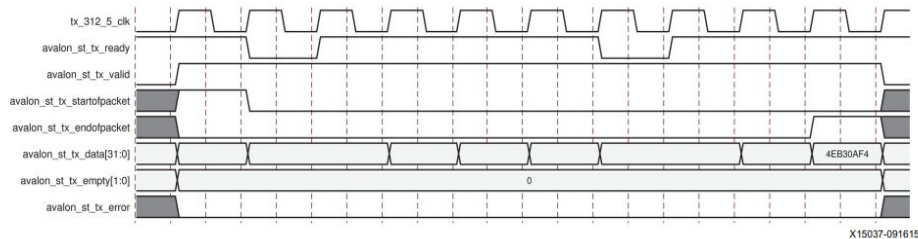


Figure 5-1: Avalon ST Interface

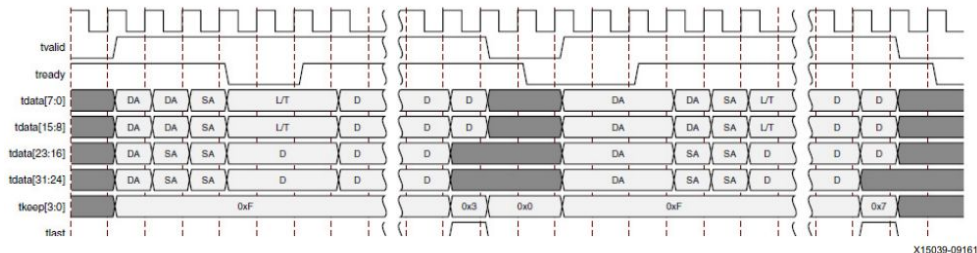


Figure 5-2: AXI ST Interface

# Intellectual Property (IP) Blocks



“Predesigned, preverified silicon circuit block, usually containing 5000 gates, that can be used in building larger application on a semiconductor chip”



# Intellectual Property (IP) Blocks



## Pros:

- Performance !
- Blocks are often optimized, validated and tested

## Cons:

- Cost
- Difficulty interfacing with the rest of the system
- Complexity of certain user licenses

# Intellectual Property (IP) Blocks



The screenshot shows the OpenCores website homepage. At the top is the OpenCores logo and a navigation bar. Below the logo is a login/register section with fields for Username and Password, and buttons for Login, Register, and Remember me. A 'Browse' section lists various categories like Projects, Forums, About, HowTo/FAQ, Media, Licensing, Commerce, Partners, Maintainers, and Contact Us. A 'Tools' section includes a Google search bar. The main content area is divided into several sections: 'What is OpenCores?' which describes the community and its mission; 'Registered OpenCores users' showing a count of 340186; 'Last updated projects' listing recent updates; 'Most popular projects' listing popular cores; 'Projects', 'Forum', and 'WebShop' sections with icons and links; and 'Professional support' from oliscience, which offers development services.

**OpenCores**  
www.opencores.org

Username:  
Password:  
☐ Remember me  
[Login](#)  
[Register](#)

**Browse**

- [PROJECTS](#)
- [FORUMS](#)
- [ABOUT](#)
- [HowTo/FAQ](#)
- [MEDIA](#)
- [LICENSING](#)
- [COMMERCE](#)
- [PARTNERS](#)
- [MAINTAINERS](#)
- [CONTACT US](#)

**Tools**  
ENHANCED BY Google

**What is OpenCores?**

The reference community for Free and Open Source gateway IP cores

Since 1999, OpenCores is the most prominent online community for the development of gateway IP (Intellectual Properties) Cores. It is the place where such cores are shared and promoted in the spirit of Free and Open Source collaboration.

The OpenCores portal hosts the source code for different digital gateway projects and supports the users' community providing a platform for listing, presenting, and managing such projects; together with version control systems for sources management.

OpenCores is also the place where digital designers meet to showcase, promote, and talk about their passion and work. They do this through forums, news collectors, and much more!

Please join us!

**Registered OpenCores users**

**340186**  
[OpenCores statistics](#)

**Last updated projects**

- [stack\\_calculator](#)
- [Taar Microprocessor](#)
- [openVeriFLA - FPGA logic analyzer](#)
- [ZAP](#)
- [Generic AHB Master](#)
- [The HEORV32 Processor \(RISC-V\)](#)
- [Wupper: PCIe DMA Engine for Xilinx FPGAs](#)
- [Open8 uRISC](#)

**Most popular projects**

- [I2C controller core](#)
- [I2C Slave](#)
- [SPI Verilog Master & Slave modules](#)
- [SPI Master/Slave Interface](#)
- [I2C master/slave Core](#)
- [CAN Protocol Controller](#)
- [SPI core](#)
- [I2C Master Slave Core](#)

**Projects**  
[Browse all Projects \(Cores\)](#)

**Forum**  
[Communicate in the forums](#)

**WebShop**  
[Visit our Webshop](#)

**Professional support**

**oliscience**  
open logic. interconnects science

**We are the developers and maintainers of this website and community, but not only!**

If you plan to use IP Cores from OpenCores in your next design and need support, or if you require professional advise on your next challenging IP Core development, don't hesitate to contact us.

We are experts in gateway design and engineering based on the OpenCores technology, and have extensive experience in all parts of FPGA development.

Please visit [Oliscience](#) for further information and enquiries.

<https://opencores.org/projects/i2c>

<https://opencores.org/projects/pwm>



# Workload partitioning

Always device your system into basic elements which, when assembled, allow the expected functionalities to be achieved.

- The software provides flexibility at the price of low dev time
- The hardware provides high performance at the price of high dev time

**Disclaimer: Some tasks are faster to implement in hardware**



# Workload partitioning

Always device your system into basic elements which, when assembled, allow the expected functionalities to be achieved.

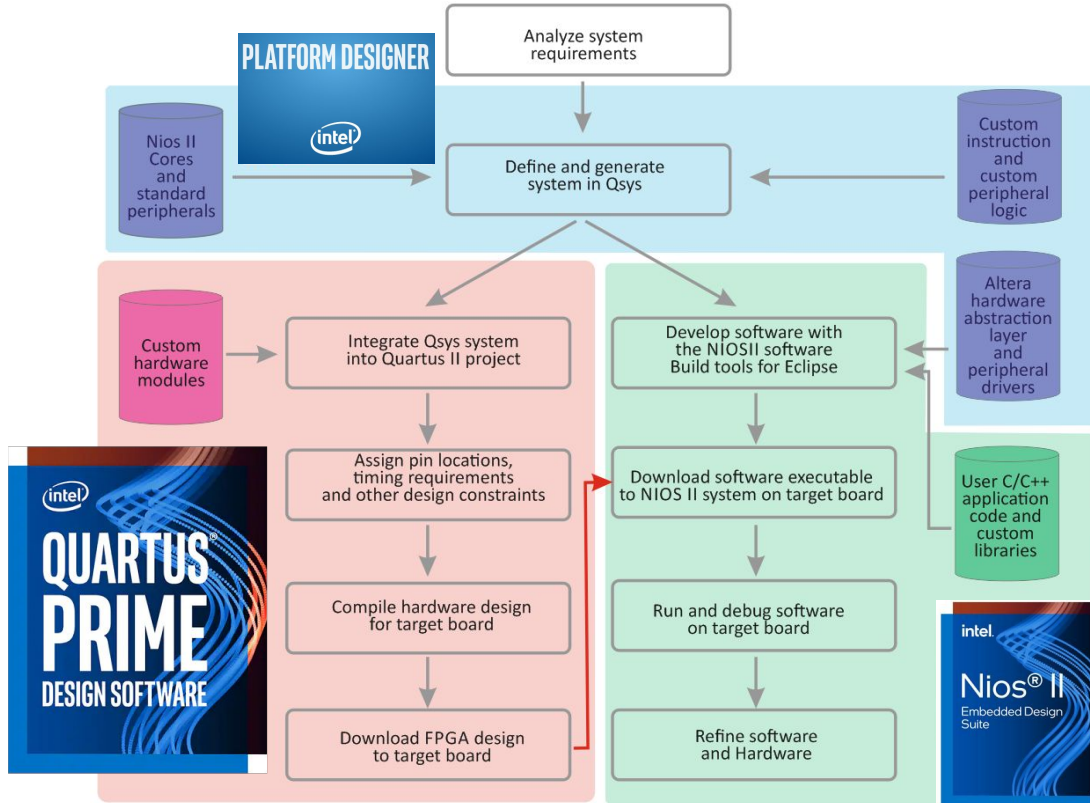
- The software provides flexibility at the price of low dev time
- The hardware provides high performance at the price of high dev time
- Use your common sense, don't spend 3 months developing a hardware block that would provide +0.5% acceleration ...
- Ideally, start from a full software implementation, then **PROFILE !!**
- Remember: IP Blocks exist





**Practice time**

# SoPC design flow on Intel FPGA suite



# System design with platform designer



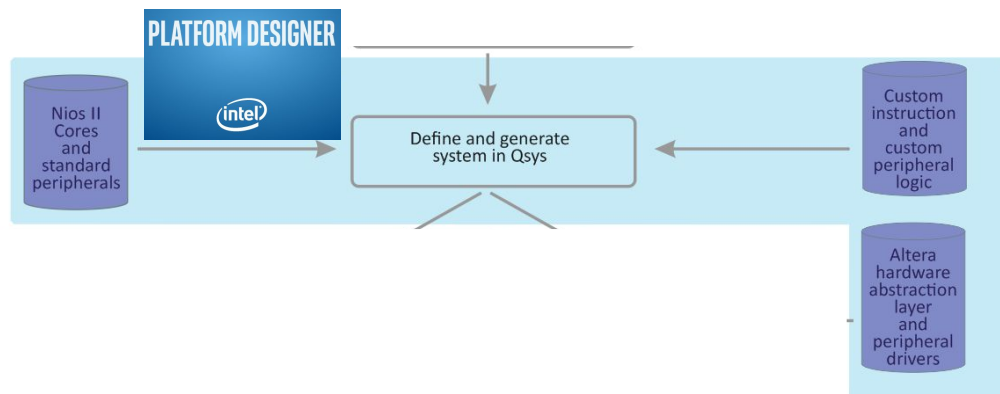
System design follows a **Lego's like** approach with Platform Designer (formally known as QSYS)

Platform designer builds SoPC hardware easily

## Alternative:

- Built-in Vivado in Xilinx
- Opensource: LiteX

<https://github.com/enjoy-digital/litex>



# System design with platform designer



Qsys - PWM.qsys\* (C:\Pise\Lab9\_PWM\PWM.qsys)

File Edit System View Tools Help

Component Library

Project

- New Component...
- alu\_avalon
- System
- Library
- Bridges
- Clock and Reset
- Configuration & Programming
- DSP
- Embedded Processors
- Interface Protocols
- Memories and Memory Controller
- Merlin Components
- Microcontroller Peripherals
- Peripherals
- PLL
- Qsys Interconnect
- SLS
- University Program
- Verification
- Window Bridge

System Contents Address Map Clock Settings Project Settings Instance Parameters System Inspector HDL Example Generation

| Use                                 | Connections | Name                    | Description                 | Export                 | Clock  | Base        | End         | IRQ   | Tags   |
|-------------------------------------|-------------|-------------------------|-----------------------------|------------------------|--------|-------------|-------------|-------|--------|
| <input checked="" type="checkbox"/> |             | clk_0                   | Clock Source                |                        |        |             |             |       |        |
|                                     |             | clk_in                  | Clock Input                 |                        |        |             |             |       |        |
|                                     |             | clk_in_reset            | Reset Input                 |                        |        |             |             |       |        |
|                                     |             | clk                     | Clock Output                |                        |        |             |             |       |        |
|                                     |             | clk_reset               | Reset Output                |                        |        |             |             |       |        |
| <input checked="" type="checkbox"/> |             | nios2_qsys_0            | Nios II Processor           |                        |        |             |             |       |        |
|                                     |             | clk                     | Clock Input                 | Double-click to export | clk_0  |             |             |       |        |
|                                     |             | reset_n                 | Reset Input                 | Double-click to export | [clk]  |             |             |       |        |
|                                     |             | data_master             | Avalon Memory Mapped Master | Double-click to export | [clk]  |             |             |       |        |
|                                     |             | instruction_master      | Avalon Memory Mapped Master | Double-click to export | [clk]  |             |             |       |        |
|                                     |             | d_irq                   | Interrupt Receiver          | Double-click to export | [clk]  |             |             |       |        |
|                                     |             | jtag_debug_module_re... | Reset Output                | Double-click to export | [clk]  |             |             |       |        |
|                                     |             | jtag_debug_module       | Avalon Memory Mapped Slave  | Double-click to export | [clk]  | 0x0001_0800 | 0x0001_0fff | IRQ 0 | IRQ 31 |
|                                     |             | custom_instruction_m... | Custom Instruction Master   | Double-click to export | [clk]  |             |             |       |        |
| <input checked="" type="checkbox"/> |             | jtag_uart_0             | JTAG UART                   |                        |        |             |             |       |        |
|                                     |             | clk                     | Clock Input                 | Double-click to export | clk_0  |             |             |       |        |
|                                     |             | reset                   | Reset Input                 | Double-click to export | [clk]  |             |             |       |        |
|                                     |             | avalon_jtag_slave       | Avalon Memory Mapped Slave  | Double-click to export | [clk]  | 0x0001_1000 | 0x0001_1007 |       |        |
|                                     |             | irq                     | Interrupt Sender            | Double-click to export | [clk]  |             |             |       |        |
| <input checked="" type="checkbox"/> |             | onchip_memory2_0        | On-Chip Memory (RAM or ROM) |                        |        |             |             |       |        |
|                                     |             | clk1                    | Clock Input                 | Double-click to export | clk_0  |             |             |       |        |
|                                     |             | s1                      | Avalon Memory Mapped Slave  | Double-click to export | [clk1] | 0x0000_8000 | 0x0000_fcff |       |        |
|                                     |             | reset1                  | Reset Input                 | Double-click to export | [clk1] |             |             |       |        |

Messages

| Description                                          | Path                    |
|------------------------------------------------------|-------------------------|
| 1 Info Message                                       |                         |
| Memory will be initialized from onchip_memory2_0.hex | System.onchip_memory2_0 |

0 Errors, 0 Warnings

# System design with platform designer



Select your component

Qsys - PWM.qsys\* (C:\Pise\Lab9\_PWM\PWM.qsys)

Tools Help

System Contents Address Map Clock Settings Project Settings Instance Parameters System Inspector HDL Example Generation

Use Connections Name Description Export Clock Base End IRQ Tags

clk\_0  
clk\_in  
clk\_in\_reset  
clk  
clk\_reset  
Double-click to export  
Double-click to export  
clk\_0

nios2\_qsys\_0  
clk  
reset\_n  
data\_master  
instruction\_master  
d\_irq  
jtag\_debug\_module\_re...  
jtag\_debug\_module  
custom\_instruction\_m...  
Double-click to export  
Double-click to export  
Double-click to export  
Double-click to export  
Double-click to export  
Double-click to export  
clk\_0  
[clk]  
[clk]  
[clk]  
[clk]  
[clk]  
IRQ 0  
IRQ 31  
0x0001\_0800  
0x0001\_0fff

jtag\_uart\_0  
clk  
reset  
avalon\_jtag\_slave  
irq  
Double-click to export  
Double-click to export  
Double-click to export  
Double-click to export  
clk\_0  
[clk]  
[clk]  
[clk]  
0x0001\_1000  
0x0001\_1007

onchip\_memory2\_0  
clk1  
s1  
reset1  
Double-click to export  
Double-click to export  
Double-click to export  
clk\_0  
[clk1]  
[clk1]  
0x0000\_8000  
0x0000\_fcff

New... Edit... Add...

Messages

| Description                                          | Path                    |
|------------------------------------------------------|-------------------------|
| 1 Info Message                                       |                         |
| Memory will be initialized from onchip_memory2_0.hex | System.onchip_memory2_0 |

0 Errors, 0 Warnings

# System design with IPD



Select your component

The screenshot shows the Qsys software interface with the title bar "Qsys - PWM.qsys\* (C:\Pise\Lab9\_PWM\PWM.qsys)". The main window is divided into several panes. On the left, the "Library" pane lists various components under "System" and "Library" categories. A blue callout box with the text "Select your component" points to this pane. The central pane shows a "Connections" diagram with lines connecting components. The right pane displays a table of components and their properties.

| Use                                 | Connections | Name                    | Description                 | Export                 | Clock | Base        | End         | IRQ   | Tags   |
|-------------------------------------|-------------|-------------------------|-----------------------------|------------------------|-------|-------------|-------------|-------|--------|
| <input checked="" type="checkbox"/> |             | clk_0                   | Clock Source                |                        |       |             |             |       |        |
| <input checked="" type="checkbox"/> |             | clk_in                  | Clock Input                 | clk                    |       |             |             |       |        |
| <input checked="" type="checkbox"/> |             | clk_in_reset            | Reset Input                 | reset                  |       |             |             |       |        |
| <input checked="" type="checkbox"/> |             | clk                     | Clock Output                | Double-click to export | clk_0 |             |             |       |        |
| <input checked="" type="checkbox"/> |             | clk_reset               | Reset Output                | Double-click to export |       |             |             |       |        |
| <input checked="" type="checkbox"/> |             | nios2_qsys_0            | Nios II Processor           |                        |       |             |             |       |        |
| <input checked="" type="checkbox"/> |             | clk                     | Clock Input                 | Double-click to export | clk_0 |             |             |       |        |
| <input checked="" type="checkbox"/> |             | reset_n                 | Reset Input                 | Double-click to export | [clk] |             |             |       |        |
| <input checked="" type="checkbox"/> |             | data_master             | Avalon Memory Mapped Master | Double-click to export | [clk] |             |             |       |        |
| <input checked="" type="checkbox"/> |             | instruction_master      | Avalon Memory Mapped Master | Double-click to export | [clk] |             |             |       |        |
| <input checked="" type="checkbox"/> |             | d_irq                   | Interrupt Receiver          | Double-click to export | [clk] |             |             |       |        |
| <input checked="" type="checkbox"/> |             | jtag_debug_module_re... | Reset Output                | Double-click to export | [clk] |             |             |       |        |
| <input checked="" type="checkbox"/> |             | jtag_debug_module       | Avalon Memory Mapped Slave  | Double-click to export | [clk] | 0x0001_0800 | 0x0001_0fff | IRQ 0 | IRQ 31 |
| <input checked="" type="checkbox"/> |             | custom_instruction_m... | Custom Instruction Master   | Double-click to export |       |             |             |       |        |
| <input checked="" type="checkbox"/> |             | jtag_uart_0             | JTAG UART                   |                        |       |             |             |       |        |
| <input checked="" type="checkbox"/> |             | clk                     | Clock Input                 | Double-click to export | clk_0 |             |             |       |        |
| <input checked="" type="checkbox"/> |             | reset                   | Reset Input                 | Double-click to export | [clk] |             |             |       |        |
| <input checked="" type="checkbox"/> |             | avalon_jtag_slave       | Avalon Memory Mapped Slave  | Double-click to export | [clk] | 0x0001_1000 | 0x0001_1007 |       |        |
| <input checked="" type="checkbox"/> |             | onchip_memory2_0        | On-Chip Memory              |                        |       |             |             |       |        |
| <input checked="" type="checkbox"/> |             | clk1                    | Clock                       |                        |       |             |             |       |        |
| <input checked="" type="checkbox"/> |             | s1                      | Avalon Memory Mapped Slave  |                        |       |             |             |       |        |
| <input checked="" type="checkbox"/> |             | reset1                  | Reset                       |                        |       |             |             |       |        |

Components can be:

- IPs provided by Intel
- IPs you purchased
- Opencore IPs
- Written by yourself (with AV-MM,AV-ST compliance)

Messages

1 Info Message

Memory will be initialized from onchip\_memory2\_0.hex

0 Errors, 0 Warnings



## Instantiate





# System design with platform designer



Select your component

Instantiate

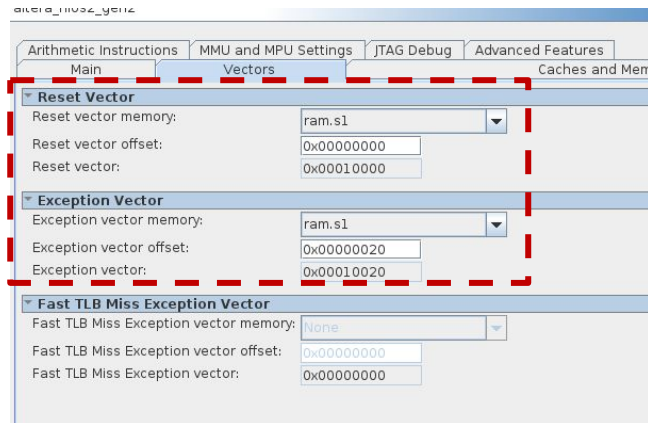
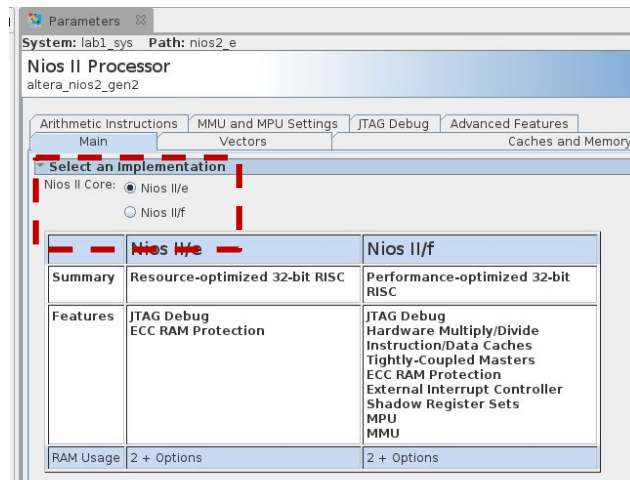
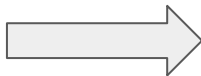
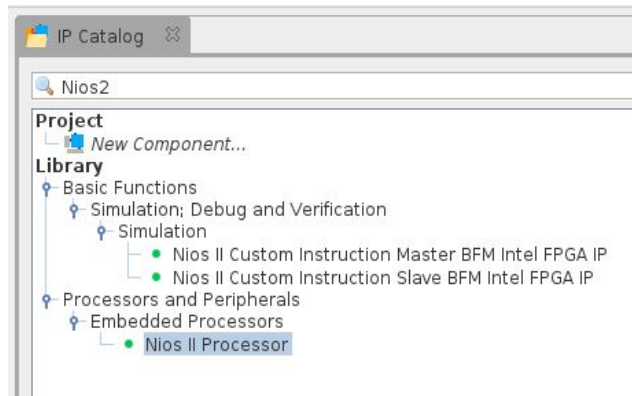
The screenshot shows the Qsys Platform Designer interface. On the left, the 'Library' pane lists various components. A blue arrow points from the 'Select your component' text to the 'Library' pane. In the center, the 'Connections' pane shows a block diagram with components like 'clk\_0', 'nios2\_qsys\_0', 'jtag\_uart\_0', and 'onchip\_memory2\_0'. A red oval highlights the 'nios2\_qsys\_0' component, and a red arrow points from the 'Instantiate' text to it. On the right, the 'Parameters' pane shows a table of component parameters.

| Component        | Parameter                   | Value |
|------------------|-----------------------------|-------|
| clk_0            | Clock Source                | clk_0 |
|                  | Clock Input                 | clk_0 |
|                  | Reset Input                 | [clk] |
|                  | Reset Output                | [clk] |
| nios2_qsys_0     | Clock Input                 | clk_0 |
|                  | Reset Input                 | [clk] |
|                  | Avalon Memory Mapped Master | [clk] |
|                  | Avalon Memory Mapped Master | [clk] |
| jtag_uart_0      | Clock Input                 | clk_0 |
|                  | Reset Input                 | [clk] |
|                  | Avalon Memory Mapped Slave  | [clk] |
|                  | Custom Instruction Master   | [clk] |
| onchip_memory2_0 | Clock                       | clk_0 |
|                  | Reset                       | [clk] |
|                  | Avalon Memory Mapped Slave  | [clk] |
|                  | Interrupt                   | [clk] |

Once a component is selected, a configuration window can appear. It provides configs to apply the components



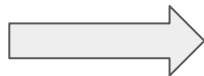
# Basic Platform designer IPs: Nios2



Pay attention to:

1. The Implementation of the core
2. The reset and instruction vectors
3. The wiring of your component

# Basic Platform designer IPs: On-chip memory



On-Chip Memory (RAM or ROM) Intel FPGA IP  
altera\_avalon\_onchip\_memory2

Details

**Memory type**

Type: RAM (Writable) ▼

☐ Dual-port access

☐ Single clock operation

Read During Write Mode: DONT\_CARE ▼

Block type: AUTO ▼

**Size**

☐ Enable different width for Dual-port access

Slave S1 Data width: 32 ▼

Total memory size: 40000 bytes

☐ Minimize memory block usage (may impact fmax)

**Read latency**

Slave s1 Latency: 1 ▼

Slave s2 Latency: 1 ▼

**ROM/RAM Memory Protection**

Reset Request: Enabled ▼

**ECC Parameter**

Extend the data width to support ECC bits: Disabled ▼

**Memory initialization**

☐ Initialize memory content

☐ Enable non-default initialization file

Type the filename (e.g. my\_ram.hex) or select the hex file using the file browser button.

User created initialization file: onchip\_memory2.hex

☐ Enable Partial Reconfiguration Initialization Mode

☐ Enable In-System Memory Content Editor feature

Instance ID: NONE

Pay attention to:

1. The size of your RAM
2. Initialization

# System design with platform designer



Select your component

Instantiate

The screenshot displays the Qsys Platform Designer interface for a project named 'PWM.qsys'. The interface is divided into several panes:

- Library:** A list of components available for instantiation, including System, alu\_avalon, Bridges, Clock and Reset, Configuration & Programming, DSP, Embedded Processors, Interface Protocols, Memories and Memory Controllers, Merlin Components, Microcontroller Peripherals, Peripherals, PLL, Qsys Interconnect, SLS, University Program, Verification, and Window Bridge.
- Connections:** A diagram showing the interconnection of components. A red oval highlights the 'nios2\_qsys\_0' component, which is being instantiated. A blue arrow points from the 'Library' pane to the 'Connections' pane, and a red arrow points from the 'Connections' pane to the 'Messages' pane.
- Parameters:** A table listing the parameters for the instantiated components. The table has columns for Name, Description, Export, Clock, Base, End, IRQ, and Tags.
- Messages:** A pane showing system messages. A message indicates that memory will be initialized from onchip\_memory2\_0.hex.

| Name                    | Description                 | Export                 | Clock  | Base        | End         | IRQ   | Tags   |
|-------------------------|-----------------------------|------------------------|--------|-------------|-------------|-------|--------|
| clk_0                   | Clock Source                |                        |        |             |             |       |        |
| clk_in                  | Clock Input                 |                        |        |             |             |       |        |
| clk_in_reset            | Reset Input                 |                        |        |             |             |       |        |
| clk                     | Clock Output                | clk                    | clk_0  |             |             |       |        |
| clk_reset               | Reset Output                | reset                  |        |             |             |       |        |
| nios2_qsys_0            | Nios II Processor           |                        |        |             |             |       |        |
| clk                     | Clock Input                 | Double-click to export | clk_0  |             |             |       |        |
| reset_n                 | Reset Input                 | Double-click to export | [clk]  |             |             |       |        |
| data_master             | Avalon Memory Mapped Master | Double-click to export | [clk]  |             |             |       |        |
| instruction_master      | Avalon Memory Mapped Master | Double-click to export | [clk]  |             |             |       |        |
| d_irq                   | Interrupt Receiver          | Double-click to export | [clk]  |             |             |       |        |
| jtag_debug_module_re... | Reset Output                | Double-click to export | [clk]  |             |             |       |        |
| jtag_debug_module       | Avalon Memory Mapped Slave  | Double-click to export | [clk]  | 0x0001_0800 | 0x0001_0fff | IRQ 0 | IRQ 31 |
| custom_instruction_m... | Custom Instruction Master   | Double-click to export | [clk]  |             |             |       |        |
| jtag_uart_0             | JTAG UART                   |                        |        |             |             |       |        |
| clk                     | Clock Input                 | Double-click to export | clk_0  |             |             |       |        |
| reset                   | Reset Input                 | Double-click to export | [clk]  |             |             |       |        |
| avalon_jtag_slave       | Avalon Memory Mapped Slave  | Double-click to export | [clk]  | 0x0001_1000 | 0x0001_1007 |       |        |
| irq                     | Interrupt Sender            | Double-click to export | [clk]  |             |             |       |        |
| onchip_memory2_0        | On-Chip Memory (RAM or ROM) |                        |        |             |             |       |        |
| clk1                    | Clock Input                 | Double-click to export | clk_0  |             |             |       |        |
| s1                      | Avalon Memory Mapped Slave  | Double-click to export | [clk1] | 0x0000_8000 | 0x0000_fcff |       |        |
| reset1                  | Reset Input                 | Double-click to export | [clk1] |             |             |       |        |

Messages

| Description                                          | Path                    |
|------------------------------------------------------|-------------------------|
| 1 Info Message                                       |                         |
| Memory will be initialized from onchip_memory2_0.hex | System.onchip_memory2_0 |

0 Errors, 0 Warnings

# System conception with platform designer



Select your component

Instantiate

Connect

Qsys - PWM.qsys\* (C:\Pise\Lab9\_PWM\PWM.qsys)

System Contents | Address Map | Parameters | System Inspector | HDL Example | Generation

Library

- alu\_avalon...
- System
- Bridges
- Clock and Reset
- Configuration & Programming
- DSP
- Embedded Processors
- Interface Protocols
- Memories and Memory Controllers
- Merlin Components
- Microcontroller Peripherals
- Peripherals
- PLL
- Qsys Interconnect
- SLS
- University Program
- Verification
- Window Bridge

Connections

| Use                                 | Connections | Name                    | Description                 | Export                 | Clock  | Base        | End         | IRQ   | Tags   |
|-------------------------------------|-------------|-------------------------|-----------------------------|------------------------|--------|-------------|-------------|-------|--------|
| <input checked="" type="checkbox"/> |             | clk_0                   | Clock Source                |                        |        |             |             |       |        |
| <input checked="" type="checkbox"/> |             | clk_in                  | Clock Input                 |                        |        |             |             |       |        |
| <input checked="" type="checkbox"/> |             | clk_in_reset            | Reset Input                 |                        |        |             |             |       |        |
| <input checked="" type="checkbox"/> |             | clk                     | Clock Output                | clk                    | clk_0  |             |             |       |        |
| <input checked="" type="checkbox"/> |             | clk_reset               | Reset Output                | reset                  |        |             |             |       |        |
| <input checked="" type="checkbox"/> |             | nios2_qsys_0            | Nios II Processor           |                        |        |             |             |       |        |
| <input checked="" type="checkbox"/> |             | clk                     | Clock Input                 | Double-click to export | clk_0  |             |             |       |        |
| <input checked="" type="checkbox"/> |             | reset_n                 | Reset Input                 | Double-click to export | [clk]  |             |             |       |        |
| <input checked="" type="checkbox"/> |             | data_master             | Avalon Memory Mapped Master | Double-click to export | [clk]  |             |             |       |        |
| <input checked="" type="checkbox"/> |             | instruction_master      | Avalon Memory Mapped Master | Double-click to export | [clk]  |             |             |       |        |
| <input checked="" type="checkbox"/> |             | d_irq                   | Interrupt Receiver          | Double-click to export | [clk]  |             |             |       |        |
| <input checked="" type="checkbox"/> |             | jtag_debug_module_re... | Reset Output                | Double-click to export | [clk]  |             |             |       |        |
| <input checked="" type="checkbox"/> |             | jtag_debug_module       | Avalon Memory Mapped Slave  | Double-click to export | [clk]  | 0x0001_0800 | 0x0001_0fff | IRQ 0 | IRQ 31 |
| <input checked="" type="checkbox"/> |             | custom_instruction_m... | Custom Instruction Master   | Double-click to export | [clk]  |             |             |       |        |
| <input checked="" type="checkbox"/> |             | jtag_uart_0             | JTAG UART                   |                        |        |             |             |       |        |
| <input checked="" type="checkbox"/> |             | clk                     | Clock Input                 | Double-click to export | clk_0  |             |             |       |        |
| <input checked="" type="checkbox"/> |             | reset                   | Reset Input                 | Double-click to export | [clk]  |             |             |       |        |
| <input checked="" type="checkbox"/> |             | avalon_jtag_slave       | Avalon Memory Mapped Slave  | Double-click to export | [clk]  | 0x0001_1000 | 0x0001_1007 |       |        |
| <input checked="" type="checkbox"/> |             | irq                     | Interrupt Sender            | Double-click to export | [clk]  |             |             |       |        |
| <input checked="" type="checkbox"/> |             | onchip_memory2_0        | On-Chip Memory (RAM or ROM) |                        |        |             |             |       |        |
| <input checked="" type="checkbox"/> |             | clk1                    | Clock Input                 | Double-click to export | clk_0  |             |             |       |        |
| <input checked="" type="checkbox"/> |             | s1                      | Avalon Memory Mapped Slave  | Double-click to export | [clk1] | 0x0000_8000 | 0x0000_fcff |       |        |
| <input checked="" type="checkbox"/> |             | reset1                  | Reset Input                 | Double-click to export | [clk1] |             |             |       |        |

Messages

| Description                                          | Path                    |
|------------------------------------------------------|-------------------------|
| 1 Info Message                                       |                         |
| Memory will be initialized from onchip_memory2_0.hex | System.onchip_memory2_0 |

0 Errors, 0 Warnings

# System conception with platform designer



Select your component

Instantiate

Connect

Connect clocks, resets and Avalon buses

The screenshot displays the Qsys Platform Designer interface for a project named 'Qsys - PWM.qsys'. The interface is divided into several panes:

- Library:** A tree view on the left showing various components like 'alu\_avalon', 'System', 'Bridges', 'Clock and Reset', 'Configuration & Programming', 'DSP', 'Embedded Processors', 'Interface Protocols', 'Memories and Memory Controller', 'Merlin Components', 'Microcontroller Peripherals', 'Peripherals', 'PLL', 'Qsys Interconnect', 'SLS', 'University Program', 'Verification', and 'Window Bridge'.
- System Contents:** A central pane showing a hierarchical tree of the system components. A green oval highlights the 'clk\_0' component, and a red oval highlights the 'nios2\_qsys\_0' component.
- Connections:** A pane on the right showing the connections between components. It lists components like 'clk\_0', 'nios2\_qsys\_0', 'jtag\_uart\_0', and 'onchip\_memory2\_0' with their respective inputs and outputs. A red line connects the 'clk\_0' component to the 'nios2\_qsys\_0' component, and a green line connects the 'jtag\_uart\_0' component to the 'onchip\_memory2\_0' component.
- Messages:** A pane at the bottom left showing the status of the system, currently displaying '0 Errors, 0 Warnings'.

| Name                    | Description                 |
|-------------------------|-----------------------------|
| clk_0                   | Clock Source                |
| clk_in                  | Clock Input                 |
| clk_in_reset            | Reset Input                 |
| clk                     | Clock Output                |
| clk_reset               | Reset Output                |
| nios2_qsys_0            | Nios II Processor           |
| clk                     | Clock Input                 |
| reset_n                 | Reset Input                 |
| data_master             | Avalon Memory Mapped Master |
| instruction_master      | Avalon Memory Mapped Master |
| d_irq                   | Interrupt Receiver          |
| jtag_debug_module_re... | Reset Output                |
| jtag_debug_module       | Avalon Memory Mapped Slave  |
| custom_instruction_m... | Custom Instruction Master   |
| jtag_uart_0             | JTAG UART                   |
| clk                     | Clock Input                 |
| reset                   | Reset Input                 |
| avalon_jtag_slave       | Avalon Memory Mapped Slave  |
| irq                     | Interrupt Sender            |
| onchip_memory2_0        | On-Chip Memory (RAM or ROM) |
| clk1                    | Clock Input                 |
| s1                      | Avalon Memory Mapped Slave  |
| reset1                  | Reset Input                 |

# System conception with platform designer

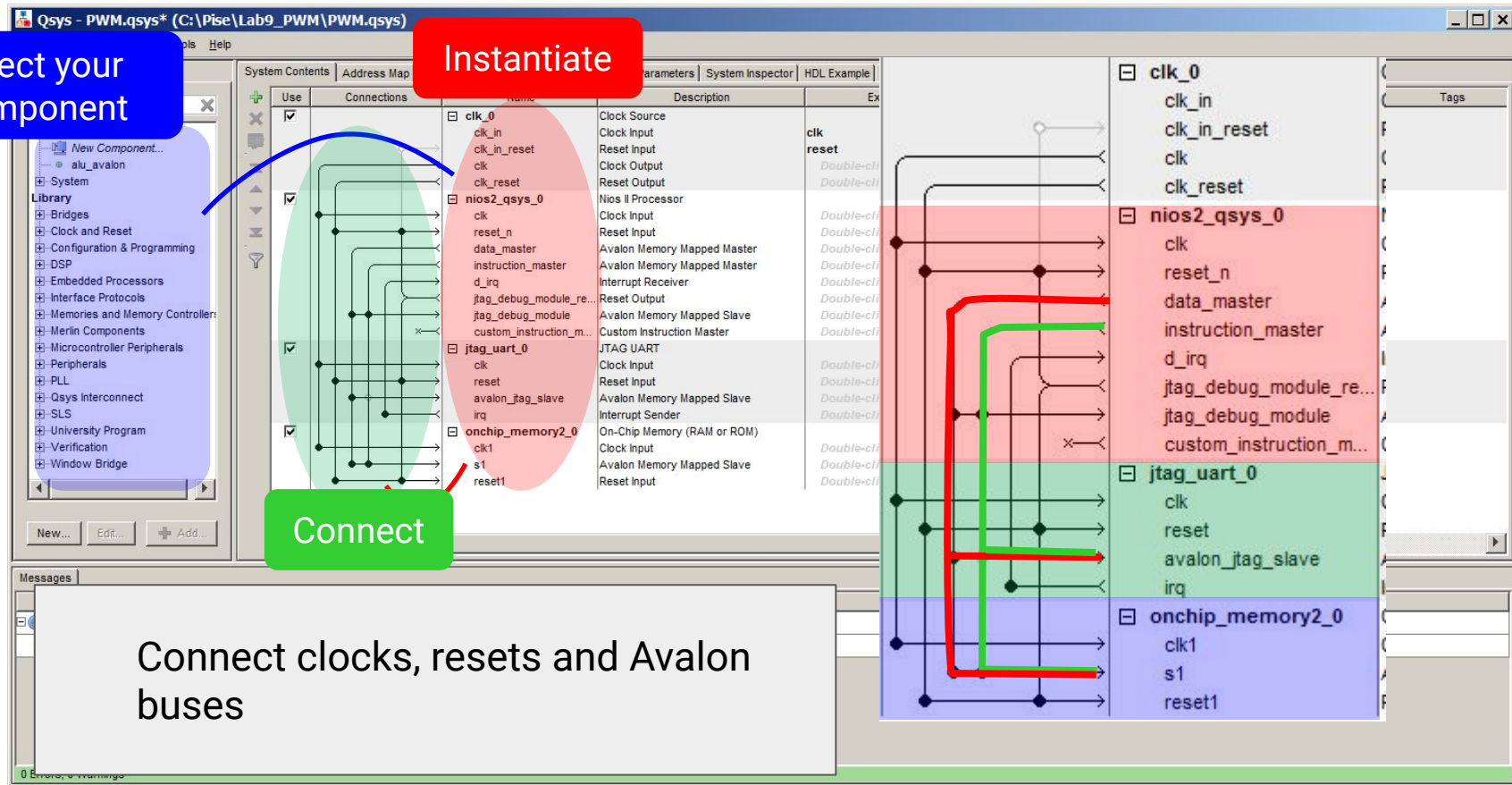


Select your component

Instantiate

Connect

Connect clocks, resets and Avalon buses





# System design with platform designer



Select your component

Instantiate

export to top level

Connect

The screenshot shows the Qsys platform designer interface with the following components and connections:

- Component Selection:** A blue oval highlights the 'Library' pane on the left, which contains various components like 'System', 'Bridges', 'Clock and Reset', 'Configuration & Programming', 'DSP', 'Embedded Processors', 'Interface Protocols', 'Memories and Memory Controller', 'Merlin Components', 'Microcontroller Peripherals', 'Peripherals', 'PLL', 'Qsys Interconnect', 'SLS', 'University Program', 'Verification', and 'Window Bridge'.
- Instantiation:** A red oval highlights the 'Name' column in the 'System Contents' pane, showing components like 'clk\_0', 'nios2\_qsys\_0', 'jtag\_uart\_0', and 'onchip\_memory2\_0'.
- Connection:** A green oval highlights the 'Connections' pane, showing a network of connections between the instantiated components.
- Export to Top Level:** A purple oval highlights the 'Export' column in the 'System Contents' pane, showing options to 'Double-click to export' various components.

| Component               | Description                 | Base | End | IRQ | Tags |
|-------------------------|-----------------------------|------|-----|-----|------|
| clk_0                   | Clock Source                |      |     |     |      |
| clk_in                  | Clock Input                 |      |     |     |      |
| clk_in_reset            | Reset Input                 |      |     |     |      |
| clk                     | Clock Output                |      |     |     |      |
| clk_reset               | Reset Output                |      |     |     |      |
| nios2_qsys_0            | Nios II Processor           |      |     |     |      |
| clk                     | Clock Input                 |      |     |     |      |
| reset_n                 | Reset Input                 |      |     |     |      |
| data_master             | Avalon Memory Mapped Master |      |     |     |      |
| instruction_master      | Avalon Memory Mapped Master |      |     |     |      |
| d_irq                   | Interrupt Receiver          |      |     |     |      |
| jtag_debug_module_re... | Reset Output                |      |     |     |      |
| jtag_debug_module       | Avalon Memory Mapped Slave  |      |     |     |      |
| custom_instruction_m... | Custom Instruction Master   |      |     |     |      |
| jtag_uart_0             | JTAG UART                   |      |     |     |      |
| clk                     | Clock Input                 |      |     |     |      |
| reset                   | Reset Input                 |      |     |     |      |
| avalon_jtag_slave       | Avalon Memory Mapped Slave  |      |     |     |      |
| irq                     | Interrupt Sender            |      |     |     |      |
| onchip_memory2_0        | On-Chip Memory (RAM or ROM) |      |     |     |      |
| clk1                    | Clock Input                 |      |     |     |      |
| s1                      | Avalon Memory Mapped Slave  |      |     |     |      |
| reset1                  | Reset Input                 |      |     |     |      |

Messages:

- 1 Info Message
- Memory will be initialized from onchip\_memory2\_0.hex

0 Errors, 0 Warnings

# System design with platform designer



Select your component

Instantiate

export to top level

The screenshot shows the Qsys Platform Designer interface. On the left, the 'Library' pane lists various components like 'alu\_avalon', 'Clock and Reset', 'DSP', etc. A blue arrow points from the 'Select your component' callout to the 'Library' pane. In the center, the 'Connections' pane shows a list of components and their ports. A red arrow points from the 'Instantiate' callout to the 'Connections' pane. On the right, the 'Parameters' pane shows a table of parameters for the selected component. A purple arrow points from the 'export to top level' callout to the 'Parameters' pane. At the bottom, a code editor shows the instantiation of the component in Verilog.

```
test_sys_inst : component test_sys
port map (
    clk_clk
    reset_reset_n
    opencores_i2c_0_export_0_scl_pad_io
    opencores_i2c_0_export_0_sda_pad_io
    nrst_trig_external_connection_export
    pll_out_clk_38_clk
    pll_out_clk_25_clk
);
```

Some signals have to be exported and represent the ports of your systems



# System design with platform designer



Select your component

Instantiate

export to top level

Address mapping

Connect

The screenshot shows the Qsys platform designer interface. On the left, the 'Library' pane lists various components like 'alu\_avalon', 'Clock and Reset', 'DSP', 'Embedded Processors', 'Interface Protocols', 'Memories and Memory Controller', 'Merlin Components', 'Microcontroller Peripherals', 'Peripherals', 'PLL', 'Qsys Interconnect', 'SLS', 'University Program', 'Verification', and 'Window Bridge'. A blue oval highlights the 'Library' pane with the text 'Select your component'. In the center, the 'Connections' pane shows a network of components including 'clk\_0', 'nios2\_qsys\_0', 'jtag\_uart\_0', and 'onchip\_memory2\_0'. A green oval highlights the 'Connections' pane with the text 'Connect'. On the right, the 'Address Map' pane shows a table of memory-mapped blocks. A red oval highlights the 'Instantiate' button, a purple oval highlights the 'export to top level' button, and an orange oval highlights the 'Address mapping' button. A white box at the bottom right contains the following text:

- Remember Avalon MM ... each peripheral is a memory address range ...
- IPD provides a way to automatically assign addresses to memory mapped blocks

system -> assign base address

# System design with platform designer



Select your component

Instantiate

export to top level

Address mapping

Interrupts

Connect

The screenshot displays the Qsys Platform Designer interface for a project named 'Qsys - PWM.qsys'. The interface is divided into several panes:

- Library:** A list of components available for selection, including 'System', 'Bridges', 'Clock and Reset', 'Configuration & Programming', 'DSP', 'Embedded Processors', 'Interface Protocols', 'Memories and Memory Controller', 'Merlin Components', 'Microcontroller Peripherals', 'Peripherals', 'PLL', 'Qsys Interconnect', 'SLS', 'University Program', 'Verification', and 'Window Bridge'.
- System Contents:** A diagram showing the instantiated components and their interconnections. A green oval highlights the 'nios2\_qsys\_0' component, and a red oval highlights the 'onchip\_memory2\_0' component.
- Address Map:** A table showing the address mapping for the instantiated components. The table has columns for 'Name', 'Description', 'Clock', 'IRQ', and 'Tags'.
- Messages:** A pane at the bottom showing system messages, including an info message: 'Memory will be initialized from onchip\_memory2\_0.hex'.

The 'Address Map' table is as follows:

| Name                    | Description                 | Clock | IRQ | Tags |
|-------------------------|-----------------------------|-------|-----|------|
| clk_0                   | Clock Source                | clk_0 |     |      |
| clk_in                  | Clock Input                 | clk_0 |     |      |
| clk_in_reset            | Reset Input                 | clk_0 |     |      |
| clk_reset               | Clock Output                | clk_0 |     |      |
| reset                   | Reset Output                | clk_0 |     |      |
| nios2_qsys_0            | Nios II Processor           | clk_0 |     |      |
| clk                     | Clock Input                 | clk_0 |     |      |
| reset_n                 | Reset Input                 | clk_0 |     |      |
| data_master             | Avalon Memory Mapped Master | clk_0 |     |      |
| instruction_master      | Avalon Memory Mapped Master | clk_0 |     |      |
| d_irq                   | Interrupt Receiver          | clk_0 |     |      |
| jtag_debug_module_re... | Reset Output                | clk_0 |     |      |
| jtag_debug_module       | Avalon Memory Mapped Slave  | clk_0 |     |      |
| custom_instruction_m... | Custom Instruction Master   | clk_0 |     |      |
| jtag_uart_0             | JTAG UART                   | clk_0 |     |      |
| clk                     | Clock Input                 | clk_0 |     |      |
| reset                   | Reset Input                 | clk_0 |     |      |
| avalon_jtag_slave       | Avalon Memory Mapped Slave  | clk_0 |     |      |
| irq                     | Interrupt Sender            | clk_0 |     |      |
| onchip_memory2_0        | On-Chip Memory (RAM or ROM) | clk_0 |     |      |
| clk1                    | Clock Input                 | clk1  |     |      |
| s1                      | Avalon Memory Mapped Slave  | clk1  |     |      |
| reset1                  | Reset Input                 | clk1  |     |      |

The 'Messages' pane shows the following message:

1 Info Message

Memory will be initialized from onchip\_memory2\_0.hex

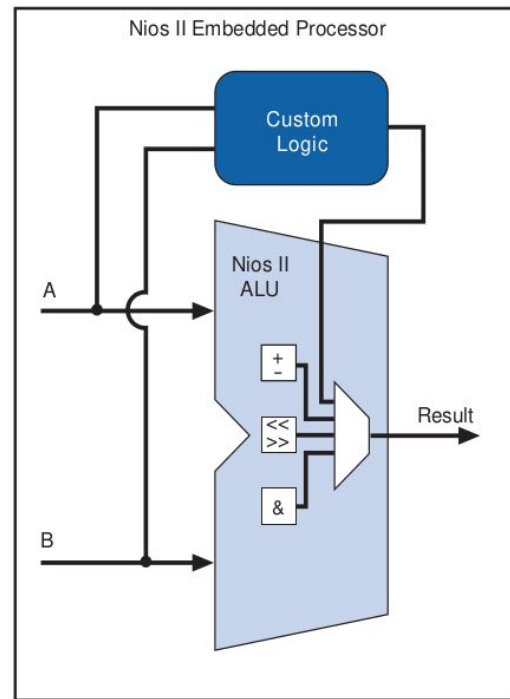
System.onchip\_memory2\_0

# System design with platform designer



Example of blocks that can be added to the system:

- DMA controllers
- IO handlers
- Communication blocks (SPI/I2C/ ...)
- Processing IP blocks (video encoders, ...)
- **Overloading Nios II instructions**





# Debugging your system through JTAG

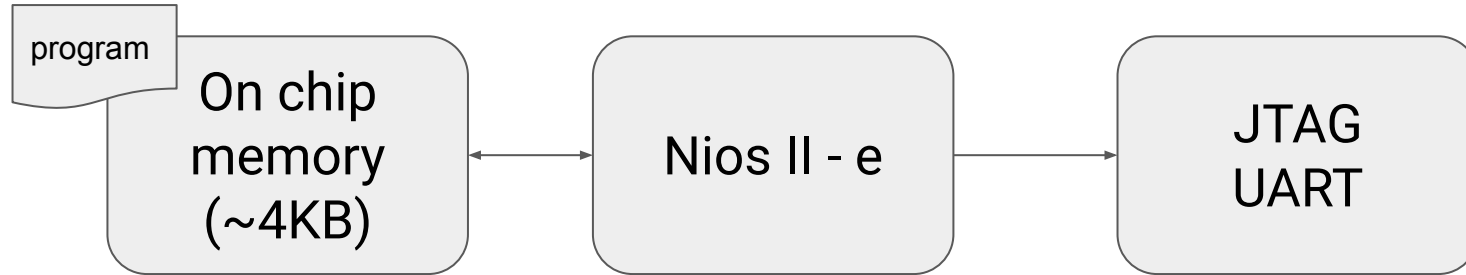
- During development, add a JTAG/UART block to your system
- Don't forget the `nios2-terminal` feature
- **JTAG\_UART is blocking**, if you do not read the data, code execution is stalled
- JTAG UART is a AV-MM peripheral. The UART connection is handled by Quartus through the JTAG programmer link

The diagram illustrates the configuration of the JTAG\_UART block in a Quartus schematic. The block is shown with its inputs and outputs: `clk` (Clock Input), `reset` (Reset Input), `avalon_jtag_slave` (Avalon Memory Mapped Slave), and `irq` (Interrupt Sender). A green arrow points from the schematic to a screenshot of the Nios2 terminal window.

The screenshot shows the Nios2 terminal window with the following text:

```
radu : nios2-terminal-  
File Edit View Scrollback Bookmarks Settings Help  
[radu@radu-linux ~]$ ~/altera/13.1/nios2eds/nios2_command_shell.sh  
-----  
Altera Nios2 Command Shell [GCC 4]  
  
Version 13.1, Build 158  
-----  
[radu@radu-linux ~]$ nios2-terminal  
nios2-terminal: connected to hardware target using JTAG UART on cable  
nios2-terminal: 'USB-BlasterII [1-1.3.3]', device 2, instance 0  
nios2-terminal: (Use the IDE stop button or Ctrl-C to terminate)
```

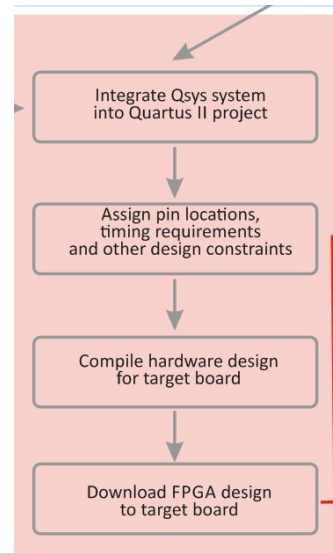
# Basic Nios-II based SoPC (Exemple lab 1)



# Hardware flow with Quartus



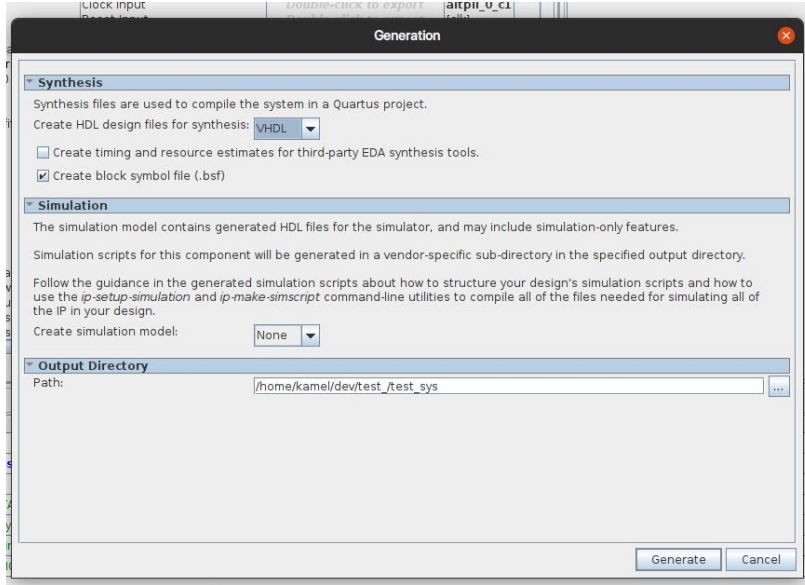
1. Use Platform designer to compile the SoPC
2. Instantiate the resulting hardware in your design
3. Add extra, non-system, components
4. Simulation
5. Pin assignment / design constraints
6. Design compilation
7. FPGA Programming



# Hardware flow with Quartus



- The first step is to generate the hardware description of the created system
- The result is packaged as VHDL/Verilog component that can be imported



```
component test_sys is
port (
    clk_clk          : in    std_logic := '0';
    opencores_i2c_0_export_0_scl_pad_io : inout std_logic := '0';
    opencores_i2c_0_export_0_sda_pad_io : inout std_logic := '0';
    pll_out_clk_38_clk : out   std_logic := '0';
    pll_out_clk_25_clk : out   std_logic := '0';
    reset_reset_n      : in    std_logic := '0';
    nrst_trig_external_connection_export : out   std_logic := '0'
);
end component test_sys;
```

# Hardware flow with Quartus



Second, the exported system has to be instantiated in the top level RTL description:

Of course, the system can be a part of a bigger design, in this case, wire everything in your top level

```
begin
  test_sys_inst : component test_sys
    port map (
      clk_clk           => clk_50,
      reset_reset_n     => '1',
      opencores_i2c_0_export_0_scl_pad_io => sony_i2c_scl,
      opencores_i2c_0_export_0_sda_pad_io => sony_i2c_sda,
      nrst_trig_external_connection_export => test_reset_n,
      pll_out_clk_38_clk => test_clk_i,
      pll_out_clk_25_clk => vga_clk_s
    );
```

```
begin
  test_sys_inst : component test_sys
    port map (
      clk_clk           => clk_50,
      reset_reset_n     => '1',
      opencores_i2c_0_export_0_scl_pad_io => sony_i2c_scl,
      opencores_i2c_0_export_0_sda_pad_io => sony_i2c_sda,
      nrst_trig_external_connection_export => test_reset_n,
      pll_out_clk_38_clk => test_clk_i,
      pll_out_clk_25_clk => vga_clk_s
    );
end;

dcfifo_component : dcfifo
  GENERIC MAP (
    PORT MAP (
      aclr           => '0',
      data           => sony_data_o(sony_pixwidth-1 downto sony_pixwidth - vga_pixwidth),
      rdclk          => vga_clk_s,
      rdreq          => dcfifo_rd,
      wrclk          => sony_clk_o,
      wrreq          => dcfifo_wr,
      q              => dcfifo_o,
      rdempty        => dcfifo_empty,
      wrfull         => dcfifo_full
    );
  );

dcfifo_wr_proc: process(sony_clk_o)
begin
  if (reset_n = '0') then
    dcfifo_wr <= '0';
  --dcfifo_aclr <= '1';
  elsif (falling_edge(sony_clk_o)) then
    if (dcfifo_full = '1') then
      dcfifo_wr <= '0'; -- FIFO is full. Note supposed to happen ...
    elsif (sony_vsync = '1') then
      if (sony_hsync = '1') then
        dcfifo_wr <= '1'; -- Pixel is valid
      else
        dcfifo_wr <= '0'; -- Pixel is not valid
      end if;
    else
      dcfifo_wr <= '0'; -- Pixel is not valid
    end if;
  end if;
end if;
end process dcfifo_wr_proc;
```

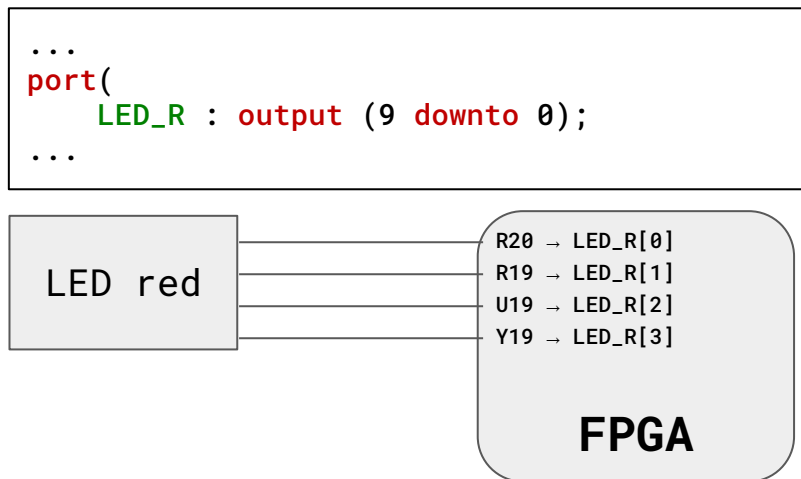


# Hardware flow with Quartus



Next, assign the pin locations:

- Match each port of your VHDL design to a corresponding physical I/O pin in the FPGA
- Use your FPGA board datasheet to get pin names



| FPGA Pin No. | Description  |
|--------------|--------------|
| PIN_R20      | LED Red[0]   |
| PIN_R19      | LED Red[1]   |
| PIN_U19      | LED Red[2]   |
| PIN_Y19      | LED Red[3]   |
| PIN_T18      | LED Red[4]   |
| PIN_V19      | LED Red[5]   |
| PIN_Y18      | LED Red[6]   |
| PIN_U18      | LED Red[7]   |
| PIN_R18      | LED Red[8]   |
| PIN_R17      | LED Red[9]   |
| PIN_U22      | LED Green[0] |
| PIN_U21      | LED Green[1] |
| PIN_V22      | LED Green[2] |
| PIN_V21      | LED Green[3] |
| PIN_W22      | LED Green[4] |
| PIN_W21      | LED Green[5] |
| PIN_Y22      | LED Green[6] |
| PIN_Y21      | LED Green[7] |

*Ex. LED pinout on the DE1 devkit*



# Hardware flow with Quartus

Not all the FPGA pins are equivalents. For example, some pins are:

- Input only
- Output only
- Dedicated to clock signals

If you designing/validating your own FPGA board, you can use the Pin assignment utility.

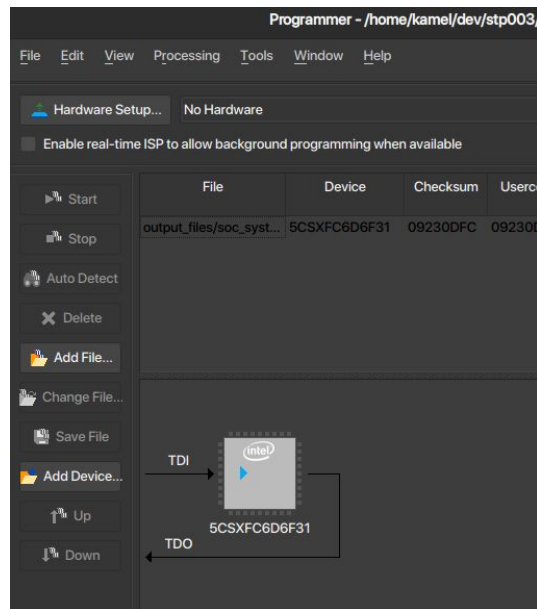
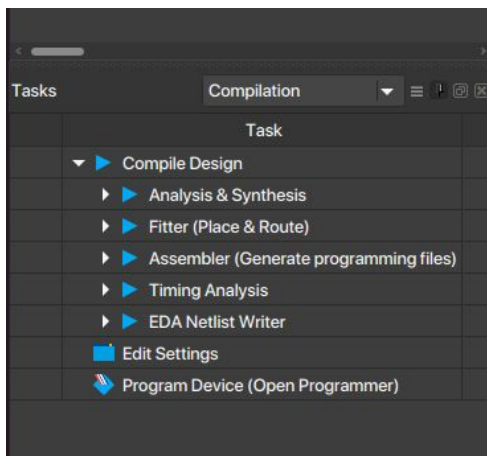
Otherwise, assign your pins directly in the qsf file

```
# -----  
# C10-LEPTON  
set_location_assignment PIN_W10 -to LEPTON_I2C_SCL  
set_location_assignment PIN_Y10 -to LEPTON_I2C_SDA  
set_location_assignment PIN_M22 -to LEPTON_PWDN_L  
set_location_assignment PIN_U21 -to LEPTON_RESET_L  
set_location_assignment PIN_W7 -to LEPTON_MC  
set_location_assignment PIN_Y7 -to LEPTON_VSYNC  
set_location_assignment PIN_Y4 -to LEPTON_SPI_CLK  
set_location_assignment PIN_V8 -to LEPTON_SPI_CS  
set_location_assignment PIN_W8 -to LEPTON_SPI_MISO  
set_location_assignment PIN_Y3 -to LEPTON_SPI_MOSI
```

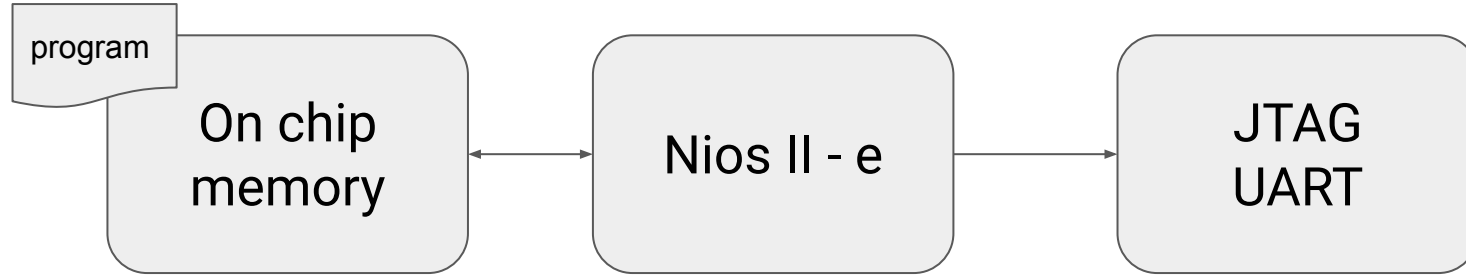
# Hardware flow with Quartus



Compile the design and program the FPGA. The system SoPC is implemented on now.



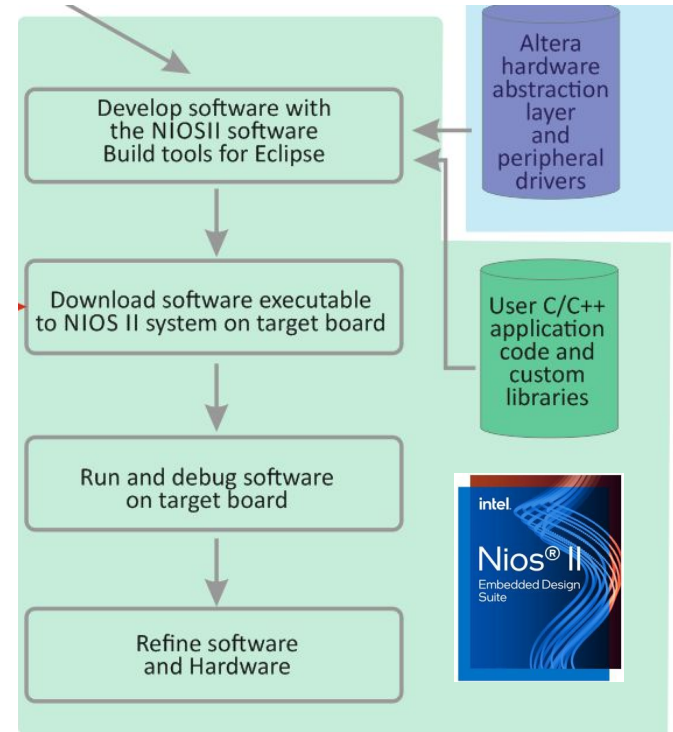
# Basic Nios-II based SoPC (Exemple lab 1)



# Software flow with Nios EDS

But what about the software part ?

1. Generate the HAL-BSP of the system
2. Write the user program that will run on the processor
3. Generate the adequate Makefile
4. Cross-compile your code
5. Download the executable file to the processor

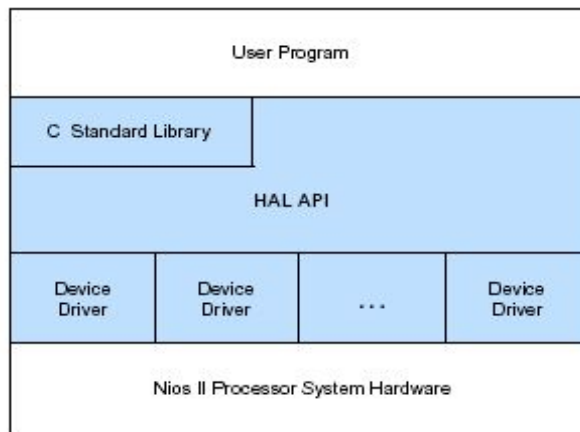


# Software flow with Nios EDS



First, one has to generate the **HAL** of the design

- HAL is software layer that forms a clear distinction between application and device-level software.
- HAL provides services such as file descriptors, I/O control, and buffering
- HAL allows programmer to control the hardware using simple function calls

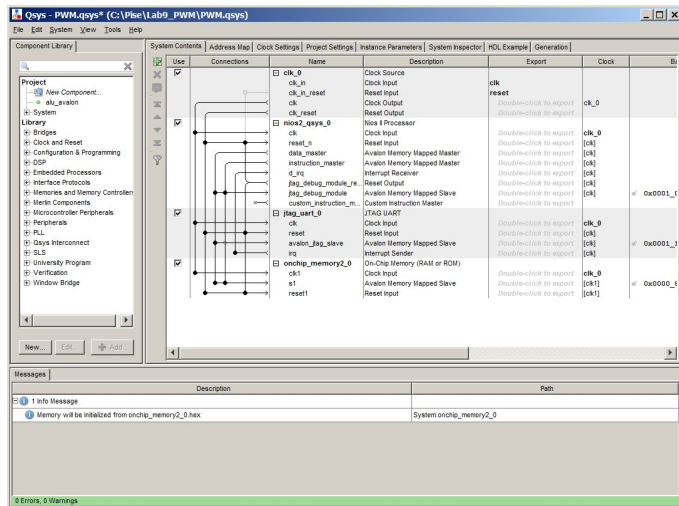


# Software flow with Nios EDS



HAL requires a bunch of header files be included in user C source file:

- “system.h” includes the basic configuration of the SoPC, processor parameters, address and data bus width, I/O Address Map, should be included if HAL is used.



```
#define ALT_CPU_ARCHITECTURE "altera_nios2_gen2"
#define ALT_CPU_BIG_ENDIAN 0
#define ALT_CPU_BREAK_ADDR 0x02020820
#define ALT_CPU_CPU_ARCH_NIOS2_R1
#define ALT_CPU_CPU_FREQ 100000000u
#define ALT_CPU_CPU_ID_SIZE 1
#define ALT_CPU_CPU_ID_VALUE 0x00000000
#define ALT_CPU_CPU_IMPLEMENTATION "tiny"
#define ALT_CPU_DATA_ADDR_WIDTH 0x1a
#define ALT_CPU_DCACHE_LINE_SIZE 0
#define ALT_CPU_DCACHE_LINE_SIZE_LOG2 0
#define ALT_CPU_EXCEPTION_ADDR 0x02010020
#define ALT_CPU_FLASH_ACCELERATOR_LINES 0
#define ALT_CPU_FLASH_ACCELERATOR_LINE_SIZE 0
#define ALT_CPU_FLUSHDA_SUPPORTED
#define ALT_CPU_FREQ 100000000
#define ALT_CPU_HARDWARE_DIVIDE_PRESENT 0
#define ALT_CPU_HARDWARE_MULTIPLY_PRESENT 0
#define ALT_CPU_HARDWARE_MULX_PRESENT 0
#define ALT_CPU_HAS_DEBUG_CORE 1
#define ALT_CPU_HAS_DEBUG_STUB
#define ALT_CPU_HAS_ILLEGAL_INSTRUCTION_EXCEPTION
#define ALT_CPU_HAS_JMPI_INSTRUCTION
#define ALT_CPU_ICACHE_LINE_SIZE 0
#define ALT_CPU_ICACHE_LINE_SIZE_LOG2 0
#define ALT_CPU_ICACHE_SIZE 0
#define ALT_CPU_INST_ADDR_WIDTH 0x1a
#define ALT_CPU_NAME "nios2_gen2_0"
#define ALT_CPU_OCI_VERSION 1
#define ALT_CPU_RESET_ADDR 0x02010000

#define ALT_MODULE_CLASS_opencores_i2c_0 opencores_i2c
#define OPENCORES_I2C_0_BASE 0x2021040
#define OPENCORES_I2C_0_IRQ 1
#define OPENCORES_I2C_0_IRQ_INTERRUPT_CONTROLLER_ID 0
#define OPENCORES_I2C_0_NAME "/dev/opencores_i2c_0"
#define OPENCORES_I2C_0_SPAN 32
#define OPENCORES_I2C_0_TYPE "opencores_i2c"
```

# Software flow with Nios EDS



- Nios EDS can generate the BSP/HAL automatically according to the SoPC specifications

```
nios2-bsp hal <out_bsp_dir> <system.sopcinfo>
```

- Changes in the hardware configuration automatically propagate to the HAL device driver configuration. So regenerate the HAL everytime you modify your hardware
- The command also includes the header files for all the IP blocks used in the project



# Software flow on a Nios EDS



- On the system is setup (in both HW and SW means), one can implement a user program on it
- Typically, this program is written in the host computer and cross-compiled to to executed on the system processor
- Nios EDS can generate a template Makefile to build this software

```
nios2-app-generate-makefile \  
  --app-dir <out_dir> \  
  --bsp-dir <bsp_dir> \  
  --elf-name <exec_name.elf> \  
  --src-files <c_src_file.c>
```

# Software flow on a Nios EDS



1. Everything is ready, now we shall write our first C code that runs on our own processor
2. Remember to update the Makefile with the source code
3. Compilation occurs on a Nios II shell:

**make**

4. The generated program can be loaded and executed in the FPGA softcore:

```
nios2-download -g -r <binary_name.elf>  
// or make download-elf
```

# Software flow on a Nios EDS



1. Everything is ready, now we shall write our first C code that runs on our own processor
2. If you didn't include source files in your Makefile, edit it !

```
75 # TYPE: APP_MAKEFILE
76
77 # This following VERSION comment indicates th
78 # generate this makefile. A makefile variable
79 # ACDS_VERSION: 20.1
80 ACDS_VERSION := 20.1
81
82 # This following BUILD_NUMBER comment indicat
83 # used to generate this makefile.
84 # BUILD_NUMBER: 720
85
86 # Define path to the application ELF.
87 # It may be used by the makefile fragments so
88 #
89 ELF := test.elf
90
91 # Paths to C, C++, and assembly source files.
92 C_SRCS := main.c
93 CXX_SRCS :=
94 ASM_SRCS :=
95
96 SDIR_C_SRCS :=
97 SDIR_CXX_SRCS :=
98 SDIR_ASM_SRCS :=
99
100 # Path to root of object file tree.
101 OBJ_ROOT_DIR := obj
102
103 # Options to control objdump.
104 CREATE_OBJDUMP := 1
105 OBJDUMP_INCLUDE_SOURCE := 0
106 OBJDUMP_FULL_CONTENTS := 0
107
```

```
int8_t sony640d_powerup_write_dict(uint32_t base, uint32_t slave_addr, struct regval_dict
{
    while (dict->reg != 0xFFFF || dict->val != 0xFF)
    {
        write_reg(base, slave_addr, dict->reg, dict->val);
        dict++;
        usleep(300);
    }
    return 1;
}

int main()
{
    //I2C Check
    alt_printf("Testing i2c slave\n");
    code_init = I2C_init(OPENCORES_I2C_0_BASE, ALT_CPU_FREQ, 100000);
    alt_printf("Code init 0x%x \n", code_init);
    present = I2C_start(OPENCORES_I2C_0_BASE, slave_addr, 0);
    if(present)
        alt_printf("Slave 0x%x found\n", slave_addr);
    else
    {
        alt_printf("Missing i2c slave");
        return 0;
    }

    //Sensor reset
    alt_printf("Sending reset ... \n");
    IOWR(NRST_TRIG_BASE, 0x0, 0x1);
    usleep(10);
    alt_printf("Done !\n");

    //config. i2c
    alt_printf("Configuring i2c ... \n");
    sony640d_powerup_write_dict(OPENCORES_I2C_0_BASE, slave_addr, |sony640d_powerup_dict);
    //usleep(1000);
    printf("Done !\n");
    return 0;
}
```

# Software flow on a Nios EDS

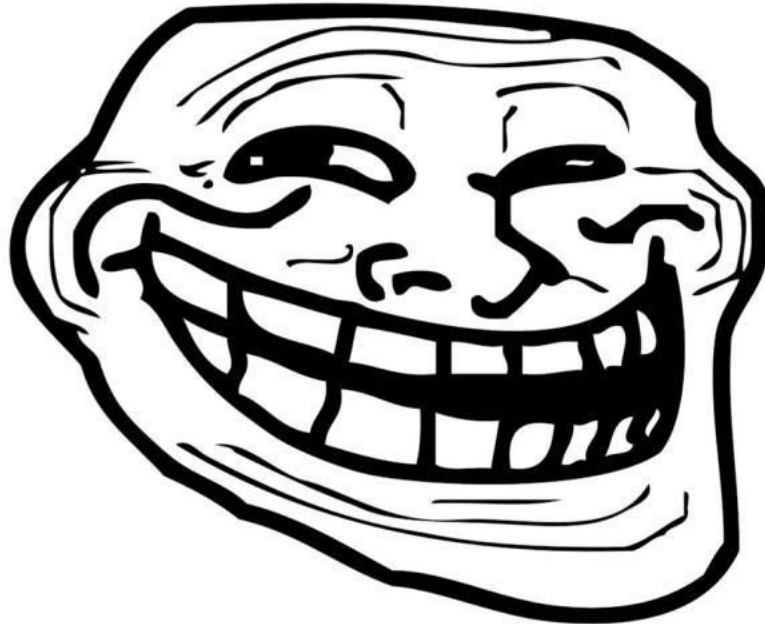


1. Everything is ready, now we shall write our first C code that runs on our own processor
2. Remember to update the Makefile with the source code
3. Compilation occurs on a Nios II shell:

**make**

4. The generated program can be loaded and executed in the FPGA softcore:

```
nios2-download -g -r <binary_name.elf>  
// or make download-elf
```



Memory overflow problem ?

# Efficient Nios II programs



- Always prefer using alt\_printf instead of printf

```
#include "sys/alt_stdio.h"
```

| Function    | Description                                                                                 |
|-------------|---------------------------------------------------------------------------------------------|
| =====       | =====                                                                                       |
| alt_printf  | Only supports %s, %x, and %c ( < 1 Kbyte)                                                   |
| alt_putstr  | Smaller overhead than puts with direct drivers<br>Note this function doesn't add a newline. |
| alt_putchar | Smaller overhead than putchar with direct drivers                                           |
| alt_getchar | Smaller overhead than getchar with direct drivers                                           |

# Efficient Nios II programs



- Always prefer using `alt_printf` instead of `printf`
- Use small C library
- Use lightweight device driver API

```
nios2-bsp-editor --settings  
<settings_file.bsp>
```

- Then

```
nios2-bsp-generate-files \  
--settings=../bsp/settings.bsp \  
--bsp-dir=../bsp
```

hal

max\_file\_descriptors: 32

☒ enable\_instruction\_related\_exceptions\_api

log\_port: none

☒ enable\_exit

☒ enable\_clean\_exit

☐ enable\_runtime\_stack\_checking

☐ enable\_c\_plus\_plus

☒ enable\_lightweight\_device\_driver\_api

☐ enable\_mul\_div\_emulation

☒ enable\_soc\_sysid\_check

custom\_newlib\_flags: none

log\_flags: 0

# Efficient Nios II programs



- Always prefer using `alt_printf` instead of `printf`
- Use small C library
- Use lightweight device driver API
- Disable C++ support

```
nios2-bsp-editor --settings  
<settings_file.bsp>
```

- Then

```
Nios2-bsp-generate-files \  
--settings=../bsp/settings.bsp \  
--bsp-dir=../bsp
```

hal

max\_file\_descriptors: 32

☒ enable\_instruction\_related\_exceptions\_api

log\_port: none

☒ enable\_exit

☒ enable\_clean\_exit

☐ enable\_runtime\_stack\_checking

☐ enable\_c\_plus\_plus

☒ enable\_lightweight\_device\_driver\_api

☐ enable\_mul\_div\_emulation

☒ enable\_sopc\_sysid\_check

custom\_newlib\_flags: none

log\_flags: 0





# Co-design project structure

toto



toto.qpf



toto.qsf



toto\_sys.qsys



toto.sdc



toto.vhd



hardware



simulation



ip



software

# Typical project structure

toto



[toto.qpf](#)



toto.qsf



toto\_sys.qsys



toto.sdc



toto.vhd



hardware



simulation



ip



software

# Typical project structure

. qpf:

- Quartus project file, links to the qsf

```
#
# -----
#
# Quartus Prime
# Version 19.1.0 Build 670 09/22/2019 SJ Standard Edition
# Date created = 10:38:17 August 04, 2021
#
# -----
QUARTUS_VERSION = "19.1"
DATE = "10:38:17 August 04, 2021"

# Revisions

PROJECT_REVISION = "toto"
```

toto



toto.qpf



toto.qsf



toto\_sys.qsys



toto.sdc



toto.vhd



hardware



simulation



ip



software

# Typical project structure

. qsf:

- Core Quartus project file
- Lists the files included in the project
- FPGA device declaration
- PIN assignment

```
set_global_assignment -name FAMILY "Cyclone 10 LP"
set_global_assignment -name DEVICE 10CL120YF484I7G
set_global_assignment -name TOP_LEVEL_ENTITY navios
set_global_assignment -name ORIGINAL_QUARTUS_VERSION 19.1.0
set_global_assignment -name NUM_PARALLEL_PROCESSORS ALL

# ----- #
set_global_assignment -name QSYS_FILE toto_sys.qsys
set_global_assignment -name SDC_FILE toto.sdc
set_global_assignment -name VERILOG_FILE toto.v
# ----- #

# ----- #
# C10 RESET
set_location_assignment PIN_N21 -to C10_RESETh
# ----- #

# ----- #
# C10 SYSTEM CLOCKS
set_location_assignment PIN_G2 -to C10_CLK50M
set_location_assignment PIN_A11 -to HBUS_CLK_50M
# ----- #

# ----- #
# C10 UART DEBUG
set_location_assignment PIN_J21 -to UART_TX
set_location_assignment PIN_H22 -to UART_RX
# ----- #
```

toto



toto.qpf



toto.qsf



toto\_sys.qsys



toto.sdc



toto.vhd



hardware



simulation



ip

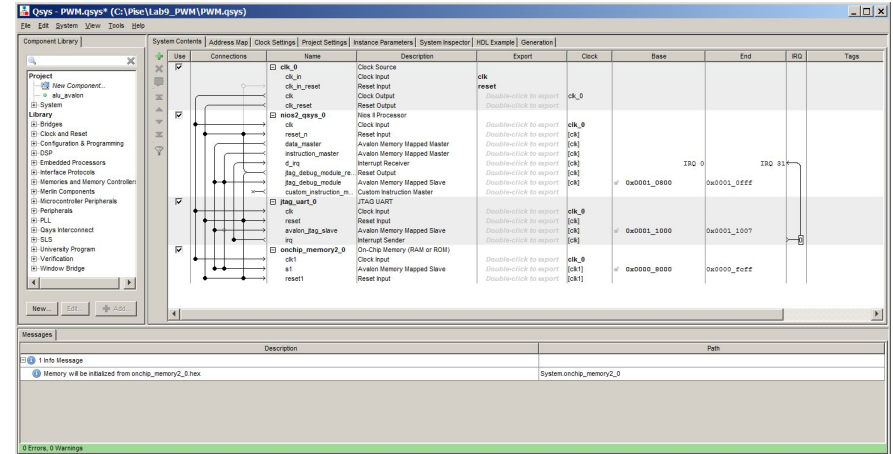


software

# Typical project structure

. qsys:

- System description
- Can be opened through Platform designer



toto



toto.qpf



toto.qsf



toto\_sys.qsys



toto.sdc



toto.vhd



hardware



simulation



ip



software

# Typical project structure

. sdc:

- Clock declaration
- Constraints description: input and output delays

```
*****  
# Time Information  
*****  
  
set_time_format -unit ns -decimal_places 3  
  
*****  
# Create Clock  
*****  
  
create_clock \  
-name {C10_CLK50M} \  
-period 20.000 \  
-waveform {0.000 10.000 } \  
[get_ports {C10_CLK50M}]  
  
create_clock \  
-name {HBUS_CLK_50M} \  
-period 20.000 \  
-waveform {0.000 10.000 } \  
[get_ports {HBUS_CLK_50M}]
```

toto



toto.qpf



toto.qsf



toto\_sys.qsys



toto.sdc



toto.vhd



hardware



simulation



ip



software

# Typical project structure

.vhd:

- Top-level hardware description
- Can instantiate the generated system and other peripherals
- Same name as the project

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity lab1 is
    port (
        reset_n : in std_logic := '0';
        clk      : in std_logic := '0'
    );
end entity lab1;

architecture rtl of lab1 is
    component lab1_sys is
        port (
            clk_clk      : in std_logic; -- clk
            reset_reset_n : in std_logic -- reset_n
        );
    end component lab1_sys;
begin
    lab1_sys_inst : component lab1_sys
        port map (
            clk_clk      => clk,      -- clk.clk
            reset_reset_n => reset_n -- reset.reset_n
        );
end architecture rtl;
```

toto

intel toto.qpf  
intel toto.qsf  
intel toto\_sys.qsys  
intel toto.sdc  
intel toto.vhd

hardware

simulation

ip

software

cmp\_1.vhd

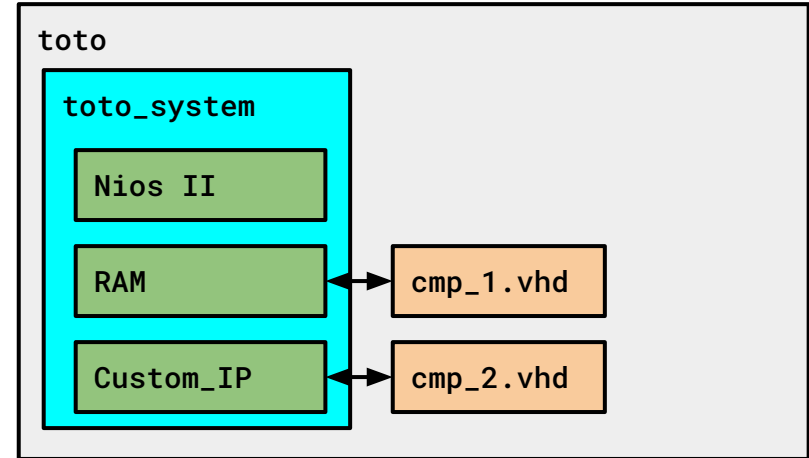
cmp\_2.vhd

cmp\_3.vhd

# Typical project structure

hardware:

- RTL description of hardware components not present in the system
- Can have their simulation files and test benches





toto

intel toto.qpf  
intel toto.qsf  
intel toto\_sys.qsys  
intel toto.sdc  
intel toto.vhd

hardware

simulation

ip

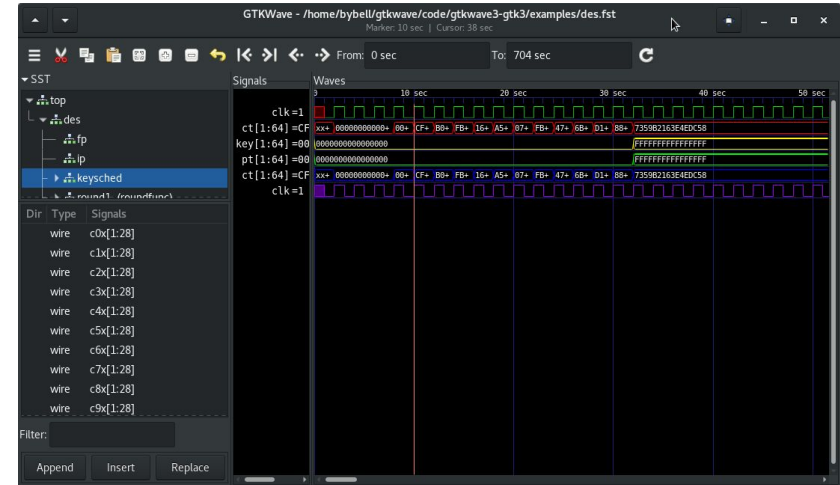
software

toto\_tb.vhd  
toto\_sim.vcd  
toto\_sim.tcl

# Typical project structure

simulation:

- Functional and timing simulation of the top-level
- Test benches
- Extracted wave forms (in .vcd formats)
- Simulation scripts (.tcl for modelsim)



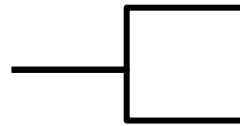
toto

 toto.qpf  
 toto.qsf  
 toto\_sys.qsys  
 toto.sdc  
 toto.vhd

 hardware

 ip

 software



Component A

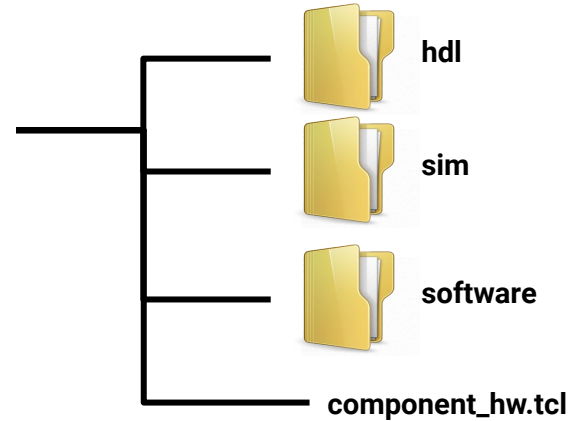


Component B

# Typical project structure

ip:

- Directory where platform designer components have to be located
- For each component, provide hardware description in hdl folder
- HAL/BSP directives are provided in software folder
- A tcl file implements the platform designer metadata



toto



toto.qpf



toto.qsf



toto\_sys.qsys



toto.sdc



toto.vhd



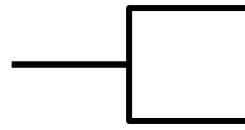
hardware



ip



software



app



bsp

# Typical project structure

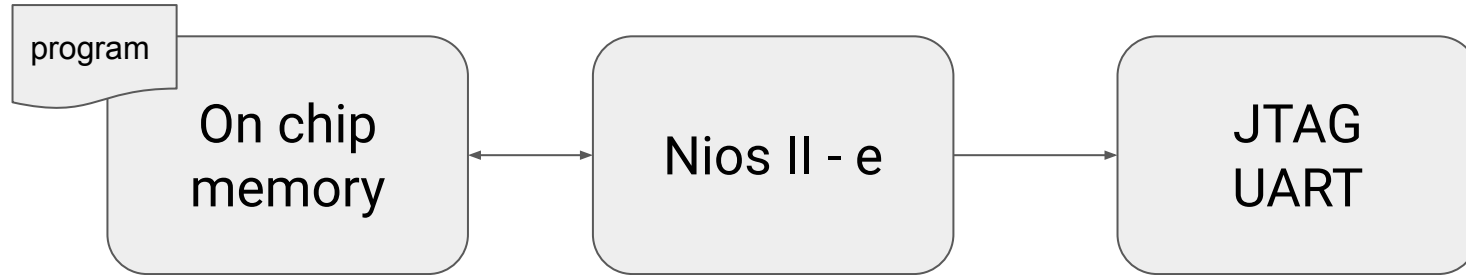
software:

- The app directory contains your application source files. You can also add your own libraries, makefiles and headers
- The bsp directory holds the generated hal/bsp files

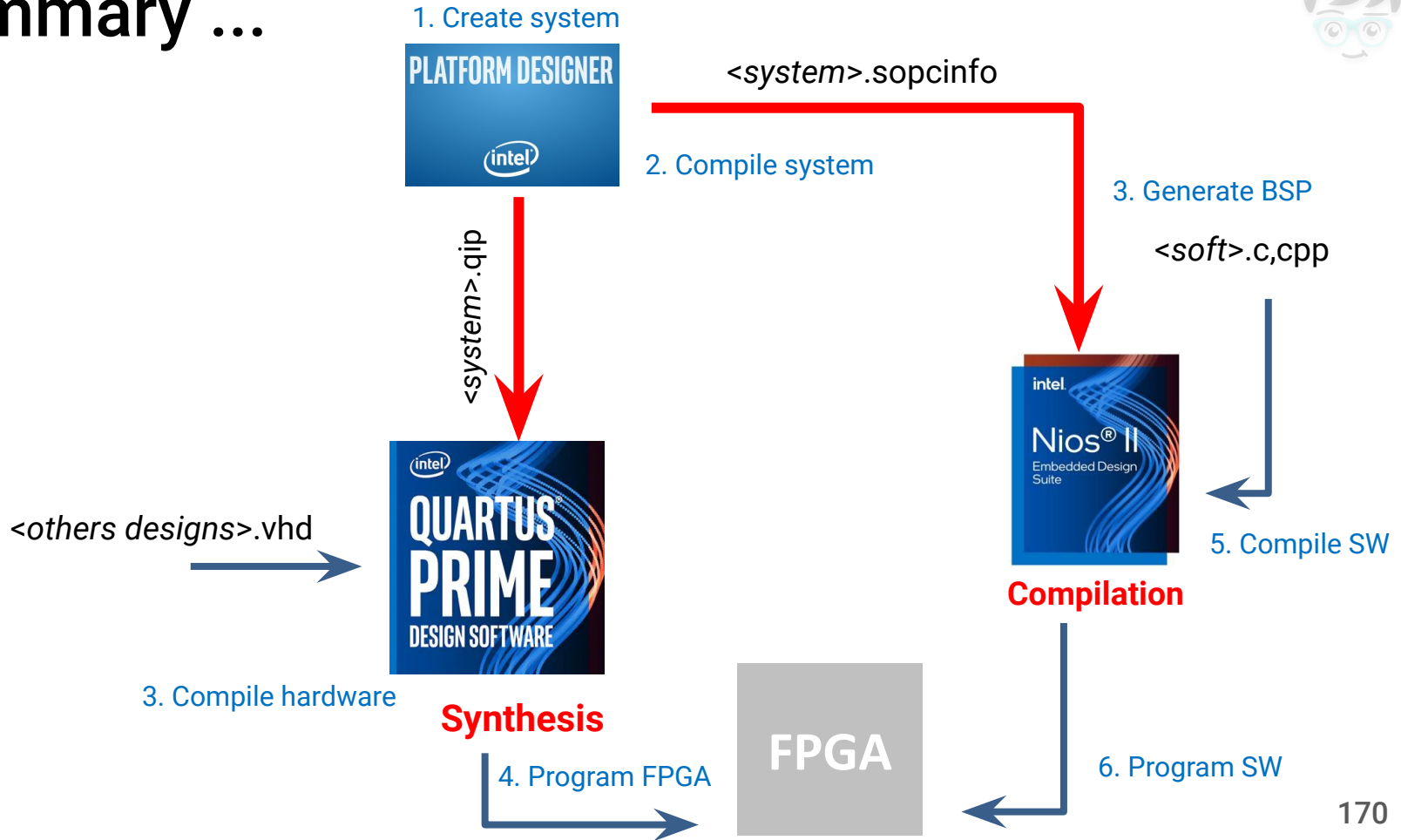


# Useful features

# Basic Nios-II based SoPC (Exemple lab 1)



# In summary ...





# Handling IOs



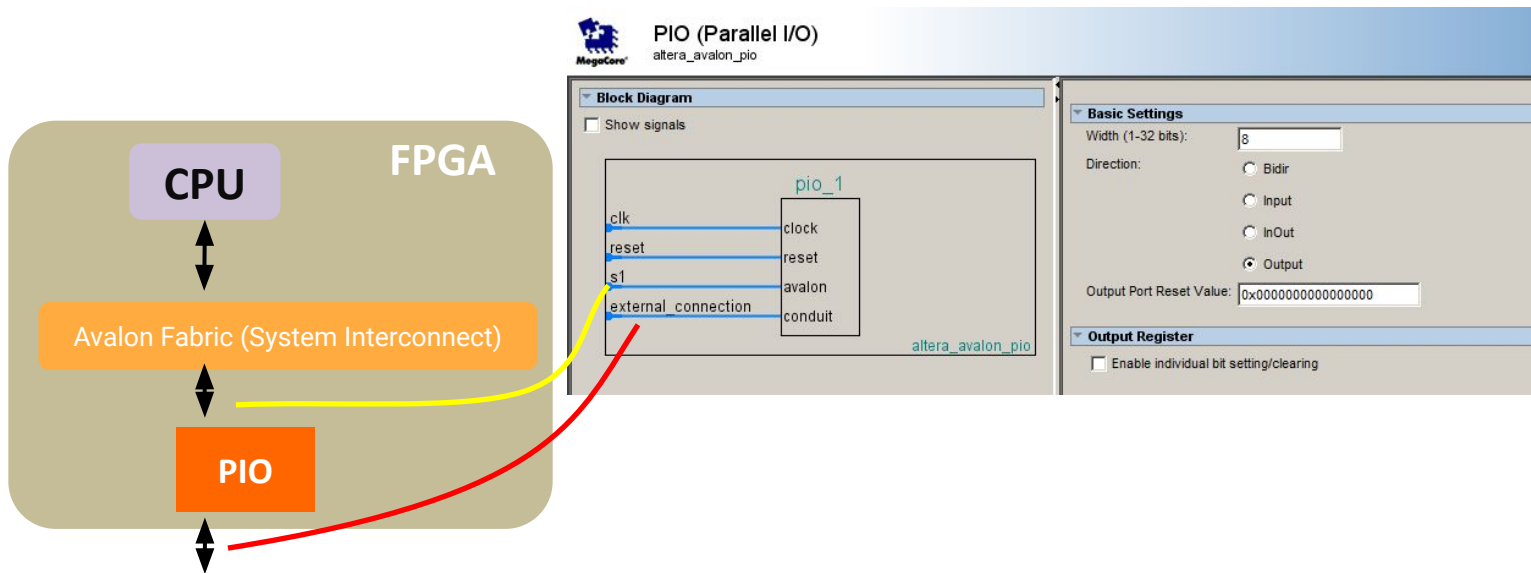
# Dealing with IOs

Parallel I/O is directly connected to processor pins, Nios II parallel I/O is addressed in 8-bit, and made up by 5 basic port registers:

- Data Register
- Direction Register
- Interrupt mask Register
- Edge Capture Register
- Outset Register
- Outclear Register

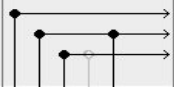


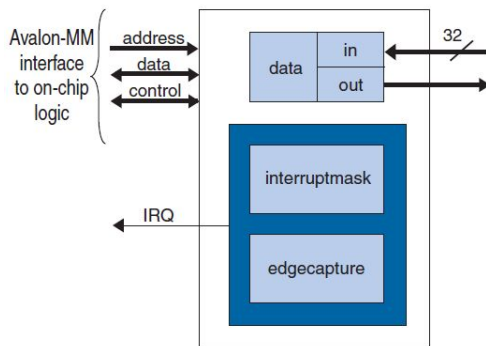
# Parallel I/O



# Parallel IO



| Use                                 | Connections                                                                       | Name                                                      | Description                                                                                        | Export                                                                                                                              | Clock                          | Base        | End         | IRQ |  |
|-------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|-------------|-------------|-----|--|
| <input checked="" type="checkbox"/> |  | <b>pio_0</b><br>clk<br>reset<br>s1<br>external_connection | PIO (Parallel I/O)<br>Clock Input<br>Reset Input<br>Avalon Memory Mapped Slave<br>Conduit Endpoint | <i>Double-click to export</i><br><i>Double-click to export</i><br><i>Double-click to export</i><br><b>pio_0_external_connection</b> | <b>clk_0</b><br>[clk]<br>[clk] | 0x0002_1000 | 0x0002_100f |     |  |

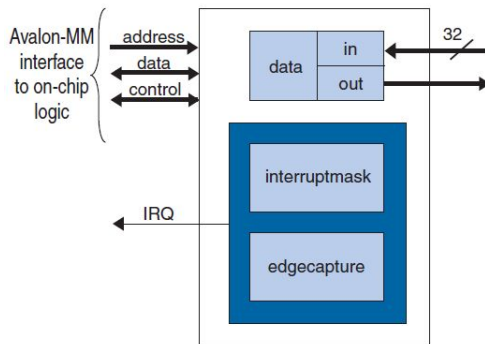


# Parallel IO



The PIO IOs are *conduits* : They are meant to be exported !

| Use                                 | Connections | Name                                                      | Description                                                                                        | Export                                                                                                                              | Clock                          | Base        | End         | IRQ |
|-------------------------------------|-------------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|-------------|-------------|-----|
| <input checked="" type="checkbox"/> |             | <b>pio_0</b><br>clk<br>reset<br>s1<br>external_connection | PIO (Parallel I/O)<br>Clock Input<br>Reset Input<br>Avalon Memory Mapped Slave<br>Conduit Endpoint | <i>Double-click to export</i><br><i>Double-click to export</i><br><i>Double-click to export</i><br><b>pio_0_external_connection</b> | <b>clk_0</b><br>[clk]<br>[clk] | 0x0002_1000 | 0x0002_100f |     |

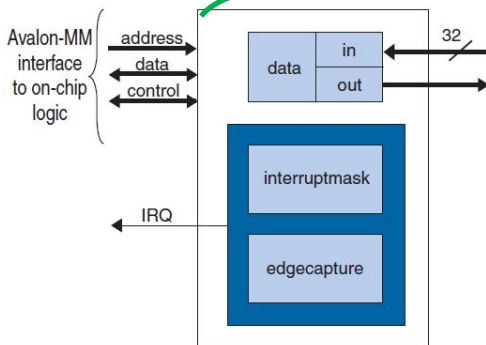


# Parallel IO



PIO IP is a memory mapped avalon slave !

| Use                                 | Connections | Name                | Description                | Export                    | Clock | Base        | End         | IRQ |
|-------------------------------------|-------------|---------------------|----------------------------|---------------------------|-------|-------------|-------------|-----|
| <input checked="" type="checkbox"/> |             | <b>pio_0</b>        | PIO (Parallel I/O)         |                           |       |             |             |     |
|                                     |             | clk                 | Clock Input                | Double-click to export    | clk_0 |             |             |     |
|                                     |             | reset               | Reset Input                | Double-click to export    | [clk] |             |             |     |
|                                     |             | s1                  | Avalon Memory Mapped Slave | Double-click to export    | [clk] |             |             |     |
|                                     |             | external_connection | Conduit Endpoint           | pio_0_external_connection |       | 0x0002_1000 | 0x0002_100f |     |



## Register Map for the PIO Core

| Offset | Register Name |              | R/W | Fields                                                                                                                    |     |   |   |   |
|--------|---------------|--------------|-----|---------------------------------------------------------------------------------------------------------------------------|-----|---|---|---|
|        |               |              |     | (n-1)                                                                                                                     | ... | 2 | 1 | 0 |
| 0      | data          | read access  | R   | Data value currently on PIO inputs.                                                                                       |     |   |   |   |
|        |               | write access | W   | New value to drive on PIO outputs.                                                                                        |     |   |   |   |
| 1      | direction     | (1)          | R/W | Individual direction control for each I/O port. A value of 0 sets the direction to input; 1 sets the direction to output. |     |   |   |   |
| 2      | interruptmask | (1)          | R/W | IRQ enable/disable for each input port. Setting a bit to 1 enables interrupts for the corresponding port.                 |     |   |   |   |
| 3      | edgecapture   | (1), (2)     | R/W | Edge detection for each input port.                                                                                       |     |   |   |   |
| 4      | outset        |              | W   | Specifies which bit of the output port to set.                                                                            |     |   |   |   |
| 5      | outclear      |              | W   | Specifies which output bit to clear.                                                                                      |     |   |   |   |

To set the register, write into the corresponding pointer:

```
volatile int *ptr_pio0=(int *) 0x00021000;
```

# Parallel IO



Why are we using **volatile** ?

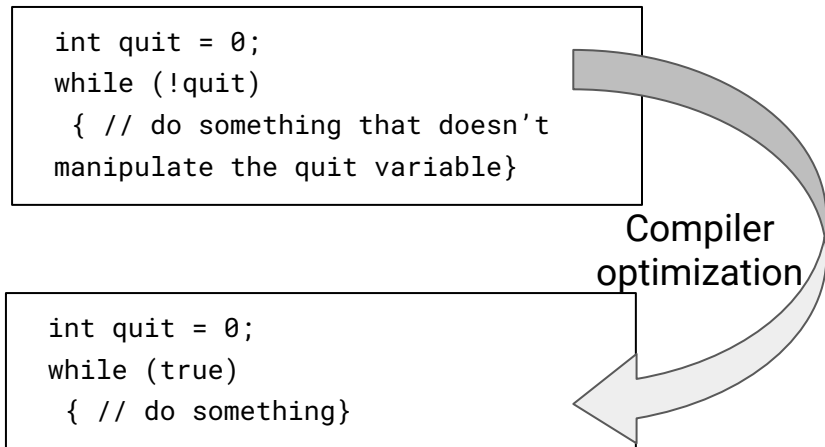
# Parallel IO



## Why are we using **volatile** ?

**Volatile** = No compiling optimization of this variable

The value can be changed by means outside of this code



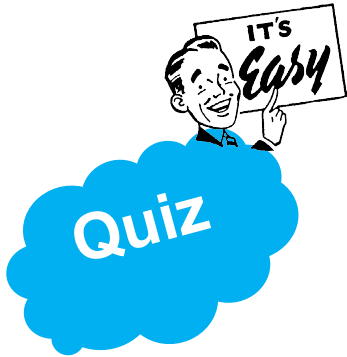
*“volatile in C actually came into existence for the purpose of not caching the values of the variable automatically. It will **tell the compiler not to cache the value of this variable**. So it will generate code to take the value of the given volatile variable from the **main memory every time it encounters it**. This mechanism is used because at any time the **value can be modified by the OS or any interrupt**. So using volatile will help us accessing the value afresh every time.” - Stackoverflow guy, 2019*

# Parallel IO controller



Try to use a minimum number of PIOs.

Ideally **ONE** PIO instance handles **ALL** the system IOs



How can we set the direction of PIO LSB to output?  
How can we set the PIO LSB to 1?

# Nios macros



## Direct assignment: In the userspace

```
#define PIO_0_BASE 0x00000010 // can be found in system.h
volatile int *ptr_pio_data      = (int *) PIO_0_BASE + 0;
volatile int *ptr_pio_direction = (int *) PIO_0_BASE + 1;
*ptr_pio_direction = 0x01;    // set the PIO LSB direction to output
*ptr_pio_data = 0x01;         // set the PIO LSB data to 1
```

| Offset | Register Name        |              | R/W | Fields                                                                                                                    |     |   |   |   |
|--------|----------------------|--------------|-----|---------------------------------------------------------------------------------------------------------------------------|-----|---|---|---|
|        |                      |              |     | (n-1)                                                                                                                     | ... | 2 | 1 | 0 |
| 0      | data                 | read access  | R   | Data value currently on PIO inputs.                                                                                       |     |   |   |   |
|        |                      | write access | W   | New value to drive on PIO outputs.                                                                                        |     |   |   |   |
| 1      | direction (1)        |              | R/W | Individual direction control for each I/O port. A value of 0 sets the direction to input; 1 sets the direction to output. |     |   |   |   |
| 2      | interruptmask (1)    |              | R/W | IRQ enable/disable for each input port. Setting a bit to 1 enables interrupts for the corresponding port.                 |     |   |   |   |
| 3      | edgecapture (1), (2) |              | R/W | Edge detection for each input port.                                                                                       |     |   |   |   |
| 4      | outset               |              | W   | Specifies which bit of the output port to set.                                                                            |     |   |   |   |
| 5      | outclear             |              | W   | Specifies which output bit to clear.                                                                                      |     |   |   |   |



# Better ... Use the hardware abstraction layer



- HAL integrated a group of macros and functions for I/O port access
- Each peripheral usually comes with it's associated HAL macros and function
- These files are copied to your project during BSP generation
- Header file requested:

```
#include "system.h"
```

```
#include "altera_avalon_pio_regs.h"
```

# Access to Parallel I/O port in Nios



## Read and write to I/O Port:

```
IORD_ALTERA_AVALON_PIO_DATA(base)
```

```
IOWR_ALTERA_AVALON_PIO_DATA(base, data)
```

## Set/Get port direction:

```
IORD_ALTERA_AVALON_PIO_DIRECTION(base)
```

```
IOWR_ALTERA_AVALON_PIO_DIRECTION(base, data)
```

## Set/Get interrupt mask:

```
IORD_ALTERA_AVALON_PIO_IRQ_MASK(base)
```

```
IOWR_ALTERA_AVALON_PIO_IRQ_MASK(base, data)
```

# Nios macros



## Direct assignment:

```
#define PIO_ADDR 0x00000010  
volatile int *ptr_pio_data      = (int *) PIO_ADDR + 0;  
volatile int *ptr_pio_direction = (int *) PIO_ADDR + 1;  
*ptr_pio_direction = 0x01;  // set the PIO LSB direction to output  
*ptr_pio_data = 0x01;       // set the PIO LSB data to 1
```

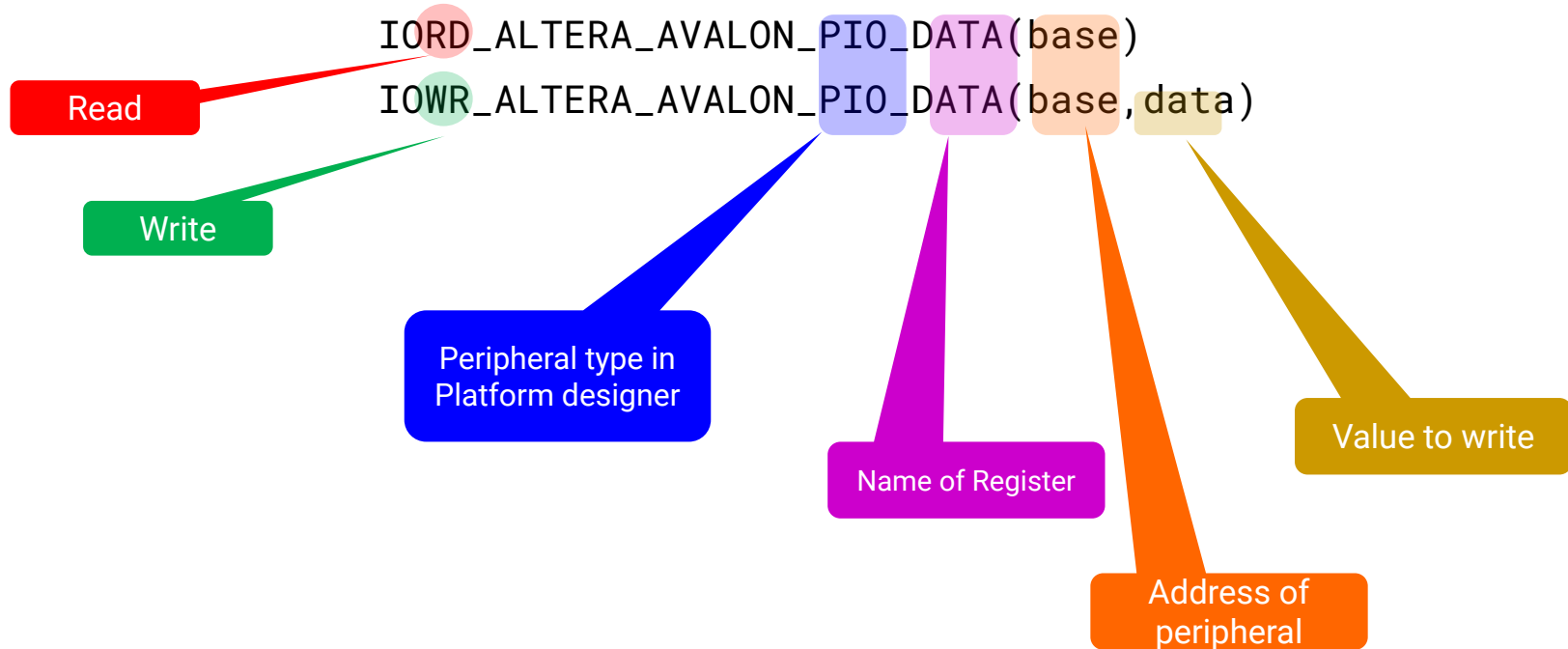
## Equivalent code using HAL Macro will be

```
IOWR_ALTERA_AVALON_PIO_DIRECTION(PIO_BASE, 0x01);  
IOWR_ALTERA_AVALON_PIO_DATA(PIO_BASE, 0xff);
```

## Difference?

- HAL provides a safer way for data transmission
- user doesn't need to be involved into complicated type cast and data alignment.

# A pattern in Nios II macros





# Interrupts

# Interrupts



- A system is much more than the processor:
  - External cards, sensors, keyboards ...
  - Screen events ...
- These devices occasionally need processor service
  - But we can't predict when !
- External events typically occur on a macroscopic timescale
  - We want to keep the processor busy between events

**Need a way for processor to find out devices need attention**



# Interrupts

An event external to the currently executing process that causes a change in the normal flow of instruction execution;

Usually generated by hardware devices external to the processor



**Interrupts are asynchronous with respect to current process**



# Possible Solution: Polling

**CPU periodically checks each device to see if it needs service**

- Takes CPU time even when no requests pending
- Overhead may be reduced at expense of response time
- Can be efficient if events arrive rapidly

“Polling is like picking up your phone every few seconds to see if you have a call. ...”





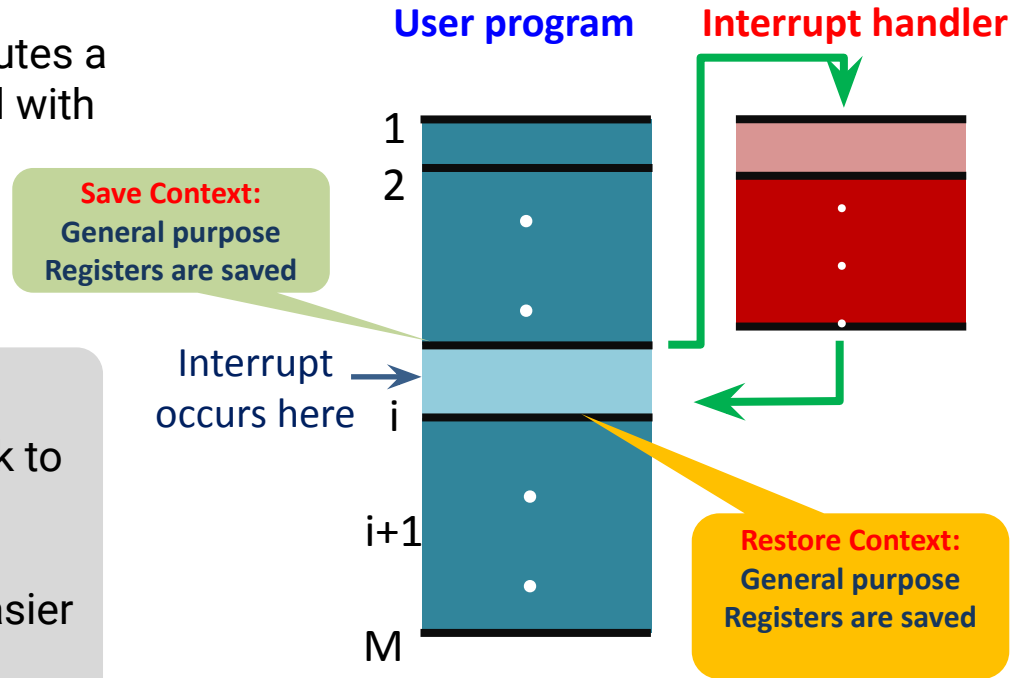
# Interrupts

**Polling alternative: Give each device a wire (interrupt line) that it can use to signal the processor**

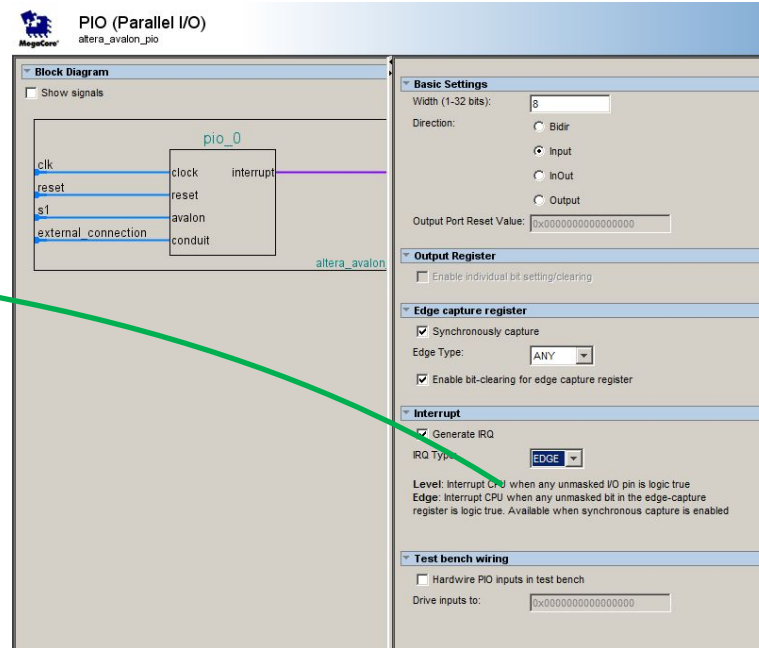
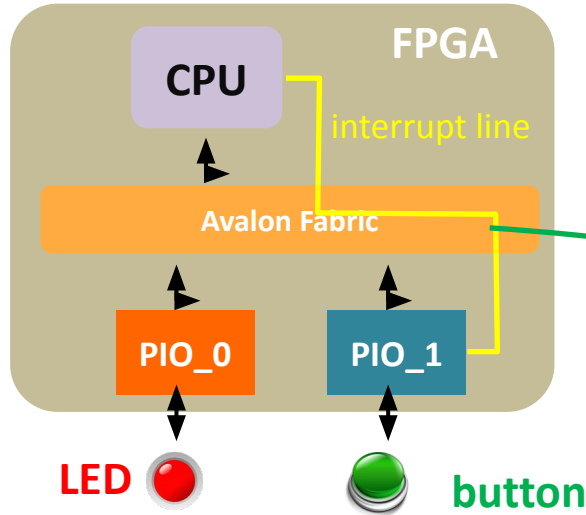
When interrupt signaled, processor executes a routine called an *interrupt handler* to deal with the interrupt

## Polling vs Interrupt:

**Interrupts** win if processor has other work to do and event response time is not critical  
**Polling** can be better if processor has to respond to an event ASAP ( and is way easier to handle ...)



# Interrupt in Nios system





# Interrupt in a Nios system

1. In your C code, you have to initialize the different signals (PIO,...)
2. Enable the corresponding interruption bits
3. Write the Interrupt Sub-Routine (ISR) function
4. Register the ISR to the corresponding Interrupt

ISR registration is done by calling the function:

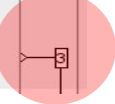
```
int alt_irq_register (alt_u32 id,  
                     void* context,  
                     void (*isr)(void*, alt_u32));
```



# Interrupt in Nios system

```
int alt_irq_register ( alt_u32 id, void* context, void (*isr)(void*, alt_u32));
```

***id*** is the hardware interrupt number for the device, as defined in **system.h**. Interrupt priority corresponds inversely to the IRQ number. Therefore, **IRQ0** represents the **highest** priority interrupt and **IRQ31** is the **lowest**.

|    | End         | IRQ                                                                               |
|----|-------------|-----------------------------------------------------------------------------------|
| 00 | 0x0000_000F |  |

***context*** is a pointer used to pass context-specific information to the ISR, and can point to any ISR-specific information.  
→ NULL pointer 99% of the time ...

The prototype for the **ISR** function is:

```
void isr(void* context, alt_u32 id);
```

***context*** is a pointer to something useful for the associated ISR (pointer on a struct)

***id*** is the irq number



# IRQ Software template

```
#include <stdio.h>
#include "system.h"
#include <sys/alt_irq.h>
#include <io.h>
#include <alt_types.h>
#include "sys/alt_sys_init.h"
static void irqhandler (void * context, alt_u32 id)
{
    // Instructions to handle IRQ goes here !
}

int main()
{
    printf("main() started\n");
    alt_irq_register( IRQSOURCE_IRQ, NULL, (void*)irqhandler ); // register the irq
    for (;;) {} // do something forever
    return 0;
}
```

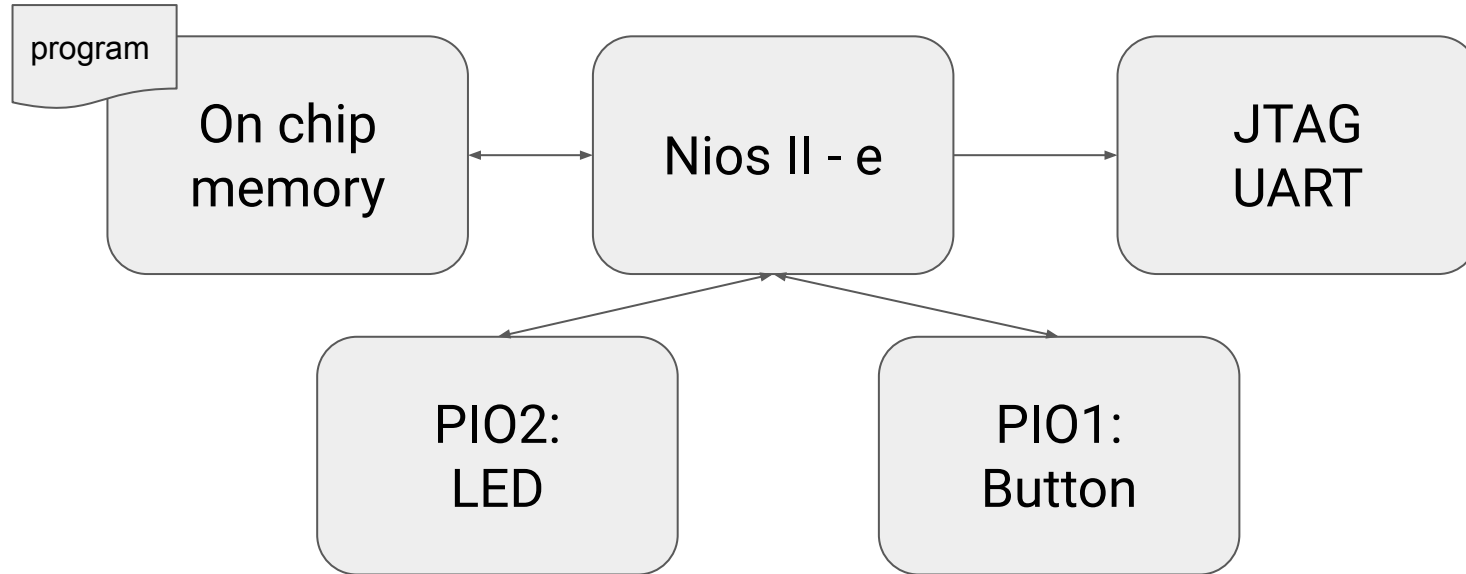


# Clear edge capture of Interrupts

- Once an interrupt is caught / handled by the system, the IRQ master shall deassert the IRQ signal
- For some peripherals, a 0 can be directly assigned to the status register
- Some peripherals have a clear edge capture option

```
static void irqhandler (void * context, alt_u32 id)
{
    // Instructions to handle IRQ goes here !
    // Remember to clear the IRQ once handled !
}
```

# Nios II based SoPC with IOs (Student LAB 1)





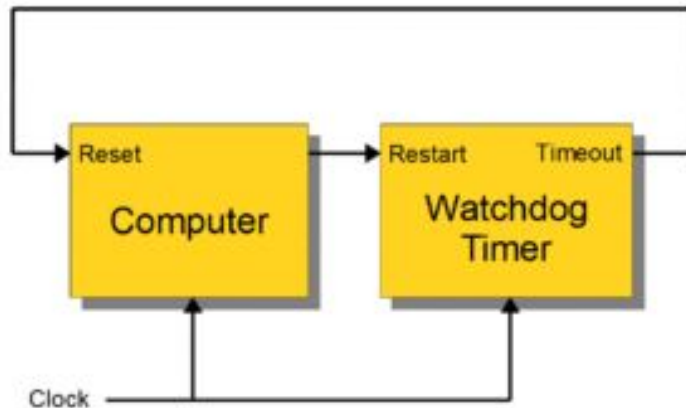
# Timers



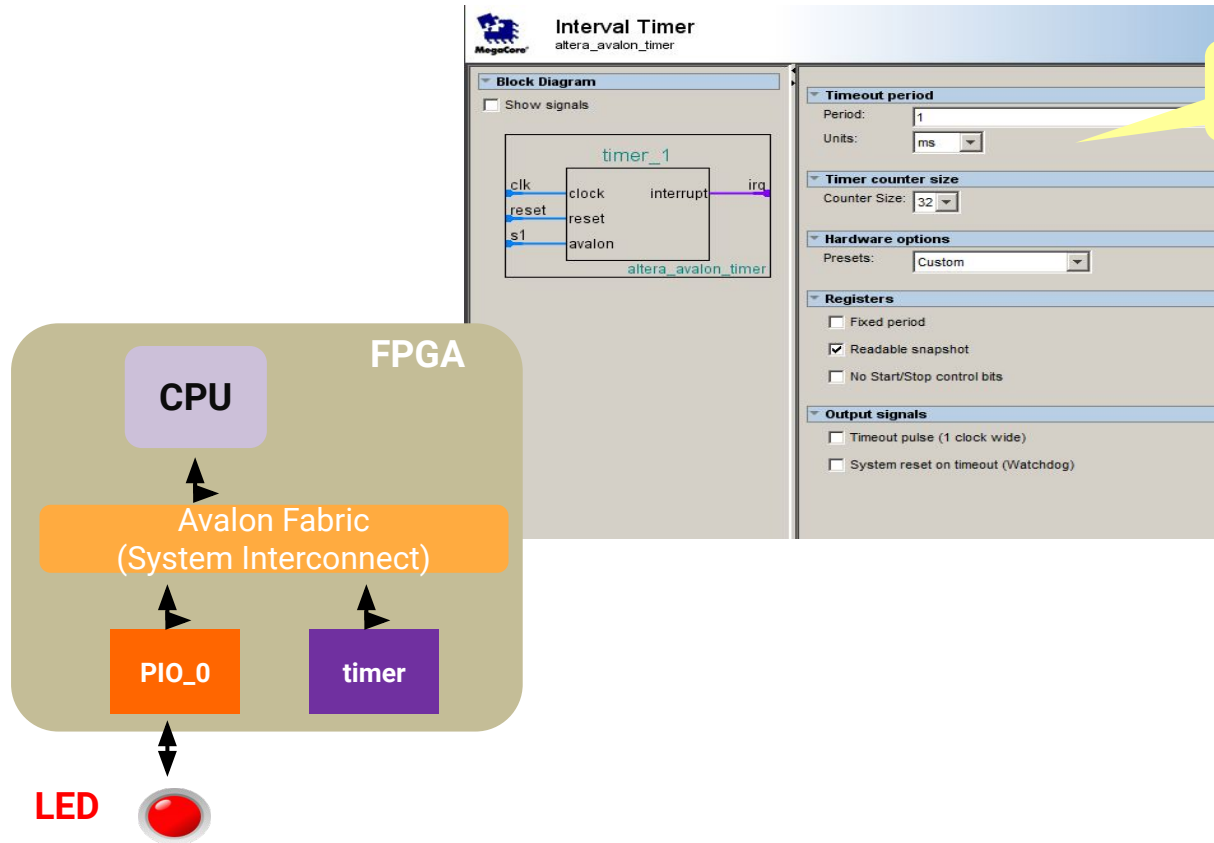
# Timer use-cases



- Watchdog: The timer runs independently from the CPU clock and can trigger interrupts
- Also useful in many cases:
  - Wakeup from sleep
  - Trigger periodic tasks
  - Reset the system if it is stuck in a bad state

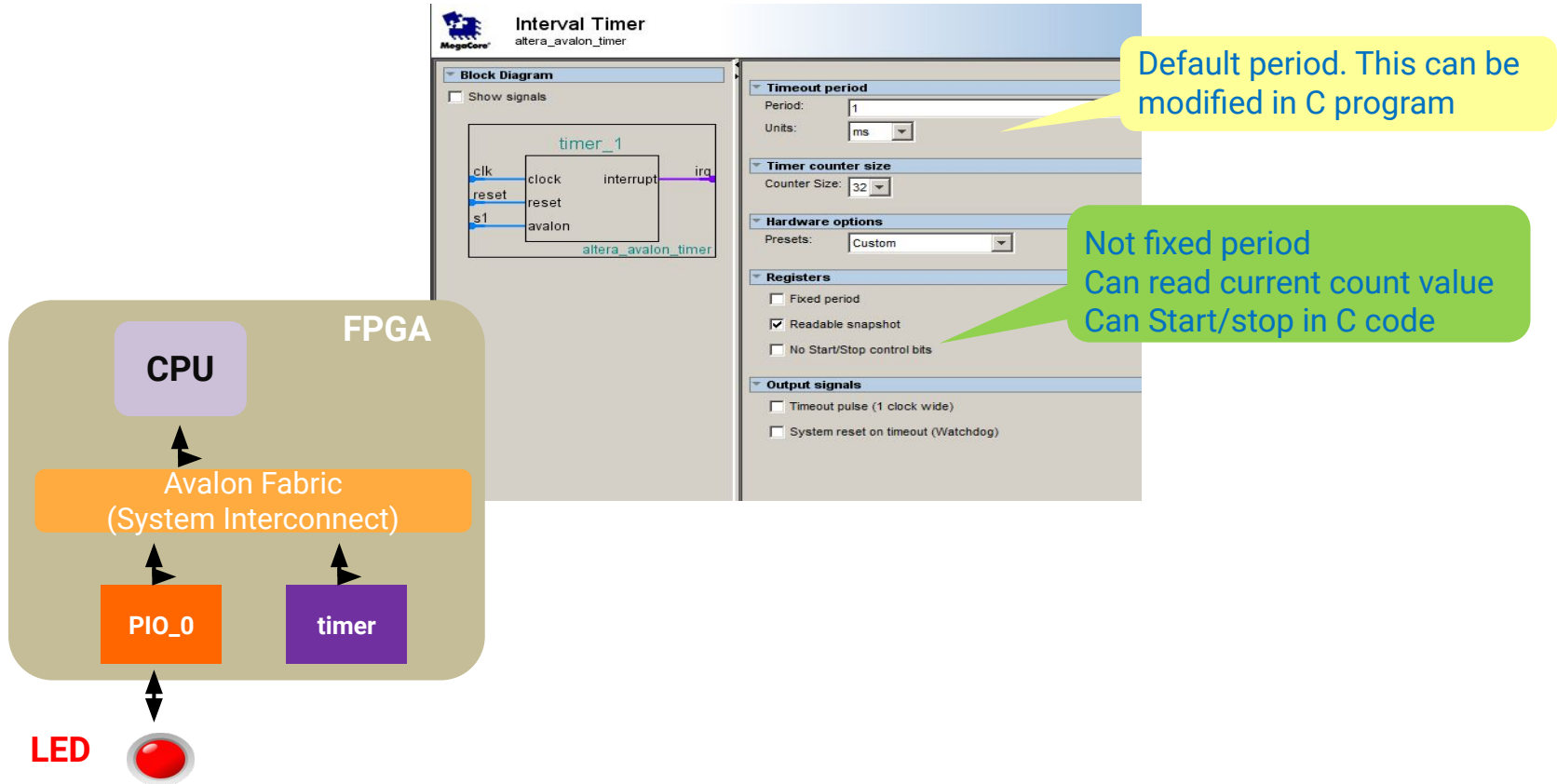


# Timer

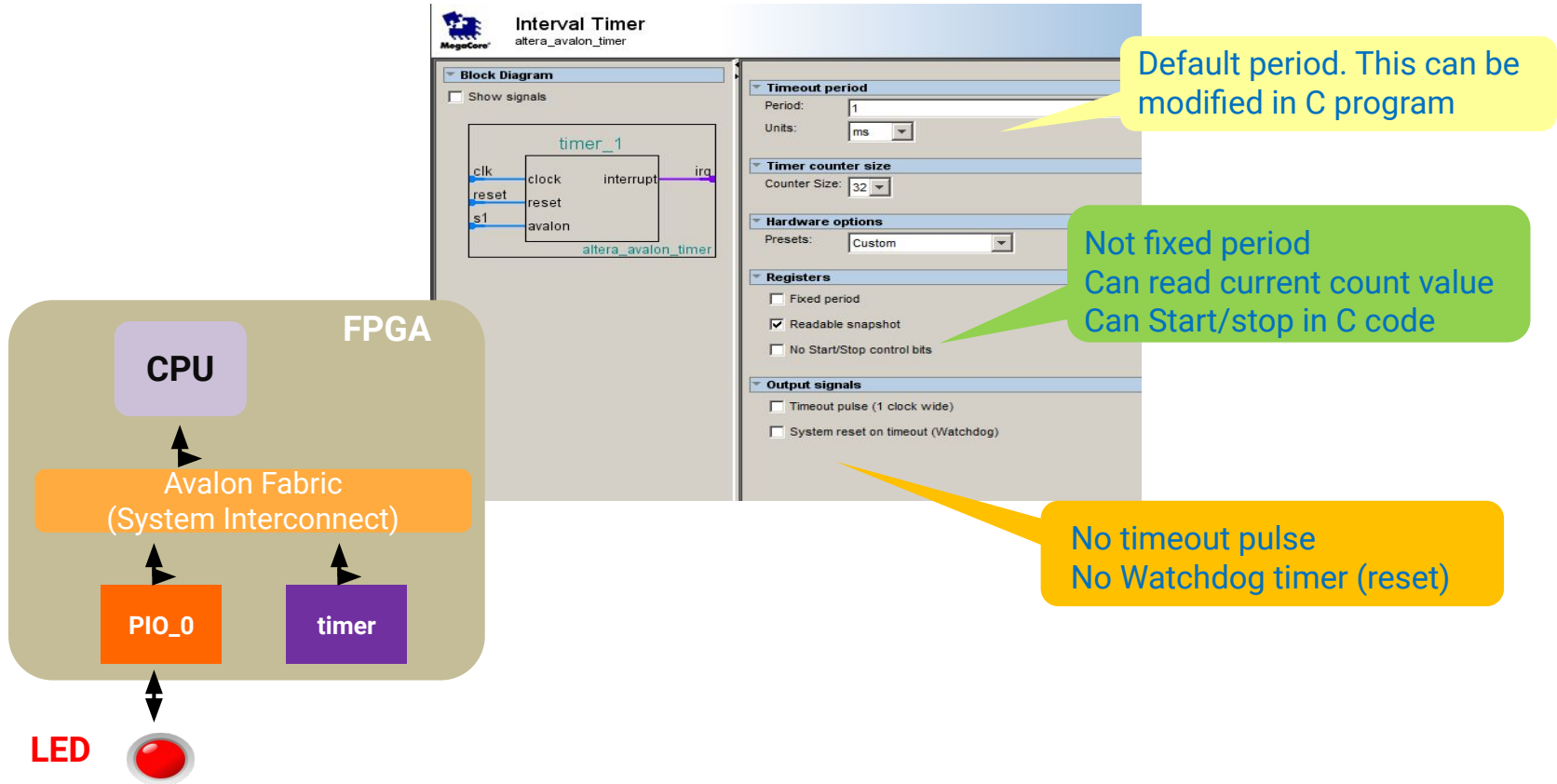


Default period. This can be modified in C program

# Timer

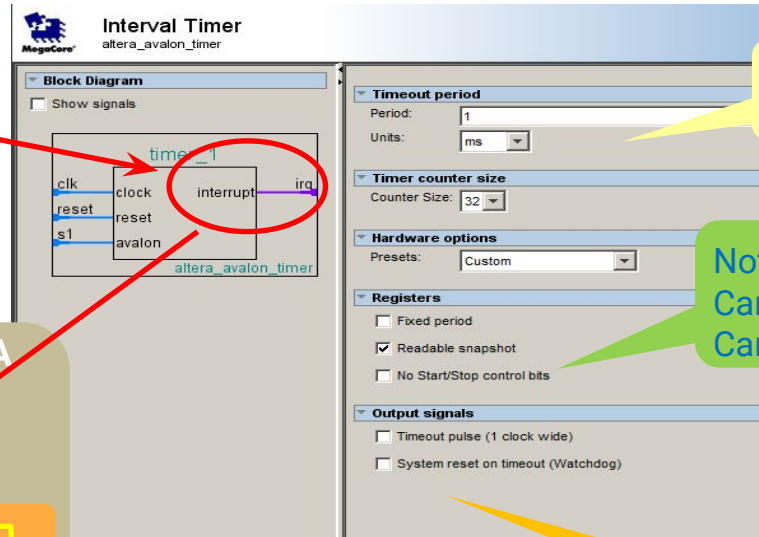
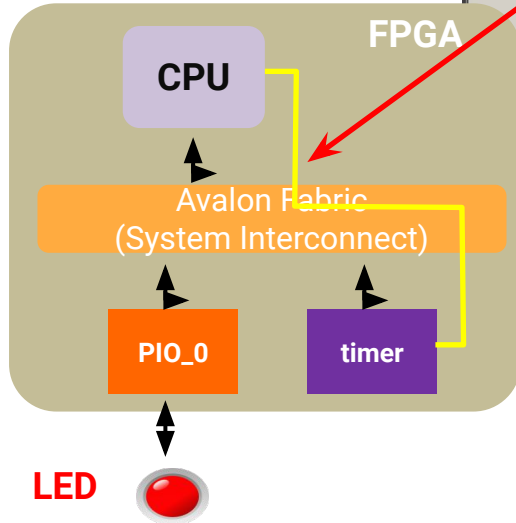


# Timer



# Timer

**Note : Timer can generate an IRQ!**



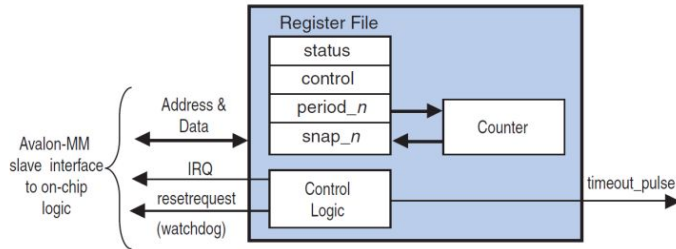
Default period. This can be modified in C program

Not fixed period  
Can read current count value  
Can Start/stop in C code

No timeout pulse  
No Watchdog timer (reset)

# Timer registers

| Connections                                                                       | Name                                                             | Description                                                                                                   | Export                                                                                                | Clock                                              | Base        | End         | IRQ                                                                                 |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------|-------------|-------------|-------------------------------------------------------------------------------------|
|  | <div>timer_0</div> <div>clk</div> <div>reset</div> <div>s1</div> | <div>Interval Timer</div> <div>Clock Input</div> <div>Reset Input</div> <div>Avalon Memory Mapped Slave</div> | <div>Double-click to export</div> <div>Double-click to export</div> <div>Double-click to export</div> | <div>clk_0</div> <div>[clk]</div> <div>[clk]</div> | 0x0002_1000 | 0x0002_101f |  |



| Offset | Name    | R/W | Description of Bits               |     |   |   |      |       |      |     |
|--------|---------|-----|-----------------------------------|-----|---|---|------|-------|------|-----|
|        |         |     | 15                                | ... | 4 | 3 | 2    | 1     | 0    |     |
| 0      | status  | RW  | (1)                               |     |   |   |      |       |      |     |
| 1      | control | RW  | (1)                               |     |   |   | STOP | START | CONT | ITO |
| 2      | periodl | RW  | Timeout Period - 1 (bits [15:0])  |     |   |   |      |       |      |     |
| 3      | periodh | RW  | Timeout Period - 1 (bits [31:16]) |     |   |   |      |       |      |     |
| 4      | snaph   | RW  | Counter Snapshot (bits [15:0])    |     |   |   |      |       |      |     |
| 5      | snaph   | RW  | Counter Snapshot (bits [31:16])   |     |   |   |      |       |      |     |

# Timer Hardware configuration



- 32-bit or 64-bit internal counter
- Starting value loaded from the period register
- Two count modes: countdown once and continuous countdown (CONT in control register)

| Offset | Name    | R/W | Description of Bits               |     |   |      |       |      |     |
|--------|---------|-----|-----------------------------------|-----|---|------|-------|------|-----|
|        |         |     | 15                                | ... | 4 | 3    | 2     | 1    | 0   |
| 0      | status  | RW  | (1)                               |     |   |      |       | RUN  | TO  |
| 1      | control | RW  | (1)                               |     |   | STOP | START | CONT | ITO |
| 2      | periodl | RW  | Timeout Period – 1 (bits [15:0])  |     |   |      |       |      |     |
| 3      | periodh | RW  | Timeout Period – 1 (bits [31:16]) |     |   |      |       |      |     |
| 4      | snaph   | RW  | Counter Snapshot (bits [15:0])    |     |   |      |       |      |     |
| 5      | snaph   | RW  | Counter Snapshot (bits [31:16])   |     |   |      |       |      |     |

# A timer from the Nios perspective

```
#include "system.h"
#include "alt_types.h"
#include "altera_avalon_timer_regs.h"
#include "altera_avalon_timer.h"
#include "sys/alt_irq.h"
#include "sys/alt_sys_init.h"

// Init Timer
IOWR_ALTERA_AVALON_TIMER_PERIOD_0(TIMER_0_BASE, TIMER_PERIOD_0);
IOWR_ALTERA_AVALON_TIMER_PERIOD_1(TIMER_0_BASE, TIMER_PERIOD_1);
IOWR_ALTERA_AVALON_TIMER_CONTROL (TIMER_0_BASE, SOME_MASKS );

// Register IRQ
alt_irq_register(TIMER_0_IRQ, NULL, simple_irq)
```



# Timer Macros

Read/write to 16 bits status register:

- `IORD_ALTERA_AVALON_TIMER_STATUS(base)`
- `IOWR_ALTERA_AVALON_TIMER_STATUS(base, data)`

Read/write to 16 bits control register:

- `IORD_ALTERA_AVALON_TIMER_CONTROL(base)`
- `IOWR_ALTERA_AVALON_TIMER_CONTROL(base, data)`

Read/write to lower 16 bits of period register:

- `IORD_ALTERA_AVALON_TIMER_PERIODL(base)`
- `IOWR_ALTERA_AVALON_TIMER_PERIODL(base, data)`

Read/write to higher 16 bits of period register:

- `IORD_ALTERA_AVALON_TIMER_PERIODH(base)`
- `IOWR_ALTERA_AVALON_TIMER_PERIODH(base, data)`



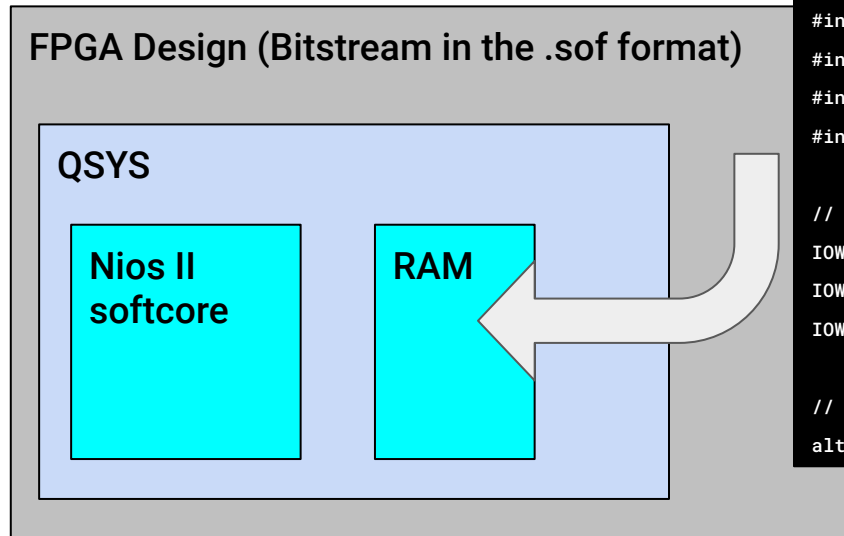
## Co-design project release

1. Program memory initialization
2. Loading a bitstream in flash memory

# Initializing the NiosII memory content



- The NiosII memory content can be initialized to execute your software as soon as the FPGA bitstream is loaded



```
#include "system.h"
#include "alt_types.h"
#include "altera_avalon_timer_regs.h"
#include "altera_avalon_timer.h"
#include "sys/alt_irq.h"
#include "sys/alt_sys_init.h"

// Init Timer
IOWR_ALTERA_AVALON_TIMER_PERIOD_0(TIMER_0_BASE, TIMER_PERIOD_0);
IOWR_ALTERA_AVALON_TIMER_PERIOD_1(TIMER_0_BASE, TIMER_PERIOD_1);
IOWR_ALTERA_AVALON_TIMER_CONTROL (TIMER_0_BASE, SOME_MASKS );

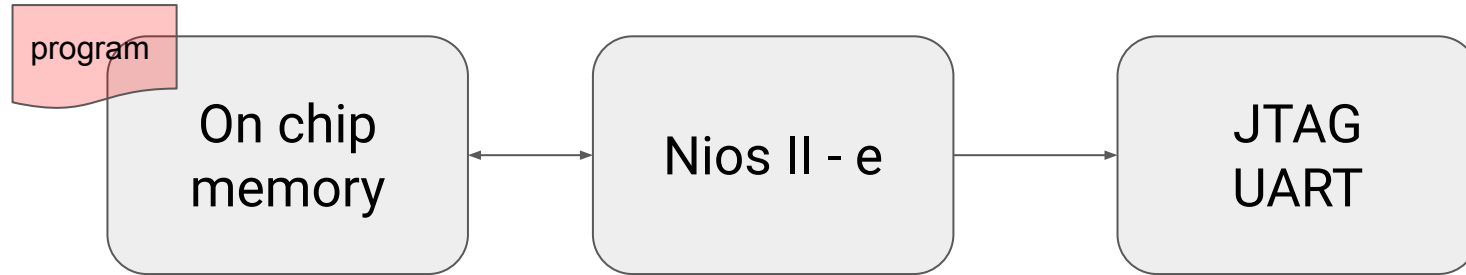
// Register IRQ
alt_irq_register(TIMER_0_IRQ, NULL, simple_irq)
```

# Initializing the NiosII memory content

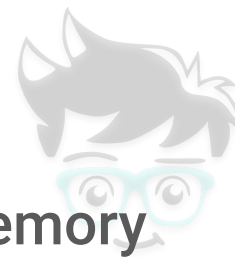


- The NiosII memory content can be initialized to execute your software as soon as the FPGA bitstream is loaded
- Instead of programming your FPGA, then downloading your program, you can directly package your bitstream with the adequate software binary

# Basic Nios-II based SoPC (Exemple lab 1)



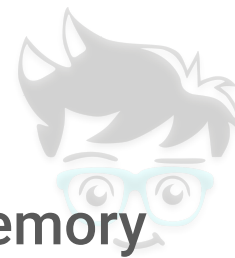
# Initializing the NiosII memory content



1. In your already compiled app directory, generate a memory initialization file using `make mem_init_generate`

```
>> cd software/app
>> make mem_init_generate
...
Info: (main.elf) 9616 Bytes program size (code + initialized data).
Info:          25 KBytes free for stack + heap.
Post-processing to create mem_init/lab1_sys_ram.hex...
elf2hex main.elf 0x00010000 0x00019c3f --width=32 --little-endian-mem --create-lanes=0
mem_init/lab1_sys_ram.hex
Post-processing to create mem_init/hdl_sim/lab1_sys_ram.dat...
elf2dat --infile=main.elf --outfile=mem_init/hdl_sim/lab1_sys_ram.dat \
    --base=0x00010000 --end=0x00019c3f --width=32 \
    --little-endian-mem --create-lanes=0
Post-processing to create mem_init/hdl_sim/lab1_sys_ram.sym...
nios2-elf-nm -n main.elf > mem_init/hdl_sim/lab1_sys_ram.sym
```

# Initializing the NiosII memory content



1. In your already compiled app directory, generate a memory initialization file using `make mem_init_generate`
2. Open your QSYS project and select the generated .hex file as a memory init for your system's RAM.

The screenshot displays the QSYS configuration window for a NiosII system. On the left, a block diagram shows the system components, including a 'ram' block and a 'jtag\_uart\_0' block. The 'ram' block is selected, and its configuration is shown on the right. The 'Memory initialization' section is expanded, showing the following settings:

- ROM/RAM Memory Protection**
  - Reset Request: Enabled
- ECC Parameter**
  - Extend the data width to support ECC bits: Disabled
- Memory initialization**
  - ☒ Initialize memory content
  - ☒ Enable non-default initialization file
  - Type the filename (e.g. my\_ram.hex) or select the hex file using the file browser button.
  - User created initialization file: `/home/kamel/dev/uca/codesign/manips/lab1/software/app/mem_init/lab1_sy`
  - ☐ Enable Partial Reconfiguration Initialization Mode
  - ☐ Enable In-System Memory Content Editor feature
  - Instance ID: `NONE`

At the bottom, a message states: "User is required to provide memory initialization files for memory. Memory will be initialized from /home/kamel/dev/uca/codesign/manips/lab1/software/app/mem\_init/lab1\_sy".

# Initializing the NiosII memory content



1. In your already compiled app directory, generate a memory initialization file using `make mem_init_generate`
2. Open your QSYS project and select the generated .hex file as a memory init for your system's RAM.
3. Generate the QSYS HDL and recompile your design

*PS: On MAX10 devices, enable mem\_init in:*

*Assignments -> Device -> Device and Pin Options -> Configuration -> Configuration Mode: Single uncompressed image with Memory Initialization*



# Initializing the NiosII memory content



1. In your already compiled app directory, generate a memory initialization file using `make mem_init_generate`
2. Open your QSYS project and select the generated `.hex` file as a memory init for your system's RAM.
3. Generate the QSYS HDL and recompile your design
4. Regenerate your BSP

```
nios2-bsp-generate-files --settings ../bsp/settings.bsp  
--bsp-dir ../bsp/
```

# Initializing the NiosII memory content



1. In your already compiled app directory, generate a memory initialization file using `make mem_init_generate`
2. Open your QSYS project and select the generated .hex file as a memory init for your system's RAM.
3. Generate the QSYS HDL and recompile your design
4. Regenerate your BSP
5. Regenerate the memory initialization file using `make mem_init_generate`
6. Update the .hex file: *Processing -> Update memory initialization file*

```
Type ID Message
▶ ⓘ *****
▶ ⓘ Running Quartus Prime MIF/HEX Update
▶ ⓘ Command: quartus_cdb lab1 -c lab1 --update_mif
▶ ⓘ 18236 Number of processors has not been specified which may cause overloading on shared machines. Set the global assignment NUM_PARALLEL_PROCESSORS in your QSF to an appropriate value for best performance.
▶ ⓘ 113015 Width of data items in "lab1_sys_ram.hex" is greater than the memory width. Wrapping data items to subsequent addresses. Found 1250 warnings, reporting 10
▶ ⓘ 39024 Processed the following Memory Initialization File(s)
▶ ⓘ Quartus Prime MIF/HEX Update was successful. 0 errors, 12 warnings
```

# Initializing the NiosII memory content



1. In your already compiled app directory, generate a memory initialization file using `make mem_init_generate`
2. Open your QSYS project and select the generated .hex file as a memory init for your system's RAM.
3. Generate the QSYS HDL and recompile your design
4. Regenerate your BSP
5. Regenerate the memory initialization file using `make mem_init_generate`
6. Update the .hex file: *Processing -> Update memory initialization file*
7. Regenerate your bitstream using: *Processing → Start → Start assembler*

```
Type  ID  Message
1  115031 *****
2  115031 Running Quartus Prime Assembler
3  115031 Command: quartus_asm --read_settings_files=on --write_settings_files=off lab1 -c lab1
4  18236 ⚠ Number of processors has not been specified which may cause overloading on shared machines. Set the global assignment NUM_PARALLEL_PROCESSORS in your QSF to an appropriate value for best performance.
5  115031 Writing out detailed assembly data for power analysis
6  115030 Assembler is generating device programming files
7  115031 Quartus Prime Assembler was successful. 0 errors, 1 warning
```

# Initializing the NiosII memory content

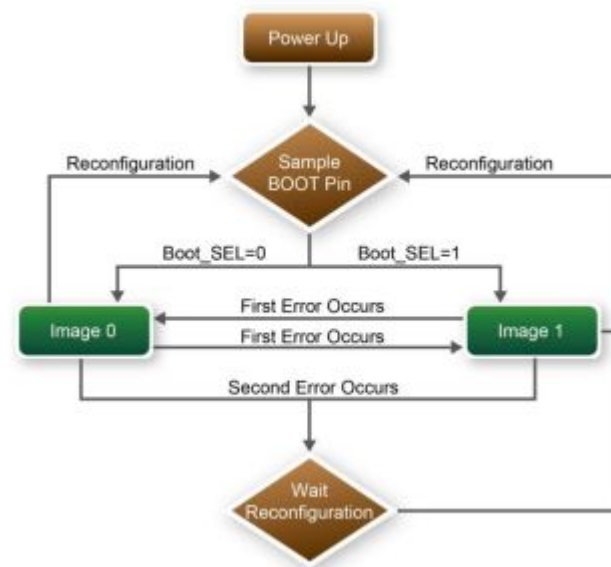


1. In your already compiled app directory, generate a memory initialization file using `make mem_init_generate`
2. Open your QSYS project and select the generated .hex file as a memory init for your system's RAM.
3. Generate the QSYS HDL and recompile your design
4. Regenerate your BSP
5. Regenerate the memory initialization file using `make mem_init_generate`
6. Update the .hex file: *Processing -> Update memory initialization file*
7. Regenerate your bitstream using: *Processing → Start → Start assembler*
8. Program your FPGA with the resulting bitstream, your program should now start automatically (i.e without the `make download-elf` command)

# Loading a bitstream from flash memory



- An FPGA design can be loaded from an on-board flash memory
- In the case of Intel MAX10 targets, an on-device (~5Kbits) flash memory can be used
- This flash can even be splitted into multiple partitions to boot different bitstreams
- In practice, convert you .sof bitstream to a .pof file



# Loading a bitstream from flash memory



Convert Programming File - /home/kamel/dev/uca/codesign/manips/lab1/lab1 - lab1

File Tools Window

Specify the Input files to convert and the type of programming file to generate.  
You can also import input file information from other files and save the conversion setup information created here for future use.

Conversion setup files

Open Conversion Setup Data... Save Conversion Setup...

Output programming file

Programming file type: Programmer Object File (.pof)

Options/Boot Info... Configuration device: EPSE10 Mode: Internal Configuration

File name: output\_files/output\_file.pof

Advanced... Remote/Local update difference file: NONE

☐ Create Memory Map File (Generate output\_file.map)

☐ Create CvP files (Generate output\_file.periph.pof and output\_file.core.rbf)

☐ Create config data RPD (Generate output\_file\_auto.rpd)

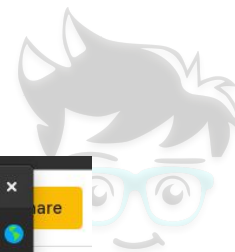
Input files to convert

| File/Data area | Properties    | Start Address |
|----------------|---------------|---------------|
| ▼ SOF Data     | Page_0        | <auto>        |
| lab1.sof       | 10M50DAF484E5 |               |

Add Hex Data  
Add Sof Page  
Add File...  
Remove  
Up  
Down  
Properties

Generate Close Help

# Loading a bitstream from flash memory



Programmer - /home/kamel/dev/uca/codesign/manips/lab1/lab1 - lab1 - [lab1.cdf]

File Edit View Processing Tools Window Help

Hardware Setup... USB-Blaster [3-2.3.1] Mode: JTAG Progress: [ ]

☐ Enable real-time ISP to allow background programming when available

| File                  | Device         | Checksum | Usercode | Program/Configure | Verify | Blank-Check | Examine | Security Bit | Erase | ISP CLAMP |
|-----------------------|----------------|----------|----------|-------------------|--------|-------------|---------|--------------|-------|-----------|
| output_files/lab1.sof | 10M50DAF484... | 0041449E | 0041449E | ✓                 | ■      | ■           | ■       | ■            | ■     | ■         |

Start Stop Auto Detect Delete Add File... Change File... Save File Add Device... Up Down

TDI TDO 10M50DAF484

Select New Programming File

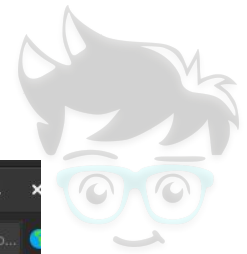
Look In: /home/kamel/dev/uca/codesign/manips/lab1/output\_files

| Name            | Size   | Type     | Date Modified        |
|-----------------|--------|----------|----------------------|
| output_file.pof | 1.4 MB | pof File | 21 Nov 2022 11:22:16 |
| lab1.sof        | 3.1 MB | sof File | 21 Nov 2022 11:01:34 |
| lab1.pof        | 1.4 MB | pof File | 21 Nov 2022 11:01:34 |

File name: lab1.pof Open

Files of type: Programming Files (\*.sof \*.pof \*.jam \*.jbc \*.ekp \*.jic) Cancel

# Loading a bitstream from flash memory



Programmer - /home/kamel/dev/uca/codesign/manips/lab1/lab1 - lab1 - [lab1.cdf]\*

File Edit View Processing Tools Window Help

Hardware Setup... USB-Blaster [3-2.3.1] Mode: JTAG Progress:

☐ Enable real-time ISP to allow background programming when available

| File                  | Device        | Checksum | Usercode | Program/Configure                   | Verify                   | Blank-Check              | Examine                  | Security Bit             | Erase                    | ISP CLAMP                | IPS File | EKP |
|-----------------------|---------------|----------|----------|-------------------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|----------|-----|
| output_files/lab1.pof | 10M50DAF484ES | 058512E7 | 0041449E | <input checked="" type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |          |     |
| CFM0                  |               |          |          | <input checked="" type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |          |     |
| UFM                   |               |          |          | <input checked="" type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |          |     |

Start Stop Auto Detect Delete Add File... Change File... Save File Add Device... Up Down

TDI →  → TDO

10M50DAF484ES



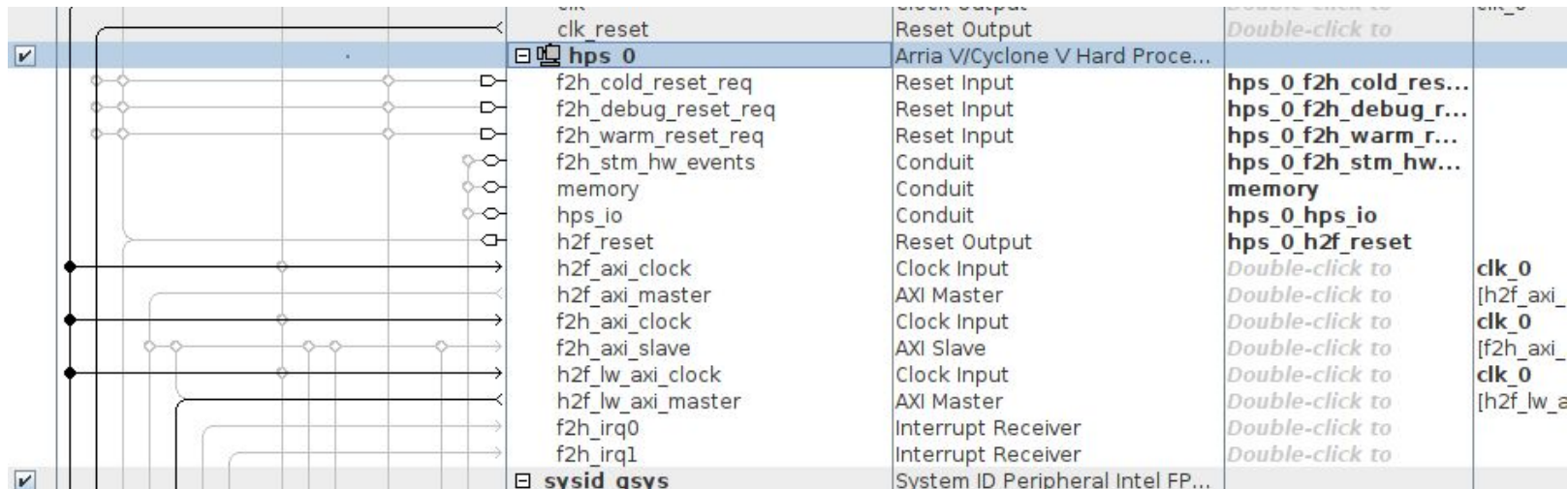


# **Hard Processing System in platform designer: The basics**

# Co-design with HPS based systems



The ARM processor can be imported as an IP from the Intel catalog



# Co-design with HPS based systems



Hard Processor System  
altera\_hps

Documentation

Block Diagram

Show signals

hps\_0

f2h\_cold\_reset\_req reset conduit  
f2h\_debug\_reset\_req reset reset  
f2h\_warm\_reset\_req reset axi  
f2h\_stm\_hw\_events conduit axi  
hps\_jo conduit  
h2f\_axi\_clock clock  
f2h\_axi\_clock clock  
f2h\_axi\_slave axi  
h2f\_fw\_axi\_clock clock  
f2h\_irq0 interrupt  
f2h\_irq1 interrupt

FPGA Interfaces Peripheral Pin Multiplexing HPS Clocks SDRAM

General

- ☐ Enable MPU standby and event signals
- ☐ Enable MPU general purpose signals
- ☐ Enable Debug APB interface
- ☒ Enable System Trace Macrocell hardware events
- ☐ Enable FPGA Cross Trigger Interface
- ☐ Enable FPGA Trace Port Interface Unit
- ☐ Enable boot from fpga signals
- ☐ Enable HL GPI Interface

AXI Bridges

FPGA-to-HPS interface width: 64-bit  
HPS-to-FPGA interface width: 64-bit  
Lightweight HPS-to-FPGA interface width: 32-bit

FPGA-to-HPS SDRAM Interface

Click the '+' and '-' buttons to add and remove FPGA-to-HPS SDRAM ports.

Presets

Project Library

- Altera Arria V SoC Development Kit HPS Presets
- Altera Cyclone V SoC Development Kit HPS Presets

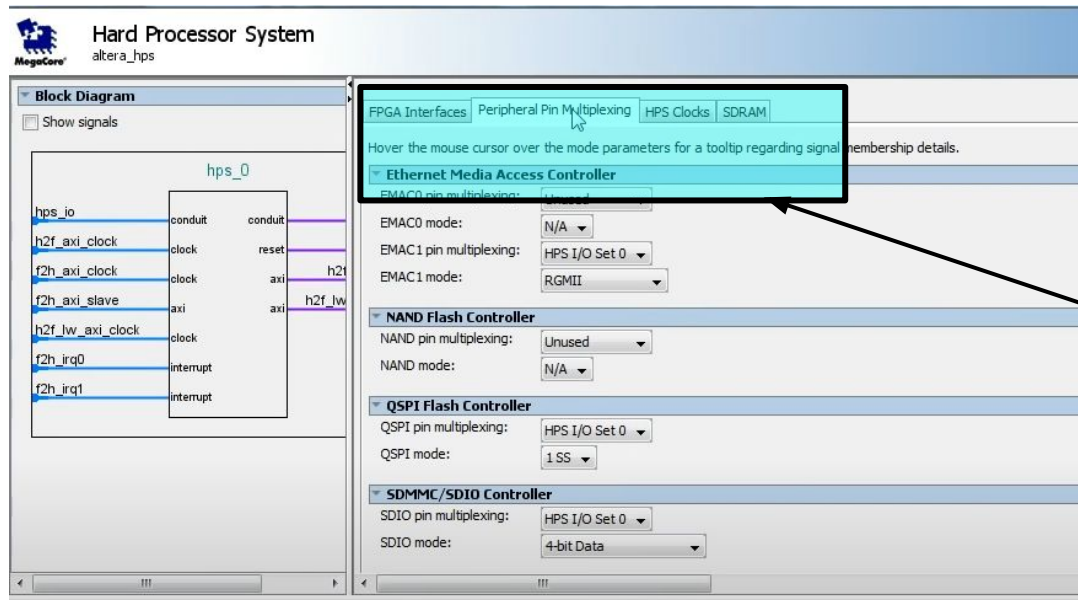
Apply Update... Delete New...

Apply this preset.

Info: hps\_0: ECC will be enabled in the preloader because an interface width of 24 or 40 has been chosen.

Platform designer provides config. preset according to existing devkits

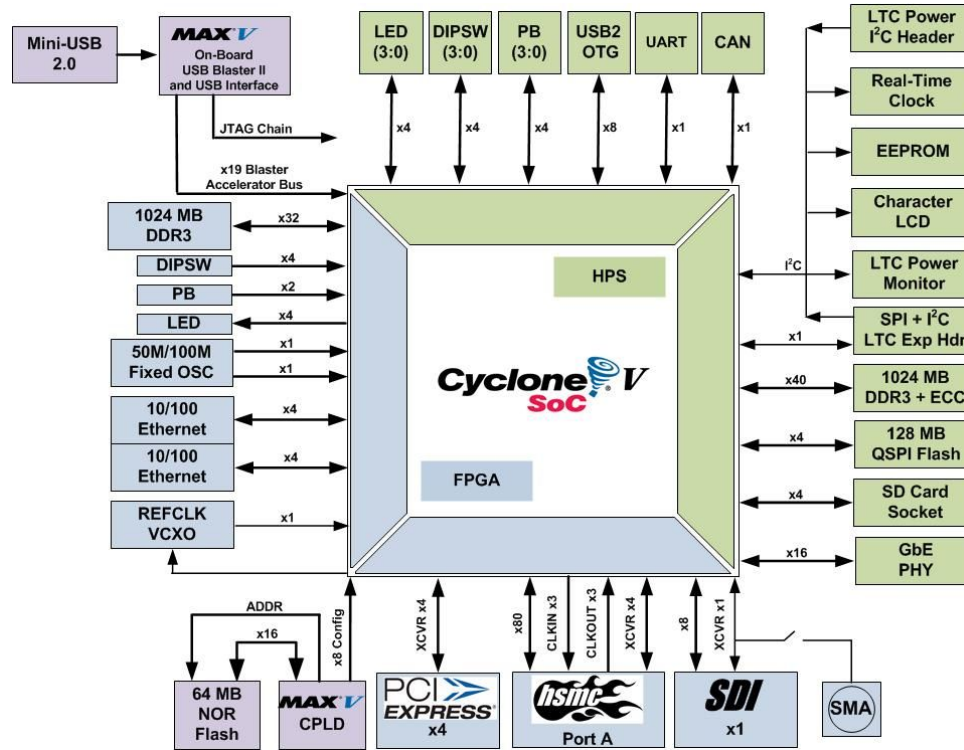
# Co-design with HPS based systems



The SoC pins can be muxed between the FPGA or the HPS. Be sure to select the correct board config.



## Exemple: Rocketboard's Cyclone V SoCKit



# Co-design with HPS based systems



The screenshot displays the 'Hard Processor System' configuration interface. On the left, a 'Block Diagram' shows the 'hps\_0' block with various input and output signals. The main configuration area on the right includes tabs for 'PHY Settings', 'Memory Parameters', 'Memory Timing', and 'Board Settings'. The 'SDRAM Protocol' is set to 'DDR3'. The 'Clocks' section shows the 'Memory clock frequency' set to 400.0 MHz, with a checkbox for 'Use specified frequency instead of calculated frequency'. The 'Advanced PHY Settings' section shows the 'Supply Voltage' set to 1.5V DDR3 and the 'I/O standard' set to SSTL-15. An information bar at the bottom states: 'Info: hps\_0: ECC will be enabled in the preloader because an interface width of 24 or 40 has been chosen.'

In terms of memory, HPS typically uses SDRAM .

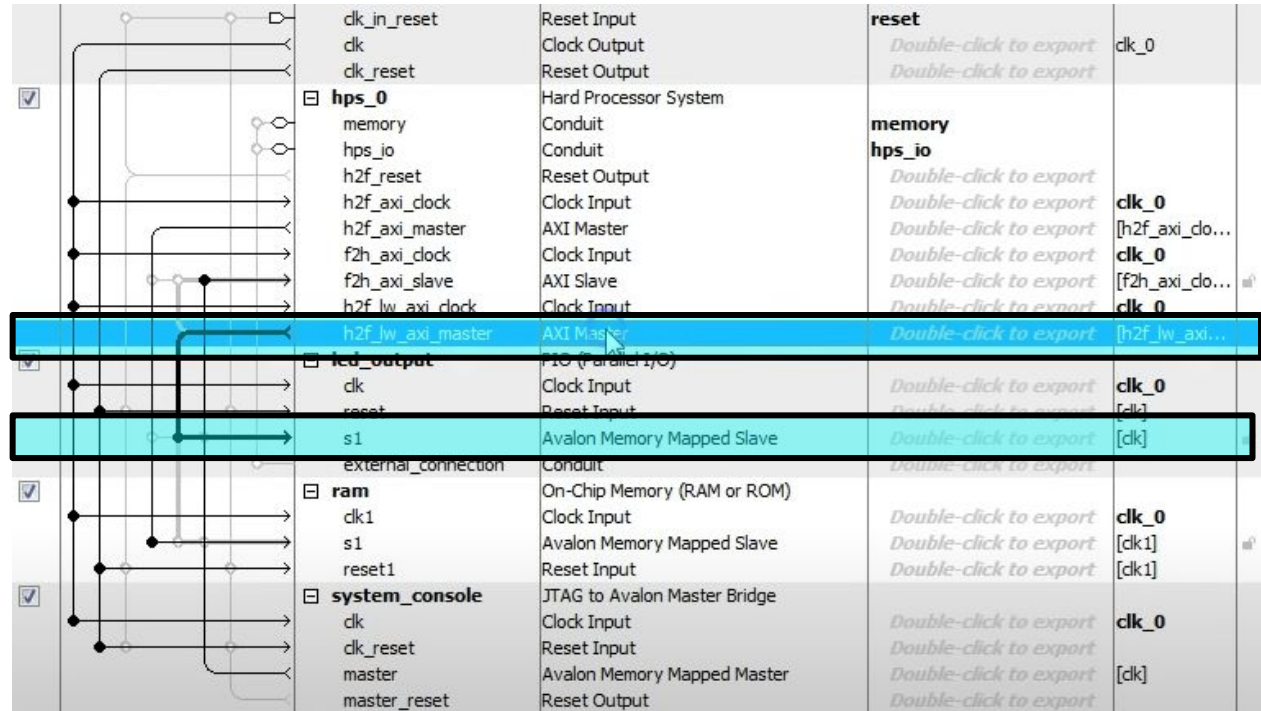
Check the board specs and configure the HPS setup according to the SDRAM Specs !

# Codesign with HPS systems



AXI and Avalon MM ports can be directly wired without resorting to addition bridges.

Platform designer handles that for you !



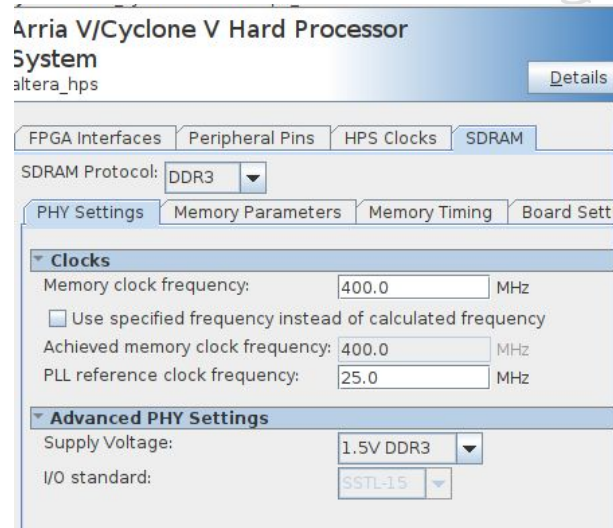
# Co-design with HPS based systems



- The HPS typically uses DDR memory, this is configurable according to the board specs.
- The HPS HAL is generated differently (used to be handled by ARM DS5):

```
sopcinfo2swinfo --input= <filename>.sopcinfo
```

```
swinfo2header --swinfo <filename>.swinfo --output-dir <hps_dir>
```



Remember to include the generated headers when cross-compiling the user code !  
(Linaro is a great ARMv7 cross compiler btw)



# Co-design with HPS based systems



The HPS user code looks like any embedded C codes. 99% of the time, you have linux !

```
int main(){
    // data from SGDMA
    if( ( fd = open( "/dev/mem", ( O_RDWR | O_SYNC ) ) ) == -1 ) {
        printf( "ERROR: could not open \"/dev/mem\"...\n" );
        return( -1 );
    }

    ...

    // Bind to server
    struct sockaddr_in srv_addr;

    ...

    if (connect(sock, (struct sockaddr *) &srv_addr, sizeof(srv_addr)) < 0) {
        printf("ERROR: Unable to connect to server %s:%d\n", SERVERNAME, PORT);
        return 1;
    }

    // Send
    ptr = ( uint32_t )ALT_LWFPGASLVS_OFST + ONCHIP_IMAGE_BASE;

    ...
}
```



## IV. Conclusion



# In conclusion

- As an FPGA engineer, SoPCs are great to design a complex system in low time-to-market constraint.
- General purpose processors are a critical part of any So(P)C, and can be soft or hard, proprietary or open.
- Now you have fundamentals in FPGA-based SoPC design, you can go deeper !

# THANKS FOR YOUR ATTENTION



[www.sma-rty.com](http://www.sma-rty.com)